

Praca Dyplomowa Inżynierska

Dominik Kubicki 210882
Jakub Kindlik 210871

System automatycznego sterowania rakieta oparty na symulacjach komputerowych i algorytmach sztucznej inteligencji

Praca dyplomowa na kierunku
Informatyka

Praca wykonana pod kierunkiem
dra inż. Pawła Hosera
Instytut Informatyki Technicznej
Katedra Sztucznej Inteligencji

Warszawa, rok 2025



SZKOŁA GŁÓWNA
GOSPODARSTWA
WIEJSKIEGO

Wydział Zastosowań
Informatyki
i Matematyki

Oświadczenie Promotora pracy

Oświadczam, że niniejsza praca została przygotowana pod moim kierunkiem i stwierdzam, że spełnia ona warunki do przedstawienia tej pracy w postępowaniu o nadanie tytułu zawodowego.

Data

Podpis promotora

Oświadczenie autora pracy

Świadom/a odpowiedzialności prawnej, w tym odpowiedzialności karnej za złożenie fałszywego oświadczenia, oświadczam, że niniejsza praca dyplomowa została napisana przeze mnie samodzielnie i nie zawiera treści uzyskanych w sposób niezgodny z obowiązującymi przepisami prawa, w szczególności z ustawą z dnia 4 lutego 1994 r. o prawie autorskim i prawach pokrewnych (Dz. U. 2019 poz. 1231 z późn. zm.)

Oświadczam, że przedstawiona praca nie była wcześniej podstawą żadnej procedury związanej z nadaniem dyplomu lub uzyskaniem tytułu zawodowego.

Oświadczam, że niniejsza wersja pracy jest identyczna z załączoną wersją elektroniczną. Przyjmuję do wiadomości, że praca dyplomowa poddana zostanie procedurze antyplagiatowej.

Data

Podpis autora pracy

Streszczenie

System automatycznego sterowania rakieta oparty na symulacjach komputerowych i algorytmach sztucznej inteligencji

Celem pracy jest stworzenie systemu autonomicznego sterowania rakieta, łączącego symulator fizyczny z technikami uczenia maszynowego. Opracowano dwa główne moduły: symulacji dynamiki lotu oraz uczenia maszynowego na bazie algorytmu PPO. Wyniki eksperymentów wykazały skuteczność algorytmu w stabilizacji lotu pionowego oraz ujawniły ograniczenia zastosowanego modelu fizycznego. Praca otwiera perspektywy dalszego rozwoju systemu i testów symulacyjnych.

Słowa kluczowe – rakieta, symulacja, uczenie maszynowe, algorytmy sterujące, PPO

Summary

Automatic rocket control system based on computer simulations and artificial intelligence algorithms

The aim of this thesis is to create an autonomous rocket control system combining a physics simulation engine with machine learning techniques. Two main modules were developed: a rocket flight dynamics simulator and a machine learning system based on the PPO algorithm. Experiments demonstrated the algorithm's ability to maintain stable vertical flight and revealed limitations of the applied physical model. The work opens perspectives for further system development and simulation testing.

Keywords – rocket, simulation, machine learning, control algorithms, PPO

Spis treści

1	Wstęp	11
1.1	Cel, założenia i zakres pracy	11
1.2	Struktura pracy	11
1.3	Użyte technologie	12
1.4	Przegląd dostępnych rozwiązań.	13
1.4.1	System symulacji lotów raketowych.	14
1.4.2	Algorytmy uczenia maszynowego oparte o RL.	16
1.4.3	Podsumowanie.	19
2	Moduł I: System symulacji lotów raketowych.	20
2.1	Założenia projektowe.	20
2.2	Teoretyczne podstawy fizyczne.	21
2.2.1	Opis sił działających na rakietę	22
2.2.2	Równania ruchu postępowego	26
2.2.3	Ruch obrotowy i momenty sił	27
2.2.4	Modelowanie środowiska atmosferycznego	31
2.2.5	Kąt natarcia rakiety	34
2.2.6	Geometria rakiety	35
2.2.7	Obroty układów współrzędnych	38
2.2.8	Model symulacji ruchu rakiety	42
2.2.9	Podsumowanie	43
2.3	Użyte technologie.	43
2.4	Projekt silnika fizycznego.	45
2.4.1	Architektura systemu symulacji	45
2.4.2	Interfejsy	46
2.4.3	Klasy	52
2.4.4	Proces symulacji.	52
2.4.5	Konfiguracja symulacji (Setup)	52

2.4.6	Symulacja	53
2.4.7	Zapisywanie danych symulacji	54
2.4.8	Uzasadnienie przyjętego rozwiązania	56
2.5	Model sił działających na raketę	57
2.5.1	Funkcje pomocnicze	58
2.5.2	Siły działające na raketę	60
2.5.3	Obliczanie zmian w ruchu rakiety	63
2.5.4	Podsumowanie	66
2.6	Moduł wizualizacji symulacji.	66
2.6.1	Okno główne	66
2.6.2	Okno wizualizacji	68
2.7	Dokładność symulacji.	69
2.7.1	Elementy sugerujące dokładność symulacji	70
2.7.2	Kierunki dalszych badań	71
2.8	Ewolucja projektu i wnioski	73
2.8.1	Historia powstania projektu	73
2.8.2	Wnioski	75
3	Moduł II: Algorytm sterujący rakieta.	76
3.1	Założenia i cele projektowe.	76
3.2	Użyte technologie.	76
3.2.1	Biblioteki pomocnicze	77
3.3	Podstawy teoretyczne uczenia maszynowego.	77
3.3.1	Podejścia do uczenia maszynowego.	78
3.3.2	Algorytmy uczenia przez wzmacnianie	79
3.3.3	Modele stosowane w sterowaniu rakieta.	81
3.4	System sterowania lotu rakieta	82
3.4.1	Funkcja aktywacji	83
3.4.2	Inicjalizacja wag	85
3.4.3	Działanie sieci MLP	85
3.5	Algorytm uczący - PPO	87
3.5.1	Charakterystyka teoretyczna	87
3.5.2	Aktualizacja wag sieci MLP	87

3.5.3	Podsumowanie	89
3.6	Architektura i implementacja modelu uczącego	89
3.6.1	Architektura systemu uczącego	89
3.6.2	Model uczący	92
3.6.3	Proces uczenia	93
3.6.4	Podsumowanie	94
3.7	Wyniki i wnioski.	94
3.7.1	Cel działania algorytmu	94
3.7.2	Metodyka pomiaru skuteczności	95
3.7.3	Wyniki	95
3.7.4	Wnioski	95
3.7.5	Podsumowanie	98
4	Podsumowanie	99
4.1	Podsumowanie	99
4.2	Wnioski końcowe	99
4.3	Perspektywy rozwoju i dalsze badania	100

1 Wstęp

1.1 Cel, założenia i zakres pracy

Celem niniejszej pracy inżynierskiej jest stworzenie systemu sterującego, opartego na algorytmach samouczących, zdolnego do przeprowadzenia stabilnego, pionowego lotu rakiety o naturalnie niestabilnej charakterystyce. W celu spełnienia tych założeń konieczne było stworzenie środowiska, pozwalającego na wielokrotne prowadzenie symulacji lotu rakiety i trenowanie algorytmu sterującego. Stworzenie silnika fizycznego spełniającego te wymagania było pośrednim celem niniejszej pracy.

W celu osiągnięcia założonego celu, poczynione zostały pewne założenia, które precyzują zakres pracy. Są to założenia szczegółowo opisane w rozdziałach 2.1 oraz 3.1. Zakres pracy obejmuje stworzenie silnika fizycznego symulacji lotów raketowych jako pierwszego modułu projektu, a także stworzenie i wytrenowanie modelu sterującego rakiety jako drugiego modułu projektu. Oba te moduły są ze sobą ściśle związane, a ich integracja jest kluczowa dla powodzenia projektu i stanowi, kolejny z elementów zakresu pracy.

Praca ta skupia się wyłącznie na stworzeniu symulacji lotów, a jej projektowa część jest związana z informatyczną i programistyczną sferą zagadnienia. Do zakresu niniejszej pracy inżynierskiej nie należą więc tematy związane z konstrukcją fizycznych raket ani z budową sprzętu do pomiarów lotów raketowych czy testowanie skuteczności algorytmów sterujących w lotach fizycznych raket.

1.2 Struktura pracy

Niniejsza praca inżynierska powstała w wyniku kolaboracji dwóch współtwórców, dlatego też główna jej część została podzielona na dwa odrębne moduły:

- Moduł I: System symulacji lotów raketowych. Autor: Jakub Kindlik;
- Moduł II: Algorytm sterujący rakiety. Autor: Dominik Kubicki;

Główne moduły pracy są ze sobą tak ściśle związane, że aby zachować spójność, prace inżynierskie dwóch autorów powstały jako jeden dokument. Dzięki temu możliwe jest utrzymanie ciągu logicznego, prowadzącego od powstania silnika fizycznego

przez trenowanie na nim modelu sterującego aż do integracji obu systemów i wniosków końcowych.

Każdy z modułów jest pełną pracą inżynierską samą w sobie, zawierając podstawy teoretyczne, przedstawienie wykorzystanych technologii, opis i dekonstrukcję stworzonego rozwiązania, wnioski, wyniki i opis procesu powstawania.

Łącznie oba projekty-moduły tworzą kompletny system sterujący rakietą, wytrenowany na silniku fizycznym, co spełnia założony cel pracy.

Pozostałe rozdziały pracy poświęcone są opisowi kwestii dotyczących obu modułów pracy inżynierskiej, takich jak między innymi wstęp, integracja obu części projektu, końcowe wnioski wyniki oraz podsumowanie całości rozwiązania pracy. Te części niniejszej pracy inżynierskiej są wspólne dla obu modułów i zostały napisane wspólnie przez obu autorów.

1.3 Użyte technologie

Poszczególne biblioteki i narzędzia wykorzystane przy tworzeniu każdego z modułów pracy zostały przedstawione i opisane odpowiednich rozdziałach pracy, to jest odpowiednio w rozdziale 2.3 w module I, oraz w rozdziale 3.2 w module II. Jednakże niektóre technologie oraz narzędzia zostały wykorzystane przy tworzeniu obu modułów projektu, wchodzących w skład niniejszej pracy inżynierskiej.

- **Git [4] oraz Github [13]** - narzędzia wykorzystane do kontroli wersji projektu.

Umożliwiły one wspólną pracę nad kodem projektu oraz dostęp do archiwalnych wersji projektu. Git oraz Github zostały wybrane ze względu na ich popularność, oraz wsparcie dla wielu języków, a także na znajomość tych narzędzi przez autorów pracy.

- **Python 3 [9]** - język programowania, w którym napisane zostały obydwa moduły projektu.

Python został wybrany jako język programowania w projekcie ze względu na jego prostotę oraz znajomość tego języka przez autorów pracy. Ważnym czynnikiem wpływającym na wybór tego języka jest również popularność Pythona w dziedzinie uczenia maszynowego oraz sztucznej inteligencji. Dzięki wykorzystaniu tego języka do obu modułów projektu ich integracja była znacznie prostsza.

- **PyCharm Professional [15]** - środowisko programistyczne wykorzystane do tworzenia kodu projektu.

PyCharm został wybrany ze względu na dedykowane wsparcie dla języka Python, w którym napisany został projekt, oraz na znajomość tego narzędzia przez autorów pracy.

Ponadto, do tworzenia pracy inżynierskiej wykorzystane zostały następujące technologie umożliwiające stworzenie i edycję niniejszego dokumentu:

- **LaTeX [19]** - system składu tekstu wykorzystany do napisania niniejszego dokumentu.

LaTeX został wybrany ze względu na jego popularność wśród naukowców oraz inżynierów, prostotę tworzenia pracy inżynierskiej w tym narzędziu oraz znajomość tego narzędzia przez autorów pracy.

- **SGGW-thesis [5]** - biblioteka do tworzenia pracy inżynierskiej w zgodzie z wymaganiami formatowania pracy inżynierskiej na SGGW.

Biblioteka ta została wybrana ze względu na jej prostotę użycia oraz zgodność z wymaganiami formatowania pracy inżynierskiej na SGGW.

Zastosowanie tych narzędzi pozwoliło na efektywną pracę nad projektem oraz na stworzenie tej pracy inżynierskiej. Narzędzia te są szeroko dostępne i większości darmowe, co pozwoliło na ich wykorzystanie w ramach pracy inżynierskiej.

1.4 Przegląd dostępnych rozwiązań.

Dziedzina hobbystycznej symulacji lotów rakietowych, choć jest niszowa w kontekście informatyki, inżynierii i fizyki to posiada spore grono entuzjastów oraz specjalistów, którzy przez lata rozwijali narzędzia i algorytmy umożliwiające symulację takich lotów. Istnieje również duży rynek komercyjnych narzędzi stosowanych przez przedsiębiorstwa oraz wojsko do symulacji lotów rakietowych. Systemy te charakteryzują się wysoką dokładnością, złożonością i są zazwyczaj niedostępne dla hobbystów oraz użytkowników prywatnych. W związku z tym, w niniejszym rozdziale opisane zostały jedynie rozwiązania ogólnodostępne. Poniżej przedstawiono narzędzia i technologie, które umożliwiają symulację lotów rakietowych, a także gotowe silniki fizyczne oraz algorytmy uczenia maszynowego, mogące zostać zastosowane w tym kontekście.

Znacznie bardziej popularna dziedzina uczenia maszynowego, oparta na Reinfor-

Reinforcement Learning (RL), oferuje liczne rozwiązania, które mogą zostać wykorzystane do trenowania modeli sztucznej inteligencji odpowiedzialnych za sterowanie lotem rakiety. W tym rozdziale przedstawione zostały najczęściej wykorzystywane z nich — zarówno biblioteki służące do trenowania modeli, jak i narzędzia umożliwiające przeprowadzanie treningów na symulacjach.

Dobra znajomość dostępnych na rynku rozwiązań pozwala na wybór najlepszego narzędzia do konkretnego projektu, a także na zrozumienie specyfiki symulacji lotów raketowych oraz algorytmów sterujących, co jest kluczowe dla skutecznego rozwoju systemów sterowania raketami. Dzięki zapoznaniu się z obecnymi rozwiązaniami możliwe było stworzenie własnego, dedykowanego narzędzia do symulacji lotów raketowych oraz algorytmu sterującego.

1.4.1 System symulacji lotów raketowych.

Na rynku dostępnych jest wiele modeli silników fizycznych umożliwiających symulację lotów raketowych, jak i narzędzi umożliwiających tworzenie własnych symulacji. Poniżej zaprezentowane zostały najpopularniejsze z nich.

Narzędzia służące do ogólnego modelowania symulacji fizycznych:

- **Symulink [17].**

Bardzo zaawansowane narzędzie do modelowania i prowadzenia symulacji systemów dynamicznych. Symulink wchodzi w skład pakietu Matlab. System ten umożliwia elastyczne i dokładne modelowanie sił. Narzędzie to bazuje na krokowych modelach składania sił, co pozwala na bardzo dokładne modelowanie sił działających na obiekt. Symulink jest narzędziem płatnym oraz wymagającym dużej wiedzy z zakresu modelowania systemów dynamicznych.

Symulink może zostać wykorzystany do stworzenia bardzo dokładnego modelu symulacji lotu rakiety poprzez zamodelowanie sił działających na obiekt i symulację ich działania w czasie.

- **ANSYS Fluent [3].**

Zaawansowane narzędzie do symulacji dynamiki płynów (CFD – Computational Fluid Dynamics) wykorzystywane w inżynierii aerodynamicznej i mechanicznej. ANSYS Fluent umożliwia modelowanie przepływów płynów, wymiany ciepła, oraz oddziaływania sił aerodynamicznych na obiekty w różnych warunkach środowiskowych. ANSYS Fluent bazuje na rozwiązaniach równań

Naviera-Stokesa, co pozwala na dokładne odwzorowanie zjawisk takich jak opór aerodynamiczny, turbulencje oraz efekty cieplne. Dzięki zaawansowanym algorytmom i funkcjom, Fluent umożliwia analizę złożonych scenariuszy, takich jak przepływ wokół wieloelementowych konstrukcji czy interakcje z gazami spalinowymi. Oprogramowanie to jest komercyjne, a jego obsługa wymaga wiedzy z zakresu CFD oraz doświadczenia w konfiguracji symulacji. ANSYS Fluent może być wykorzystany do szczegółowego modelowania lotu rakiety poprzez analizę przepływu powietrza wokół korpusu rakiety, co pozwala na optymalizację jej konstrukcji i poprawę stabilności aerodynamicznej.

- **OpenFOAM [11].**

Otwartoźródłowe oprogramowanie do symulacji CFD, które oferuje zaawansowane możliwości modelowania przepływów płynów, wymiany ciepła oraz innych procesów fizycznych. OpenFOAM opiera się na podejściu siatkowym (mesh-based), co pozwala na precyzyjne odwzorowanie geometrii i rozwiązywanie równań Naviera-Stokesa w ramach analiz aerodynamicznych. Główną zaletą OpenFOAM jest jego elastyczność i dostępność – oprogramowanie jest darmowe i można je dostosować do indywidualnych potrzeb poprzez tworzenie własnych modułów i skryptów. Jednak jego obsługa wymaga zaawansowanej wiedzy technicznej oraz umiejętności pracy w środowisku tekstowym, co może być barierą dla początkujących użytkowników[10].

OpenFOAM może zostać wykorzystany do symulacji lotu rakiety poprzez analizę sił aerodynamicznych oraz optymalizację kształtu i konstrukcji rakiety w celu minimalizacji oporu i poprawy wydajności lotu.

Dedykowane środowiska i systemy, służące do symulowania lotów raketowych:

- **OpenRocket [26].**

Bezpłatne i otwartoźródłowe narzędzie stworzone specjalnie do symulacji lotów raket. OpenRocket jest łatwe w obsłudze, dzięki intuicyjnemu interfejsowi graficznemu, który umożliwia szybkie projektowanie raket oraz przeprowadzanie symulacji ich lotu. Program oferuje możliwość modelowania sił aerodynamicznych, takich jak opór powietrza, siła nośna oraz działanie silników raketowych w różnych warunkach. OpenRocket obsługuje różne typy silników, pozwala na analizę stabilności rakiety, trajektorii lotu oraz przewidywanie maksymalnej wysokości i zasięgu. Narzędzie to jest szczególnie popularne wśród hobbystów oraz studentów, dzięki swojej prostocie i dostępności, choć może mieć ograni-

czenia w przypadku bardziej zaawansowanych symulacji.

Narzędzie to było też w dużym stopniu inspiracją niniejszej pracy inżynierskiej[20].

- **RockSim [2].**

Komercyjne oprogramowanie do projektowania i symulacji lotów raket, dedykowane zarówno dla entuzjastów, jak i profesjonalistów. RockSim umożliwia projektowanie rakiet od podstaw, uwzględniając szczegółowe parametry takie jak geometria, masa komponentów czy rozmieszczenie środka ciężkości i ciśnienia. Program oferuje zaawansowane funkcje symulacji aerodynamicznych, takich jak analiza stabilności, przewidywanie trajektorii lotu oraz obliczanie wpływu warunków atmosferycznych (wiatr, temperatura) na zachowanie rakiety. Dzięki swojej wszechstronności, RockSim jest używany zarówno w edukacji, jak i w projektach zaawansowanych konstrukcji raketowych.

- **RAS Aero [23].**

RAS Aero to zaawansowane narzędzie do symulacji lotów raket dedykowane użytkownikom, którzy potrzebują dużej dokładności w analizie aerodynamicznej. Program oferuje możliwość modelowania trajektorii z uwzględnieniem szczegółowych parametrów aerodynamicznych, takich jak współczynniki oporu czy siły nośnej, oraz wpływu zmiennych warunków atmosferycznych na raketę. RAS Aero jest szczególnie cenione za swoją zdolność do przewidywania zachowania rakiet przy dużych prędkościach oraz podczas przejścia przez bariery dźwiękowe. Oprogramowanie to jest płatne i wymaga większego doświadczenia w pracy z danymi aerodynamicznymi, ale oferuje bardzo dokładne wyniki symulacji.

1.4.2 Algorytmy uczenia maszynowego oparte o RL.

W dziedzinie algorytmów uczenia maszynowego opartych o Reinforcement Learning istnieje wiele narzędzi, jak i gotowych programów, które mogą być wykorzystane do uczenia modeli na symulacjach fizycznych.

Najpopularniejsze narzędzia stosowane do uczenia maszynowego RL w symulacjach fizycznych to m.in.:

- **TensorFlow [1].**

TensorFlow to popularna platforma open-source'owa do uczenia maszynowego, która znalazła szerokie zastosowanie w symulacjach fizycznych, w tym w dzie-

dzinie reinforcement learning (RL). Dzięki wsparciu dla zaawansowanych algorytmów optymalizacji, TensorFlow umożliwia trenowanie modeli RL na danych pochodzących z symulacji fizycznych. Framework ten oferuje bogatą funkcjonalność do tworzenia i trenowania sieci neuronowych, a także integrację z innymi narzędziami do analizy danych i wizualizacji. TensorFlow wspiera różnorodne techniki optymalizacji, w tym metody oparte na gradientach, co pozwala na skuteczne rozwiązywanie problemów związanych z dynamiką fizyczną. Jego rozbudowana dokumentacja oraz wsparcie dla rozwoju modeli na dużą skalę czynią go jednym z najczęściej wykorzystywanych narzędzi w branży.

- **PyTorch [21].**

PyTorch to kolejny popularny framework open-source'owy, który jest szczególnie ceniony wśród badaczy i inżynierów zajmujących się uczeniem maszynowym. Dzięki swojej elastyczności oraz wsparciu dla dynamicznych grafów obliczeniowych, PyTorch doskonale nadaje się do tworzenia i trenowania modeli reinforcement learning w kontekście symulacji fizycznych. PyTorch jest często wykorzystywany w projektach związanych z robotyką oraz modelowaniem dynamicznych systemów, takich jak rakiety, ponieważ pozwala na łatwą integrację z fizycznymi symulacjami i oferuje szeroki zakres algorytmów RL. PyTorch charakteryzuje się także rozbudowaną społecznością oraz licznymi zasobami edukacyjnymi, co czyni go atrakcyjnym wyborem dla osób rozpoczynających pracę z uczeniem maszynowym.

- **Stable Baselines3 [22].**

Stable Baselines3 to biblioteka open-source'owa zbudowana na bazie PyTorch, która oferuje gotowe implementacje popularnych algorytmów reinforcement learning, takich jak PPO (Proximal Policy Optimization), A2C (Advantage Actor-Critic) czy DQN (Deep Q-Network). Jest to narzędzie szczególnie przydatne w kontekście symulacji fizycznych, gdzie wymagana jest szybka i efektywna implementacja algorytmów RL do testowania i optymalizacji systemów sterowania. Stable Baselines3 jest dobrze udokumentowaną platformą, która ułatwia implementację i eksperymentowanie z różnymi metodami RL. Dzięki swojej prostocie i kompatybilności z popularnymi symulatorami, jak Gazebo czy Unity, biblioteka ta stała się jednym z najczęściej wykorzystywanych narzędzi w projektach związanych z uczeniem maszynowym i symulacjami fizycznymi.

Poniżej przedstawione zostały jedne z popularniejszych oraz częściej wykorzystywanych narzędzi służących do trenowania modeli sztucznej inteligencji sterujących lotami rakiet.

- **Simulink z Aerospace Blockset [27].**

MATLAB w połączeniu z Simulinkiem oraz dodatkiem Aerospace Blockset to zaawansowane narzędzie umożliwiające symulację lotu rakiet. Oprogramowanie to pozwala na elastyczne modelowanie dynamiki obiektów latających, uwzględniając siły aerodynamiczne, grawitację i inne istotne parametry fizyczne. Aerospace Blockset oferuje gotowe bloki do analizy trajektorii oraz projektowania systemów sterowania. Dodatkowo, MATLAB umożliwia integrację z algorytmami uczenia maszynowego, co pozwala na trenowanie modeli sterowania w oparciu o dane z symulacji. System ten jest płatny i wymaga zaawansowanej wiedzy z zakresu modelowania dynamicznego, jednak oferuje bardzo wysoką precyzję i wsparcie dla projektów naukowych i inżynierskich.

- **Microsoft AirSim [18].**

Microsoft AirSim to open-source'owy symulator lotu, pierwotnie zaprojektowany dla dronów, ale możliwy do dostosowania do symulacji rakiet. Oprogramowanie to obsługuje realistyczne warunki fizyczne, takie jak zmienne aerodynamiczne i różne środowiska lotu. AirSim integruje się z popularnymi frameworkami uczenia maszynowego, takimi jak TensorFlow czy PyTorch, umożliwiając trenowanie modeli sterowania w oparciu o reinforcement learning. Dzięki elastyczności i wsparciu dla programowania API, użytkownik może tworzyć niestandardowe środowiska do symulacji. AirSim jest darmowy, ale jego zastosowanie wymaga dostosowania środowiska do specyficznych potrzeb użytkownika, co może być czasochłonne.

- **Gazebo [8] + ROS [7].**

Gazebo to open-source'owy symulator fizyki, który w połączeniu z systemem ROS (Robot Operating System) stanowi potężne narzędzie do modelowania i sterowania obiektami dynamicznymi, takimi jak rakiety. Gazebo oferuje zaawansowaną symulację fizyczną, w tym modelowanie sił aerodynamicznych, kolizji i zmiennych środowiskowych. Integracja z ROS umożliwia tworzenie i implementację algorytmów sterowania opartych na uczeniu maszynowym, takich jak reinforcement learning. Platforma wspiera również analizę danych w

czasie rzeczywistym, co czyni ją idealnym narzędziem do testowania i optymalizacji systemów sterowania. Gazebo + ROS jest darmowe, ale wymaga zaawansowanej konfiguracji oraz wiedzy z zakresu programowania i modelowania fizycznego.

1.4.3 Podsumowanie.

Jak zostało to przedstawione w tym rozdziale, pomimo dość niszowego zagadnienia, na rynku dostępnych jest wiele narzędzi, które mogą być wykorzystane do stworzenia symulacji lotu rakiety oraz trenowania modeli sztucznej inteligencji. Narzędzia te są w większości bardzo zaawansowane i wymagają dużego doświadczenia oraz wiedzy w dziedzinie symulacji, fizyki, jak i uczenia maszynowego. Często są to także narzędzia płatne. Niewątpliwie jednak rozwiązania te oferują symulacje o dużej dokładności i szczegółowości, dzięki zastosowaniu skomplikowanych rozwiązań takich jak CFD czy analiza trajektorii lotu. Użycie dokładnych symulacji i trenowanie modeli, za pomocą dobrze dobranych algorytmów umożliwia osiągnięcie bardzo dobrych wyników w sterowaniu lotem rakiety.

W niniejszej pracy inżynierskiej wykorzystane zostały własne implementacje silnika fizycznego oraz algorytmu sterującego, co pozwoliło na pełną kontrolę nad procesem tworzenia i testowania systemu sterującego rakieta. Efektem tego jest także uproszczony model fizyczny, o czym mowa jest w kolejnych rozdziałach pracy. Rozwiązanie prezentowane w tej pracy odbiega od profesjonalnych narzędzi pod względem skomplikowania i dokładności, jednak pozwala na zrozumienie sposobu działania symulacji lotu rakiety oraz algorytmów sterujących i jest dobrą bazą do rozwoju bardziej zaawansowanych systemów w przyszłości.

2 Moduł I: System symulacji lotów rakietowych.

System do symulacji lotów rakietowych to pierwszy z modułów niniejszej pracy inżynierskiej. Za pomocą tego projektu osiągnięty zostaje cel pośredni pracy - stworzenie środowiska, w którym możliwe jest testowanie algorytmów sterujących rakieta.

Autorem tego modułu jest Jakub Kindlik.

W tym rozdziale przedstawione zostały teoretyczne podstawy fizyczne, użyte technologie, oraz trzy główne podrozdziały tego modułu:

- Projekt architektury systemu.
- Model sił fizycznych w systemie.
- Moduł wizualizacji symulacji.

Omówione zostały także zagadnienia związane ze sprawdzeniem dokładności symulacji, opisem wyników oraz z ewolucją projektu.

2.1 Założenia projektowe.

W trakcie pracy nad modułem silnika fizycznego symulacji lotów rakietowych poczyniono szereg założeń, które miały na celu ułatwienie implementacji oraz zapewnienie odpowiedniej wydajności symulacji. Poniżej przedstawiono najważniejsze z nich:

- **Uprozczone modelowanie sił działających na raketę.**

W celu zmniejszenia złożoności obliczeniowej model sił działających na raketę został odpowiednio uproszczony. Uwzględnione zostały kluczowe siły, takie jak grawitacja, siła oporu powietrza i siła ciągu silnika czy siła nośna, ale zostały one zamodelowane w uproszczony sposób. Takie podejście umożliwiło łatwiejsze zamodelowanie systemu i zredukowało zapotrzebowanie na zasoby obliczeniowe.

- **Optymalizacja pod kątem pracy na komputerze osobistym.**

Symulacje były wykonywane na komputerze osobistym, dlatego system symulacji lotów rakietowych został zoptymalizowany pod kątem wydajności, jak

i okrojony w kwestii skomplikowanych obliczeń. Ograniczenie zasobożności było konieczne, aby zapewnić płynne działanie nawet na sprzęcie o przeciętnych parametrach.

- **Uproszczona geometria rakiety.**

W celu ograniczenia złożoności modelu fizycznego rakietę ma kształt walca i nie posiada skrzydeł. Taka decyzja pozwoliła na skupienie się na kluczowych aspektach dynamiki lotu bez nadmiernego komplikowania obliczeń.

- **Sterowanie poprzez obracany silnik.**

Układ sterujący rakieta opiera się na możliwości obracania silnika, co bezpośrednio wpływa na trajektorię lotu. Taki model umożliwia sterowanie rakieta przez algorytmy samouczące, poprzez zmianę jednego parametru - wektora kątów nachylenia silnika.

- **Możliwość modyfikacji parametrów symulacji w czasie rzeczywistym.**

Symulacja została zaprojektowana w sposób umożliwiający ingerencję w jej parametry w trakcie trwania. Modyfikowanym parametrem jest kąt nachylenia silnika. Taka funkcjonalność pozwalała na uczenie algorytmów sterujących w trakcie trwania symulacji.

- **Możliwość wizualizacji symulacji.**

System został wyposażony w opcjonalną funkcję wizualizacji, co umożliwiło analizę trajektorii i zachowania rakiety w sposób intuicyjny. Aby ograniczyć zużycie zasobów komputera, wizualizacja nie jest tworzona w czasie trwania symulacji, a jedynie po jej zakończeniu, na bazie pliku z zapisanymi danymi.

Założenia te konieczne są do stworzenia silnika fizycznego, który będzie w stanie współpracować z modułem algorytmu sterującego rakieta, jednocześnie będąc na tyle prostym, aby nie obciążać zasobów komputera osobistego.

2.2 Teoretyczne podstawy fizyczne.

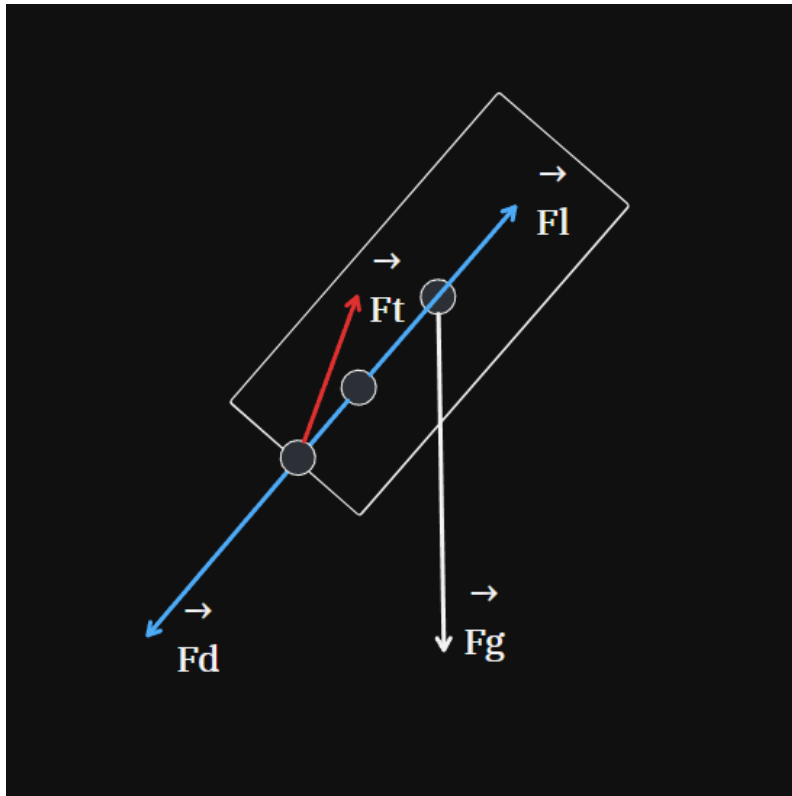
Tworzenie silnika symulacji lotów raketowych wymaga zrozumienia zasad fizyki odnoszących się do ruchu rakiety w atmosferze, a także zjawisk towarzyszących temu ruchowi. Ponadto konieczne jest zrozumienie podstawowych zagadnień z zakresu matematyki, w tym geometrii przestrzennej oraz równań różniczkowych.

W tym rozdziale po krótko omówione zostały zagadnienia, które stanowią podstawę implementacji systemu symulacji lotów raketowych.

2.2.1 Opis sił działających na rakietę

W trakcie lotu rakiety w atmosferze działają na nią różne siły, które determinują zarówno ruch postępowy, jak i obrotowy.

Najważniejsze z nich zostały przedstawione na rysunku 2.1.



Rysunek 2.1. Siły działające na rakietę w locie, źródło: Opracowanie własne.

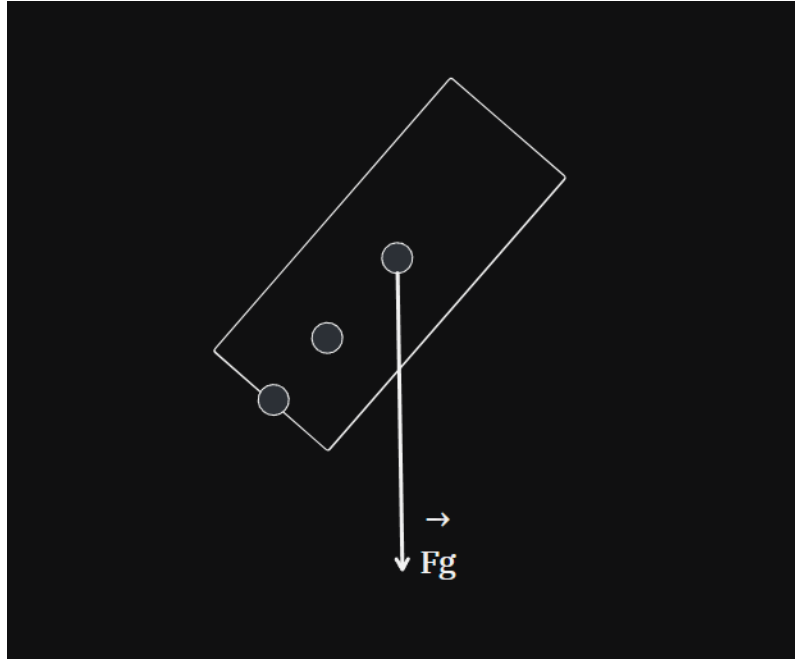
Siła ciężkości (*Gravity*)

Siła skierowana ku środkowi Ziemi, działająca na środek ciężkości rakiety (*Center of Gravity, CoG*). Jej wartość wyraża się jako:

$$\vec{F}_g = m \cdot \vec{g}, \quad (2.1)$$

, gdzie:

- m – masa rakiety [kg],
- $\vec{g}$ – wektor przyspieszenia grawitacyjnego [m/s^2], zazwyczaj do symulacji przyjmuje się wartości około 9.81 m/s^2 , odpowiadające przyspieszeniu grawitacyjnemu na poziomie morza.



Rysunek 2.2. Siła ciężkości działająca na raketę, źródło: *Opracowanie własne.*

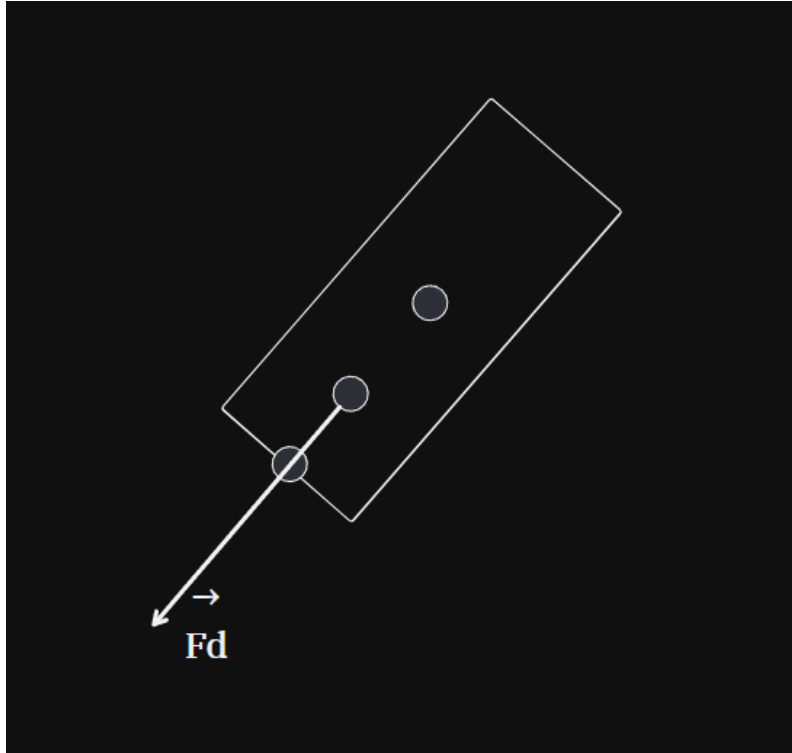
Siła oporu aerodynamicznego (*Drag*)

Siła działająca również na punkt *Center of Pressure (CoP)*, przeciwna do kierunku ruchu. Zależy od właściwości powietrza i kształtu rakiety oraz jej prędkości, a także od współczynnika oporu aerodynamicznego i efektywnej powierzchni rakiety, a jej wartość wyraża się jako:

$$\vec{F}_d = \frac{1}{2} \rho v^2 C_d A, \quad (2.2)$$

, gdzie:

- ρ – gęstość powietrza [kg/m^3],
- v – prędkość rakiety względem powietrza [m/s],
- C_l – współczynnik oporu aerodynamicznego rakiety (zależy od kąta natarcia oraz kształtu rakiety),
- A – efektywna powierzchnia rakiety [m^2].



Rysunek 2.3. Siła oporu aerodynamicznego działająca na raketę, *źródło: Opracowanie własne.*

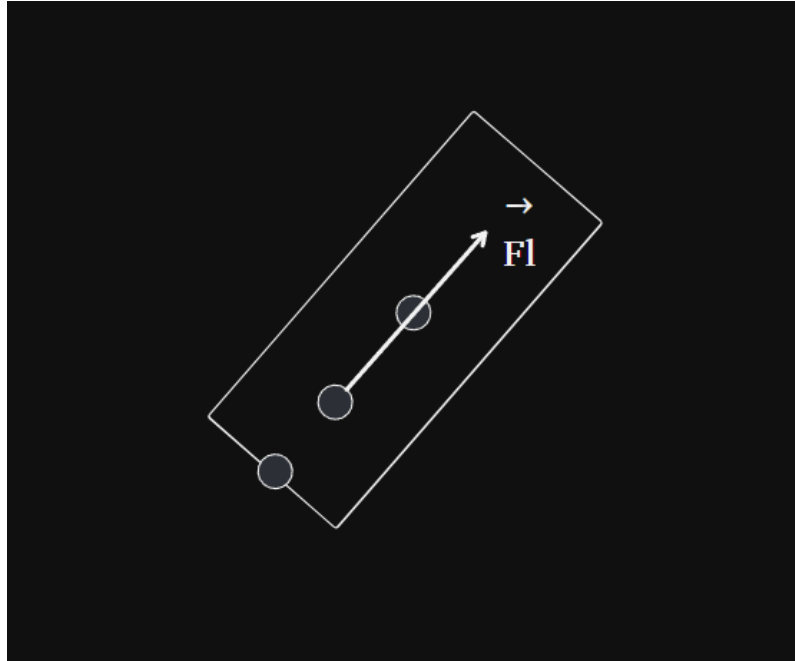
Siła nośna (*Lift*)

Siła działająca na punkt środka parcia (*Center of Pressure, CoP*) rakiety, prostopadła do kierunku przepływu powietrza. Jest istotna w stabilizacji lotu i zależy od kształtu rakiety, gęstości powietrza, prędkości rakiety, współczynnika siły nośnej oraz kąta natarcia. Wyraża się jako:

$$\vec{F}_l = \frac{1}{2} \rho v^2 C_l A, \quad (2.3)$$

, gdzie:

- ρ – gęstość powietrza [kg/m^3],
- v – prędkość rakiety względem powietrza [m/s],
- C_l – współczynnik siły nośnej (zależy od kąta natarcia oraz kształtu rakiety),
- A – efektywna powierzchnia rakiety [m^2].



Rysunek 2.4. Siła nośna działająca na raketę, źródło: *Opracowanie własne*.

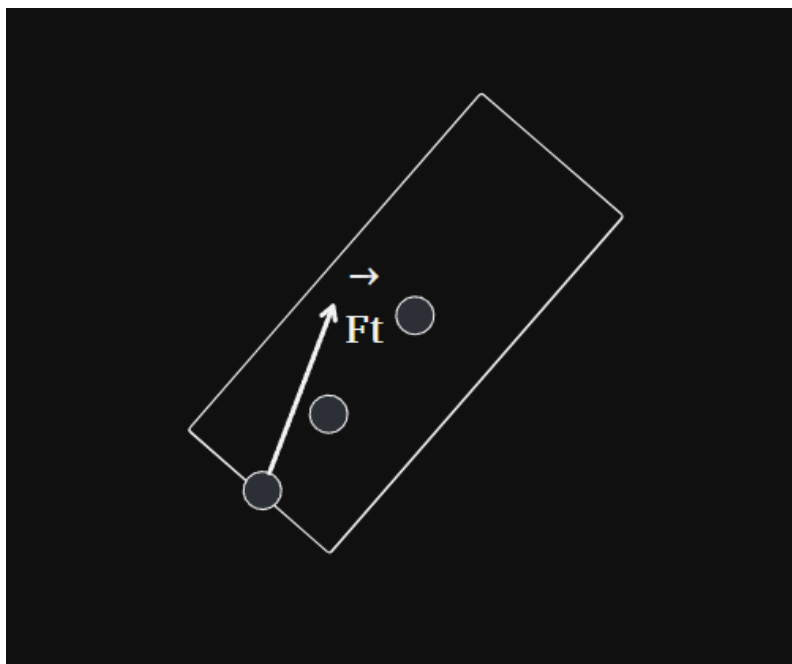
Siła ciągu (*Thrust*)

Siła generowana przez silnik rakiety, działająca na punkt mocowania silnika *Center of Thrust (CoT)* wzdłuż osi rakiety. Wyrażenie opisujące siłę ciągu to:

$$\vec{F}_t = \mu_e \cdot \vec{v}_e, \quad (2.4)$$

, gdzie:

- μ_e – wyrzucona masa spalin w jednostce czasu [kg/s],
- v_e – prędkość wyrzutu spalin [m/s].



Rysunek 2.5. Siła ciągu działająca na rakietę, *źródło: Opracowanie własne.*

2.2.2 Równania ruchu postępowego

Ruch postępowy rakiety w atmosferze opisuje druga zasada dynamiki Newtona, zgodnie z którą suma wszystkich sił działających na raketę równa jest iloczynowi masy rakiety i przyspieszenia środka masy:

$$\vec{F}_{net} = \vec{F}_g + \vec{F}_l + \vec{F}_d + \vec{F}_t. \quad (2.5)$$

Przyspieszenie rakiety można więc wyznaczyć jako:

$$\vec{a} = \frac{\vec{F}_{net}}{m} \quad (2.6)$$

natomiast prędkość w kolejnych momentach czasu oblicza się poprzez całkowanie przyspieszenia względem czasu:

$$\vec{v}(t) = \vec{v}_0 + \int_0^t \vec{a}(\tau) d\tau, \quad (2.7)$$

gdzie:

- $\vec{v}_0$ – początkowa prędkość rakiety [m/s].

Pozycję rakiety można wyznaczyć poprzez kolejne całkowanie prędkości:

$$\vec{x}(t) = \vec{x}_0 + \int_0^t \vec{v}(\tau) d\tau \quad (2.8)$$

gdzie:

- $\vec{x}_0$ – początkowa pozycja rakiety [m].

2.2.3 Ruch obrotowy i momenty sił

Ruch obrotowy rakiety jest determinowany momentami sił działającymi na raketę. Kluczowe znaczenie ma tutaj punkt centralny, często z powodu wygody obliczeniowej przyjmowany jako środek masy (CoG), wokół którego rakietę może rotować. Moment siły względem środka masy wyraża się jako:

$$\vec{M} = \vec{r} \times \vec{F}, \quad (2.9)$$

Iloczyn wektorowy $\vec{M} = \vec{r} \times \vec{F}$ można rozwinąć współrzędnie jako:

$$\vec{M} = \begin{bmatrix} M_x \\ M_y \\ M_z \end{bmatrix} = \begin{bmatrix} r_y F_z - r_z F_y \\ r_z F_x - r_x F_z \\ r_x F_y - r_y F_x \end{bmatrix}. \quad (2.10)$$

gdzie:

- $\vec{r} = [r_x, r_y, r_z]^T$ – wektor od punktu centralnego do punktu przyłożenia siły (często za punkt centralny przyjmuje się środek masy obiektu) [m],
- $\vec{F} = [F_x, F_y, F_z]^T$ – siła działająca na raketę [N].

Równania ruchu obrotowego

Moment obrotowy generuje przyspieszenie kątowe zgodnie z równaniem:

$$\vec{\varepsilon} = \hat{\mathbf{I}}^{-1} \vec{M}, \quad (2.11)$$

lub w postaci macierzowo-wektorowej:

$$\begin{bmatrix} \varepsilon_x \\ \varepsilon_y \\ \varepsilon_z \end{bmatrix} = \begin{bmatrix} I_{xx} & I_{xy} & I_{xz} \\ I_{yx} & I_{yy} & I_{yz} \\ I_{zx} & I_{zy} & I_{zz} \end{bmatrix}^{-1} \begin{bmatrix} M_x \\ M_y \\ M_z \end{bmatrix}, \quad (2.12)$$

gdzie:

- $\vec{\varepsilon} = [\varepsilon_x, \varepsilon_y, \varepsilon_z]^T$ – wektor przyspieszenia kąowego [rad/s²],
- $\hat{\mathbf{I}}$ – tensor bezwładności rakiety (symetryczna macierz momentów i iloczynów bezwładności) [kg · m²],
- $\vec{M} = [M_x, M_y, M_z]^T$ – wektor momentu sił (moment obrotowy) [N · m].

Definicja tensora bezwładności Tensor bezwładności $\hat{\mathbf{I}}$ opisuje rozkład masy względem osi obrotu i wyraża się całkowo jako:

$$I_{ij} = \int_V \rho(\vec{r}) \left(\delta_{ij} \|\vec{r}\|^2 - r_i r_j \right) dV, \quad (2.13)$$

gdzie:

- I_{ij} – element tensora bezwładności,
- $\rho(\vec{r})$ – gęstość masy w punkcie $\vec{r}$,
- δ_{ij} – delta Kroneckera,
- r_i, r_j – współrzędne wektora $\vec{r}$.

W przypadku obiektu dyskretnego (np. modelowanego jako zbiór punktów masy), tensor przyjmuje postać sumacyjną:

$$I_{ij} = \sum_{k=1}^n m_k \left(\delta_{ij} \|\vec{r}_k\|^2 - (r_k)_i (r_k)_j \right), \quad (2.14)$$

gdzie m_k i $\vec{r}_k$ to masa i położenie k -tego punktu.

Tensor bezwładności dla układów o symetrii osiowej często upraszcza się do postaci diagonalnej:

$$\hat{\mathbf{I}} = \begin{bmatrix} I_{xx} & 0 & 0 \\ 0 & I_{yy} & 0 \\ 0 & 0 & I_{zz} \end{bmatrix}, \quad (2.15)$$

co znacznie upraszcza analizę dynamiki rakiety.

Następnie prędkość kątowna rakiety $\vec{\omega}$ zmienia się w czasie zgodnie z wyrażeniem wektorowym:

$$\frac{d\vec{\omega}}{dt} = \vec{\varepsilon}, \quad (2.16)$$

gdzie:

- $\vec{\varepsilon}$ – wektor przyspieszenia kąowego [rad/s²].

Rozwiązując powyższe równanie przez całkowanie, otrzymujemy:

$$\vec{\omega}(t) = \vec{\omega}_0 + \int_0^t \vec{\varepsilon}(\tau) d\tau, \quad (2.17)$$

gdzie:

- $\vec{\omega}_0$ – początkowa prędkość kątowna [rad/s].

Jeśli przyspieszenie kątowe $\vec{\varepsilon}$ jest wyrażone w zależności od momentu sił $\vec{M}$ i tensora bezwładności $\hat{\mathbf{I}}$ zgodnie z równaniem:

$$\vec{\varepsilon} = \hat{\mathbf{I}}^{-1} \vec{M}, \quad (2.18)$$

możemy uwzględnić tę zależność w powyższej formule, aby obliczyć $\vec{\omega}(t)$ przy znanym momencie sił $\vec{M}$ w funkcji czasu.

Kąt orientacji rakiety w przestrzeni zmienia się w czasie zgodnie z wyrażeniem:

$$\frac{d\vec{\theta}}{dt} = \vec{\omega}, \quad (2.19)$$

gdzie:

- $\vec{\omega}$ – wektor prędkości kątowej [rad/s].

Rozwiązując powyższe równanie przez całkowanie, otrzymujemy:

$$\vec{\theta}(t) = \vec{\theta}_0 + \int_0^t \vec{\omega}(\tau) d\tau, \quad (2.20)$$

gdzie:

- $\vec{\theta}_0$ – początkowy kąt orientacji rakiety [rad].

W przypadku, gdy prędkość kątowa $\vec{\omega}$ jest związana z momentem sił $\vec{M}$ i tensorem bezwładności $\hat{\mathbf{I}}$, zależność ta może być użyta do obliczenia kąta orientacji w funkcji momentu sił w czasie.

Równoważenie sił i momentów – warunki stabilności

Podczas lotu rakietą doświadcza działania sił aerodynamicznych oraz grawitacyjnych. Stabilność lotu zależy od odpowiedniego rozmieszczenia środka masy (CoG, ang. *Center of Gravity*) oraz środka parcia (CoP, ang. *Center of Pressure*). Aby rakietą pozostawała stabilna w locie, konieczne jest, aby środek masy znajdował się przed środkiem parcia, patrząc w kierunku lotu.

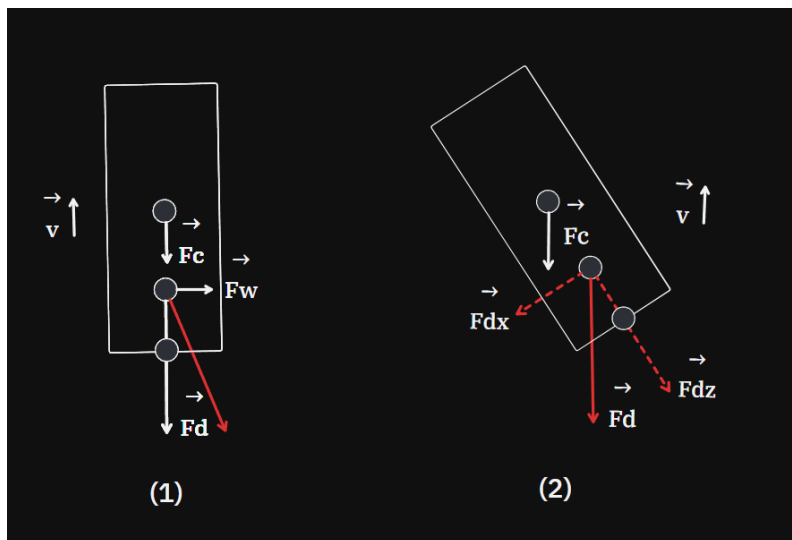
Gdy CoG znajduje się bliżej nosa rakiety niż CoP, moment siły generowany przez siły aerodynamiczne ma charakter stabilizujący. W przypadku chwilowego odchylenia rakiety od osi lotu, na przykład pod wpływem podmuchu wiatru, siła aerodynamiczna działająca w CoP wytwarza moment względem CoG. Ten moment skierowany jest tak, aby przywrócić rakietę do pierwotnej trajektorii lotu. Natomiast gdyby CoG znajdował się za CoP, moment aerodynamiczny działałby w odwrotnym kierunku, zwiększając odchylenie i prowadząc do destabilizacji.

Rozważmy przypadek, w którym na rakietę oddziałuje podmuch wiatru, przykładając chwilową siłę boczną, patrz rysunek 2.6 (1). Siła ta powoduje odchylenie rakiety od osi lotu, co zmienia kierunek działania sił aerodynamicznych. Siła aerodynamiczna $\vec{F}_{aero}$, która działa w punkcie CoP, generuje moment względem CoG, zgodnie z wyrażeniem:

$$\vec{M}_{aero} = \vec{r}_{CoP-CoG} \times \vec{F}_{aero}, \quad (2.21)$$

gdzie $\vec{r}_{CoP-CoG}$ to wektor łączący położenie CoG i CoP. Patrz rysunek 2.6 (2).

W sytuacji, gdy CoG znajduje się przed CoP, moment $\vec{M}_{aero}$ działa stabilizująco, kierując rakietę z powrotem na pierwotną trajektorię. Stabilizacja odbywa się dzięki temu, że odległość między CoG a CoP ($d = |\vec{r}_{CoP} - \vec{r}_{CoG}|$) w połączeniu z siłą aerodynamiczną generuje wystarczający moment przywracający. Proces ten jest naturalnym efektem rozmieszczenia masy rakiety i nie wymaga dodatkowych mechanizmów sterujących.



Rysunek 2.6. Przykład działania sił stabilizujących rakietę w locie, *źródło: Opracowanie własne.*

Gdyby jednak środek masy znajdował się za środkiem parcia, moment siły $\vec{M}_{aero}$

działałby w przeciwnym kierunku, zwiększając odchylenie rakiety od osi lotu. W takim przypadku każdy podmuch wiatru mógłby prowadzić do narastającej niestabilności i ostatecznej utraty kontroli nad rakieta.

Podsumowując, stabilność rakiety w locie zależy od odpowiedniego rozmieszczenia CoG i CoP, które decyduje o kierunku działania momentów siły aerodynamicznej. W przypadku chwilowych zakłóceń, takich jak podmuch wiatru, stabilizujący moment aerodynamiczny może skutecznie przywrócić raketę na właściwą trajektorię, pod warunkiem spełnienia wspomnianego warunku stabilności.

2.2.4 Modelowanie środowiska atmosferycznego

Środowisko atmosferyczne wpływa na dynamikę ruchu rakiety poprzez zmienne fizyczne, takie jak gęstość powietrza (ρ), ciśnienie atmosferyczne (p) oraz temperatura (T). Zmiany tych parametrów w funkcji wysokości nad powierzchnią ziemi mają istotny wpływ na opór aerodynamiczny, przyspieszenie rakiety oraz jej trajektorię. W związku z tym modelowanie tych zjawisk jest kluczowe dla przewidywania ruchu rakiety. Poniżej przedstawiono główne parametry atmosferyczne oraz ich modelowanie:

- **Zmiana gęstości powietrza w zależności od wysokości:**

$$\rho(h) = \rho_0 \cdot \exp\left(-\frac{h}{H}\right), \quad (2.22)$$

gdzie:

- ρ_0 – gęstość powietrza na poziomie morza [kg/m^3],
- h – wysokość [m],
- H – charakterystyczna wysokość skali atmosfery, zazwyczaj przyjmowana na poziomie 8 500 m [m].

Gęstość powietrza maleje wykładniczo wraz z wysokością, co wpływa na opór aerodynamiczny działający na raketę. Niższa gęstość powietrza oznacza mniejszy opór w wyższych partiach atmosfery.

- **Ciśnienie atmosferyczne w funkcji wysokości:** Ciśnienie atmosferyczne w funkcji wysokości nad poziomem morza można przybliżyć za pomocą wzoru barometrycznego. Dla warstw troposferycznych, gdzie temperatura spada z

wysokością, ciśnienie atmosferyczne p wyraża się równaniem:

$$p(h) = p_0 \left(1 - \frac{Lh}{T_0} \right)^{\frac{gM}{RL}}, \quad (2.23)$$

gdzie:

- p_0 – ciśnienie na poziomie morza [Pa],
- L – gradient temperatury (spadek temperatury na jednostkę wysokości) [K/m],
- h – wysokość [m],
- T_0 – temperatura na poziomie morza [K],
- g – przyspieszenie ziemskie [m/s²],
- M – masa molowa powietrza [kg/mol],
- R – stała gazowa [J/(mol · K)].

To równanie pozwala obliczyć zmiany ciśnienia atmosferycznego w zależności od wysokości.

- **Temperatura w funkcji wysokości:** Temperaturę w atmosferze modeluje się zazwyczaj poprzez gradient temperatury w troposferze, który wynosi około 6.5°C na każdy kilometr wysokości. Można to zapisać jako:

$$T(h) = T_0 - Lh, \quad (2.24)$$

gdzie:

- T_0 – temperatura na poziomie morza [K],
- L – gradient temperatury [K/m],
- h – wysokość [m].

Ta zależność jest stosowana w troposferze, czyli do wysokości około 10 km. Powyżej tej wysokości temperatura przestaje spadać i stabilizuje się lub wzrasta w stratosferze.

- **Zmiany parametrów powietrza w różnych warstwach atmosfery:** Zjawiska takie jak turbulencje, zmiany w prędkości wiatru, gradienty temperatury oraz zmieniające się ciśnienie atmosferyczne mają również wpływ na trajektorię rakiety. Wiatr, na przykład, może wpłynąć na odchylenie rakiety od pierwotnej trajektorii, wprowadzając dodatkowe momenty aerodynamiczne. W przypadku wiatru o dużej prędkości, rakietę może zostać odchylona od osi lotu,

co skutkuje zmianą sił działających na nią. Równanie opisujące wpływ wiatru na trajektorię rakiety można przedstawić jako:

$$\vec{F}_{\text{wind}} = C_w \cdot A \cdot \rho \cdot v_w^2, \quad (2.25)$$

gdzie:

- C_w – współczynnik oporu wiatru,
- A – powierzchnia rakiety,
- ρ – gęstość powietrza,
- v_w – prędkość wiatru.

W takiej sytuacji moment aerodynamiczny wytwarzany przez wiatr może wywołać chwilowe odchylenie rakiety, które będzie zrównoważone przez momenty stabilizujące, jeśli CoG znajduje się przed CoP.

Złożoność modelu atmosferycznego

W praktyce modelowanie środowiska atmosferycznego może być znacznie bardziej złożone. Istnieje możliwość uwzględnienia wielu dodatkowych czynników, takich jak zmienność parametrów atmosferycznych w różnych warstwach troposfery i stratosfery, wpływ turbulencji, lokalne różnice ciśnienia i temperatury, a także bardziej szczegółowe modele oporu aerodynamicznego uwzględniające zmiany kształtu rakiety w trakcie lotu. Możliwe jest także zastosowanie zaawansowanych metod numerycznych do dokładniejszego modelowania tych zależności, w tym modelowanie nieliniowych efektów oporu czy dynamicznych zmian parametrów atmosferycznych w czasie rzeczywistym.

Jednakże, w kontekście obliczeń inżynierskich, dla uproszczenia i osiągnięcia wystarczającej dokładności w prognozowaniu trajektorii rakiety, przyjęte wyżej wzory są wystarczające. Pozwalają one na obliczenie kluczowych parametrów, takich jak gęstość powietrza, ciśnienie i temperatura w różnych warstwach atmosfery, przy zachowaniu odpowiedniej efektywności obliczeniowej. Dalsze zgłębianie tych zależności w celu uzyskania bardziej precyzyjnych wyników może być rozważane w przypadkach wymagających szczegółowej analizy, jednak w standardowych obliczeniach rakietowych, przyjęte uproszczenia są wystarczające do uzyskania akceptowalnych wyników.

2.2.5 Kąt natarcia rakiety

Kąt natarcia (α) jest kluczowym parametrem aerodynamicznym opisującym orientację rakiety względem przepływu powietrza. Określa on, pod jakim kątem oś symetrii rakiety jest nachylona do wektora prędkości względnej powietrza.

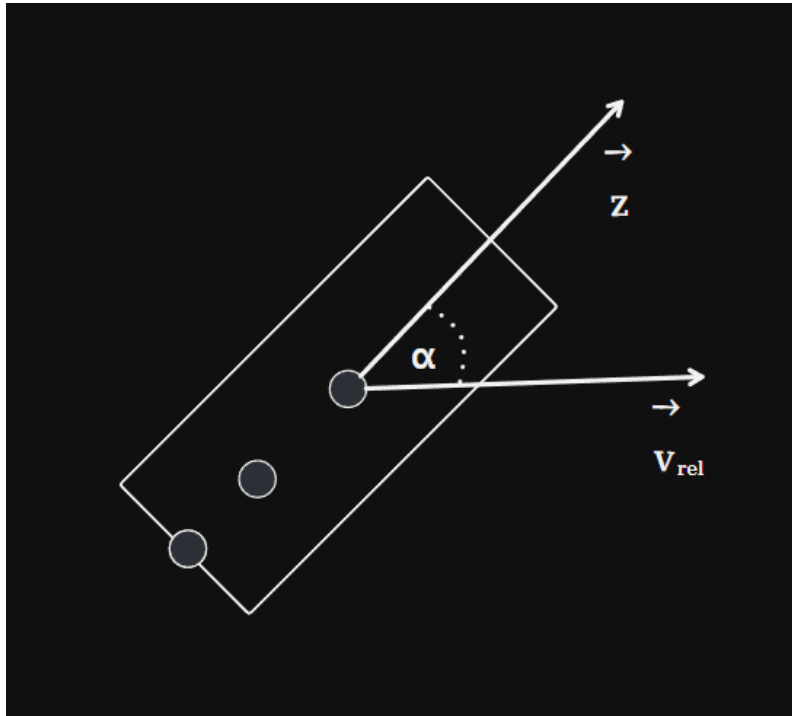
Definicja kąta natarcia

Kąt natarcia α definiuje się jako kąt pomiędzy kierunkiem wektora prędkości powietrza względem punktu środkowego rakiety $\vec{v}_{rel}$, a osią symetrii rakiety. Wyrażenie to zapisuje się jako:

$$\alpha = \arccos \left(\frac{\vec{v}_{rel} \cdot \vec{z}}{\|\vec{v}_{rel}\| \|\vec{a}\|} \right), \quad (2.26)$$

gdzie:

- $\vec{v}_{rel}$ – wektor prędkości względnej przepływu powietrza,
- $\vec{z}$ – wektor osi symetrii rakiety,
- $\|\vec{v}_{rel}\|$ – wartość prędkości względnej [m/s],
- $\|\vec{a}\|$ – wartość wektora osi symetrii (zwykle 1 jako wektor jednostkowy).



Rysunek 2.7. Kąt natarcia rakiety, źródło: Opracowanie własne.

Wpływ kąta natarcia na dynamikę rakiety

Kąt natarcia ma istotny wpływ na siły aerodynamiczne, w szczególności:

- **Siła oporu ($\vec{F}_d$):** Przy większych kątach natarcia wzrasta siła oporu, co prowadzi do większych strat energetycznych w trakcie lotu.
- **Siła nośna ($\vec{F}_l$):** Przy większych kątach natarcia wzrasta siła nośna, która stabilizuje rakietę, ale również zwiększa ryzyko niestabilności aerodynamicznej.

Zakres kąta natarcia w rakietach

Dla ракет stabilnych aerodynamicznie kąt natarcia zazwyczaj jest mały (rzędu kilku stopni), aby zminimalizować siły destabilizujące. W czasie lotu kąt natarcia zmienia się dynamicznie w zależności od:

- warunków atmosferycznych,
- trajektorii lotu,
- działania układów sterujących.

Zależność sił aerodynamicznych od kąta natarcia

Siły aerodynamiczne działające na rakiety zależą silnie od kąta natarcia α . W szczególności od kąta natarcia zależą współczynniki sił nośnej C_l i oporu C_d . Wartości tych współczynników zazwyczaj są wyznaczane eksperymentalnie za pomocą tunelu aerodynamicznego.

Od kąta natarcia zależy także powierzchnia efektywna rakiety A_α , która wpływa na siły aerodynamiczne.

Na podstawie tych zależnych od kąta natarcia parametrów wyznaczane są siły aerodynamiczne działające na rakietę w locie. Oznacza to, że są one bezpośrednio związane z orientacją rakiety względem przepływu powietrza, a co za tym idzie, z kątem natarcia.

2.2.6 Geometria rakiety

Wedle założeń modelu rakiety, jej geometria jest uproszczona do kształtu walca. Taki model pozwala na efektywne obliczenia sił aerodynamicznych i dynamiki ruchu, zachowując wystarczającą dokładność w kontekście modelowania lotu rakiety.

Geometria walca

Dla uproszczonego modelu rakiety jako walca o promieniu r i wysokości h podstawowe właściwości geometryczne wyznacza się następująco:

- Objętość walca:

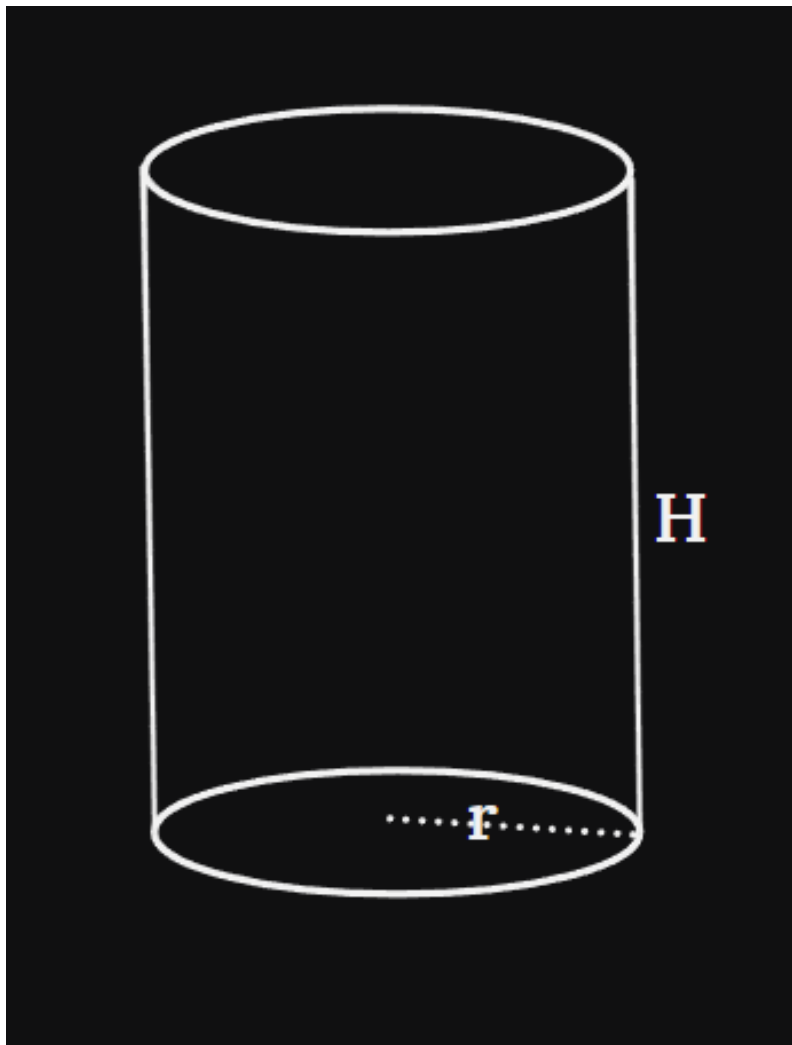
$$V_{cyl} = \pi r^2 h, \quad (2.27)$$

gdzie:

- r – promień podstawy walca [m],
- h – wysokość walca [m].

- Powierzchnia boczna walca:

$$A_{cyl} = 2\pi r h, \quad (2.28)$$



Rysunek 2.8. Geometria walca, *źródło: Opracowanie własne.*

Powierzchnia przekroju rakiety jako walca pod różnymi kątami

Powierzchnia przekroju rakiety, modelowanej jako walec o promieniu r i wysokości h , zależy od kąta natarcia α względem przepływu powietrza.

Przekrój czołowy ($\alpha = 0^\circ$): Jest to pole rzutu bryły rakiety na płaszczyznę prostopadłą do kierunku lotu. Jeżeli rakietę porusza się wzdłuż swojej osi symetrii, przekrój czołowy jest równy powierzchni podstawy cylindra:

$$A_0 = \pi r^2. \quad (2.29)$$

Przekrój pod kątem α : Jeżeli rakietę jest nachylona pod kątem α do kierunku przepływu, powierzchnia efektywna przekroju zmienia się zgodnie z rzutem walca na płaszczyznę prostopadłą do przepływu. Powierzchnię tę można wyznaczyć jako:

$$A_\alpha = \pi r^2 \cos \alpha + 2rh \sin \alpha, \quad (2.30)$$

gdzie:

- $\cos \alpha$ odpowiada zmniejszeniu widocznej powierzchni podstawy wzdłuż osi symetrii,
- $2rh \sin \alpha$ dodaje widoczną powierzchnię boczną w rzucie.

Granice powierzchni przekroju:

- Dla $\alpha = 0^\circ$: $A_\alpha = A_0 = \pi r^2$ (przekrój czołowy).
- Dla $\alpha = 90^\circ$: $A_\alpha = 2rh$ (przekrój boczny).

Wartość A_α zależy więc silnie od kąta natarcia α , co wpływa na siły aerodynamiczne działające na raketę w locie. Wartość A_α wykorzystywana jest do obliczeń oporu aerodynamicznego oraz sił nośnych rakiety - jest to bezpośrednie powiązanie między geometrią rakiety, a siłami na nią oddziałującymi.

Charakterystyka punktów charakterystycznych rakiety

W rakiecie wyróżnia się trzy kluczowe punkty geometryczne i dynamiczne, na które oddziałują wszelkie siły działające na lecącą raketę:

- **Środek ciężkości (CoG):** Punkt, w którym skupiona jest całkowita masa rakiety. Dla symetrii osiowej i jednorodnej gęstości materiałów jest zlokalizowany na osi rakiety. Położenie CoG zmienia się w trakcie lotu w zależności od spalania paliwa i zmiany masy, choć w symulacji zakłada się zazwyczaj stałe położenie CoG. Ponadto w amatorskich rakietach wpływ masy paliwa na położenie CoG jest znikomy i pomijalny.

- **Środek parcia (CoP):** Punkt, w którym można przyłożyć wypadkową siłę aerodynamiczną działającą na raketę. Położenie CoP zależy od kształtu rakiety, kontu natarcia i rozkładu ciśnienia aerodynamicznego. W modelu fizyki przyjętym na potrzeby niniejszej pracy inżynierskiej zakłada się, że CoP znajduje się na osi rakiety, oraz jest to jeden punkt zarówno dla sił nośnych, jak i oporu aerodynamicznego. Podejściem bliższym rzeczywistości jest modelowanie CoP jako punktu oddzielnie dla sił nośnych i oporu, co pozwala na bardziej precyzyjne obliczenia sił aerodynamicznych, jest to jednak bardziej skomplikowane i kłopotliwe obliczeniowo, dlatego też w modelu uproszczonym przyjmuje się wspólny punkt CoP dla obu sił.
- **Środek ciągu (CoT):** Punkt, w którym działa wypadkowa siła ciągu generowana przez silniki rakiety. Położenie CoT zależy od konfiguracji silników i ich rozmieszczenia.

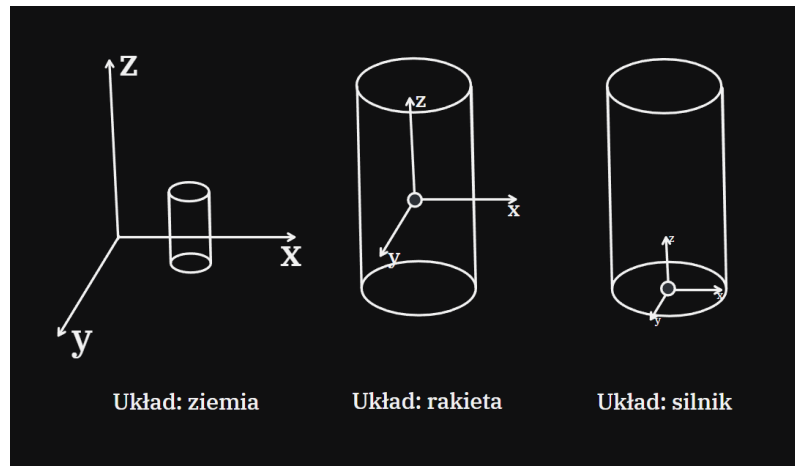
Relacje między punktami Dla zapewnienia stabilności lotu rakiety:

- CoP powinien znajdować się poniżej CoG, w głównej osi rakiety, co gwarantuje stabilność aerodynamiczną.
- CoT powinien być blisko osi rakiety, aby zminimalizować momenty obrotowe powodujące niestabilności.

2.2.7 Obroty układów współrzędnych

Układy współrzędnych w dynamice rakiety

Aby w dokładny sposób określać pozycje i obrót rakiety w przestrzeni konieczne jest zamodelowanie układów współrzędnych związanych z ruchem rakiety. W analizach ruchu rakiety wyróżnia się trzy podstawowe układy współrzędnych:

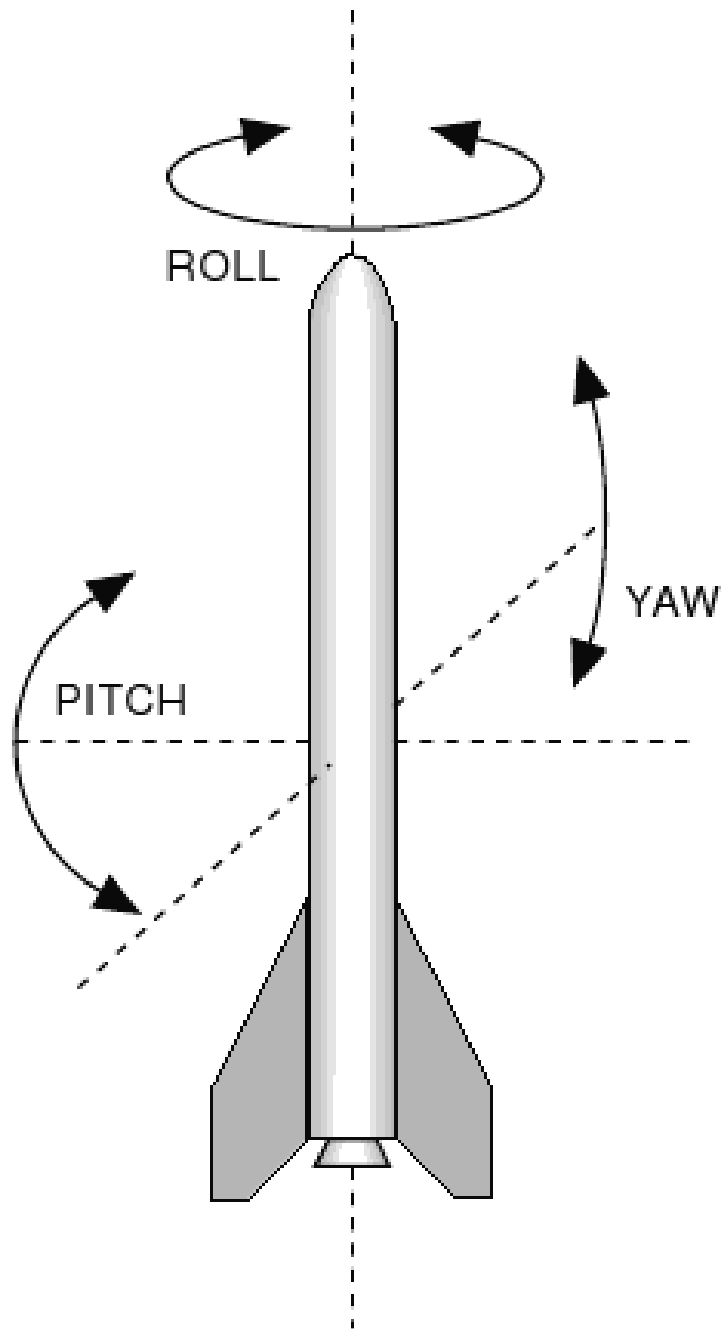


Rysunek 2.9. Układy współrzędnych w dynamice rakiety, *źródło: Opracowanie własne.*

- **Układ inercjalny (Ziemia):** Stały układ odniesienia związany z Ziemią, w którym mierzone są położenie i orientacja rakiety. Oś z jest skierowana w stronę przeciwną do środka Ziemi.
- **Układ lokalny (rakietą):** Układ poruszający się wraz z rakietą. Oś z wskazuje kierunek przodu rakiety i jest główną osią obiektu. Osie y oraz x są prostopadłe do osi z i tworzą ortogonalną do osi z płaszczyznę.
- **Układ lokalny (silnik):** Układ związany z kierunkiem działania siły ciągu. Oś z wskazuje wektor ciągu, a osie y i x są prostopadłe do z .

Kąty Eulera i transformacje układów współrzędnych

Do opisu orientacji rakiety w przestrzeni wykorzystuje się kąty Eulera, umożliwiające transformacje układów współrzędnych za pomocą operacji macierzowych. Kąty Eulera to trzy kąty opisujące orientację rakiety względem trzech osi układu inercjalnego:



Rysunek 2.10. Obroty rakiety w trzech osiach (Roll, Pitch, Yaw), źródło: *Science Learning Hub*, <https://www.sciencelearn.org.nz/resources/387-rocket-control>

- **Yaw** (ψ): Obrót wokół osi z w układzie Ziemi.
- **Pitch** (θ): Obrót wokół osi y w układzie Ziemi.
- **Roll** (ϕ): Obrót wokół osi x w układzie Ziemi.

Macierze rotacji Transformacja między układami współrzędnych odbywa się za pomocą macierzy rotacji. Dla kątów Eulera macierz rotacji R jest wynikiem trzech

obrotów:

$$R = R_z(\psi)R_y(\theta)R_x(\phi), \quad (2.31)$$

gdzie:

$$R_x(\phi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \phi & -\sin \phi \\ 0 & \sin \phi & \cos \phi \end{bmatrix}, \quad (2.32)$$

$$R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix}, \quad (2.33)$$

$$R_z(\psi) = \begin{bmatrix} \cos \psi & -\sin \psi & 0 \\ \sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix}. \quad (2.34)$$

Złożenie tych macierzy pozwala na obliczenie orientacji rakiety w przestrzeni za pomocą jednej macierzy rotacji R . Obrót wektora jednostkowego w trójwymiarowej przestrzeni możemy obliczyć za pomocą następującego równania macierzowego:

$$\vec{v}' = R\vec{v}, \quad (2.35)$$

Co zapisać można jako pełne równanie macierzowe:

$$\begin{bmatrix} v'_x \\ v'_y \\ v'_z \end{bmatrix} = \begin{bmatrix} \cos \theta \cos \psi & \cos \theta \sin \psi & -\sin \theta \\ \sin \phi \sin \theta \cos \psi - \cos \phi \sin \psi & \sin \phi \sin \theta \sin \psi + \cos \phi \cos \psi & \sin \phi \cos \theta \\ \cos \phi \sin \theta \cos \psi + \sin \phi \sin \psi & \cos \phi \sin \theta \sin \psi - \sin \phi \cos \psi & \cos \phi \cos \theta \end{bmatrix} \begin{bmatrix} v_x \\ v_y \\ v_z \end{bmatrix}, \quad (2.36)$$

gdzie:

- $\vec{v}$ – wektor jednostkowy w układzie lokalnym,
- $\vec{v}'$ – wektor jednostkowy po obrocie o kąty Eulera.
- R – macierz rotacji opisująca obroty wokół osi x , y i z .
- ϕ , θ , ψ – kąty Eulera.

Dzięki kątom Eulera i macierzom rotacji możliwa jest transformacja wektorów i równań między różnymi układami współrzędnych, co jest kluczowe dla analizy trajektorii lotu rakiety oraz sterowania jej orientacją w przestrzeni.

W modelu przyjętym na potrzeby niniejszej pracy inżynierskiej zakłada się, że

rakieta nie rotuje wokół osi z (Yaw), co oznacza, że kąt ψ jest stały. Pozwala to na uproszczenie modelu dynamiki ruchu rakiety, zachowując realistyczne warunki lotu. Pozwala to na uproszczenie modelu dynamiki ruchu, zwłaszcza przy założeniu sterowania obracającym silnikiem raketowym, który koryguje orientację rakiety w locie.

2.2.8 Model symulacji ruchu rakiety

Na podstawie przedstawionych równań i zależności można zbudować model, który wykorzystany zostanie do stworzenia programu do symulacji ruchu rakiety w atmosferze. Model ten uwzględnia zarówno dynamikę ruchu postępowego, jak i obrotowego, opierając się na równaniach dynamiki Newtona i kinematyki ruchu postępowego oraz obrotowego. Model ten uwzględnia jednocześnie pewne uproszczenia, które pozwalają na efektywne obliczenia trajektorii lotu rakiety i zmniejszenie złożoności modelu.

Model składa się z następujących, zmiennych zależnych:

- $\vec{r}$ - pozycja środka masy rakiety w układzie inercyjnym (związany z Ziemią),
- $\vec{v}$ - prędkość środka masy rakiety w układzie liczona jako pierwsza pochodna po czasie wektora pozycji, $\vec{v} = \frac{d\vec{r}}{dt}$,
- $\vec{a}$ - przyspieszenie środka masy rakiety w układzie liczone jako pierwsza pochodna po czasie wektora prędkości, $\vec{a} = \frac{d\vec{v}}{dt}$,
- $\vec{u}$ - kierunek osi symetrii rakiety w układzie inercyjnym (związany z Ziemią), wyznaczany na podstawie kątów Eulera i macierzy rotacji, $\vec{u} = R\vec{a}$,

Zmienne zależne liczone są na podstawie równań dynamiki Newtona i kinematyki, gdzie siły działające na rakiety są sumowane do siły wypadkowej za pomocą następującego równania:

$$\vec{F} = \vec{F}_{\text{thrust}} + \vec{F}_{\text{drag}} + \vec{F}_{\text{lift}} + \vec{F}_{\text{gravity}}, \quad (2.37)$$

gdzie:

- $\vec{F}_{\text{thrust}}$ – siła ciągu generowana przez silniki rakiety,
- $\vec{F}_{\text{drag}}$ – siła oporu aerodynamicznego,
- $\vec{F}_{\text{lift}}$ – siła nośna generowana przez rakiety,
- $\vec{F}_{\text{gravity}}$ – siła grawitacji działająca na rakiety.

Wypadkowy moment obrotowy $\vec{M}$ generowany przez zsumowanie momentów sił działających na raketę, przy użyciu równania:

$$\vec{M} = (\vec{r}_{CoT}) \times \vec{F}_{\text{thrust}} + (\vec{r}_{CoP}) \times \vec{F}_{\text{drag}} + (\vec{r}_{CoP}) \times \vec{F}_{\text{lift}} + (\vec{r}_{CoG}) \times \vec{F}_{\text{gravity}}, \quad (2.38)$$

gdzie:

- $\vec{r}_{CoT}$ – wektor od CoG do CoT,
- $\vec{r}_{CoP}$ – wektor od CoG do CoP,

Za pomocą momentów sił obliczane są kąty Eulera ϕ , θ i ψ , które następnie pozwalają na wyliczenie macierzy rotacji R oraz obrotu rakiety w przestrzeni. Ten komponent pozwala na obliczenie ruchu obrotowego rakiety. Ruch postępowy rakiety jest obliczany na podstawie siły wypadkowej rakiety i jej przyspieszenia, które są sumowane do pozycji i prędkości rakiety. Przyspieszenie obliczane jest na podstawie drugiej zasady dynamiki Newtona:

$$\vec{a} = \frac{\vec{F}}{m}, \quad (2.39)$$

Na podstawie przyspieszenia obliczana jest prędkość rakiety, a następnie zmiana pozycji rakiety w przestrzeni.

Złożenie tych dwóch rodzajów ruchu rakiety pozwala na uzyskanie pełnego modelu symulacji ruchu rakiety w atmosferze, uwzględniającego zarówno ruch postępowy jak i obrotowy.

2.2.9 Podsumowanie

Przedstawione równania i modele opisują w ogólny sposób dynamikę ruchu rakiety w atmosferze, uwzględniając zarówno ruch postępowy, jak i obrotowy oraz wpływ środowiska atmosferycznego. Poprawne zrozumienie i modelowanie tych elementów jest kluczowe dla skutecznej implementacji silnika symulacji fizycznych lotów rakietowych.

2.3 Użyte technologie.

Do stworzenia silnika fizycznego symulacji lotów rakietowych wykorzystane zostały technologie, które zostały opisane w rozdziale 2.3, gdzie przedstawiono technologie zastosowane w obu modułach pracy inżynierskiej.

Dodatkowo do stworzenia modułu I użyte zostały poniższe biblioteki:

- **Biblioteki do analizy danych i obliczeń numerycznych:**

- **numpy**, wersja: 1.26.4,
Podstawowa biblioteka do obliczeń numerycznych w Pythonie. Oferuje wsparcie dla wielowymiarowych tablic oraz operacji na nich.
- **pandas**, wersja: 2.2.3,
Biblioteka do analizy danych, która ułatwia manipulowanie i przetwarzanie danych tabelarycznych. Znajduje szerokie zastosowanie w analizie danych i uczeniu maszynowym.
- **trimesh**, wersja: 4.5.3,
Biblioteka do manipulacji i analizy danych 3D, zwłaszcza trójwymiarowych siatek. Używana w aplikacjach CAD i analizie przestrzennej.

- **Biblioteki do pracy z GUI (interfejsami graficznymi):**

- **PyQt5**, wersja: 5.15.11,
Biblioteka do tworzenia interfejsów graficznych opartych na Qt. Umożliwia tworzenie aplikacji z bogatym GUI, wspiera wiele platform.
- **PyQt5-Qt5**, wersja: 5.15.2,
Zestaw narzędzi i komponentów Qt5 dla PyQt5, pozwalający na korzystanie z funkcji Qt5 w aplikacjach Python. Używana do tworzenia profesjonalnych aplikacji graficznych.

- **Biblioteki do grafiki i renderowania 3D:**

- **PyOpenGL**, wersja: 3.1.7,
Biblioteka umożliwiająca tworzenie grafiki 3D przy użyciu OpenGL w Pythonie. Wykorzystywana w aplikacjach, które wymagają renderowania grafiki komputerowej.
- **PyOpenGL-accelerate**, wersja: 3.1.7,
Przyspieszenie obliczeń OpenGL w Pythonie poprzez implementację krytycznych operacji w C. Stosowana w aplikacjach wymagających wysokiej wydajności w grafice 3D.
- **pyqtgraph**, wersja: 0.13.7,
Biblioteka do tworzenia dynamicznych wizualizacji danych i wykresów w czasie rzeczywistym. Znajduje zastosowanie w aplikacjach naukowych i inżynierskich.

- **Biblioteki do testowania i rozwoju oprogramowania:**

- **pytest**, wersja: 8.3.3,

Popularne narzędzie do testowania w Pythonie, umożliwiające łatwe pisanie testów jednostkowych. Używane w procesie ciągłej integracji i weryfikacji kodu.

- **pluggy**, wersja: 1.5.0,

Biblioteka do tworzenia i zarządzania punktami rozszerzeń w aplikacjach. Jest szeroko wykorzystywana w narzędziach testowych, takich jak pytest.

- **python-utils**, wersja: 3.9.1,

Zbiór różnych narzędzi i funkcji pomocniczych dla Pythona. Zawiera funkcje do obsługi typów danych, plików i wyjątków.

- **Wiele innych mniej istotnych bibliotek.**

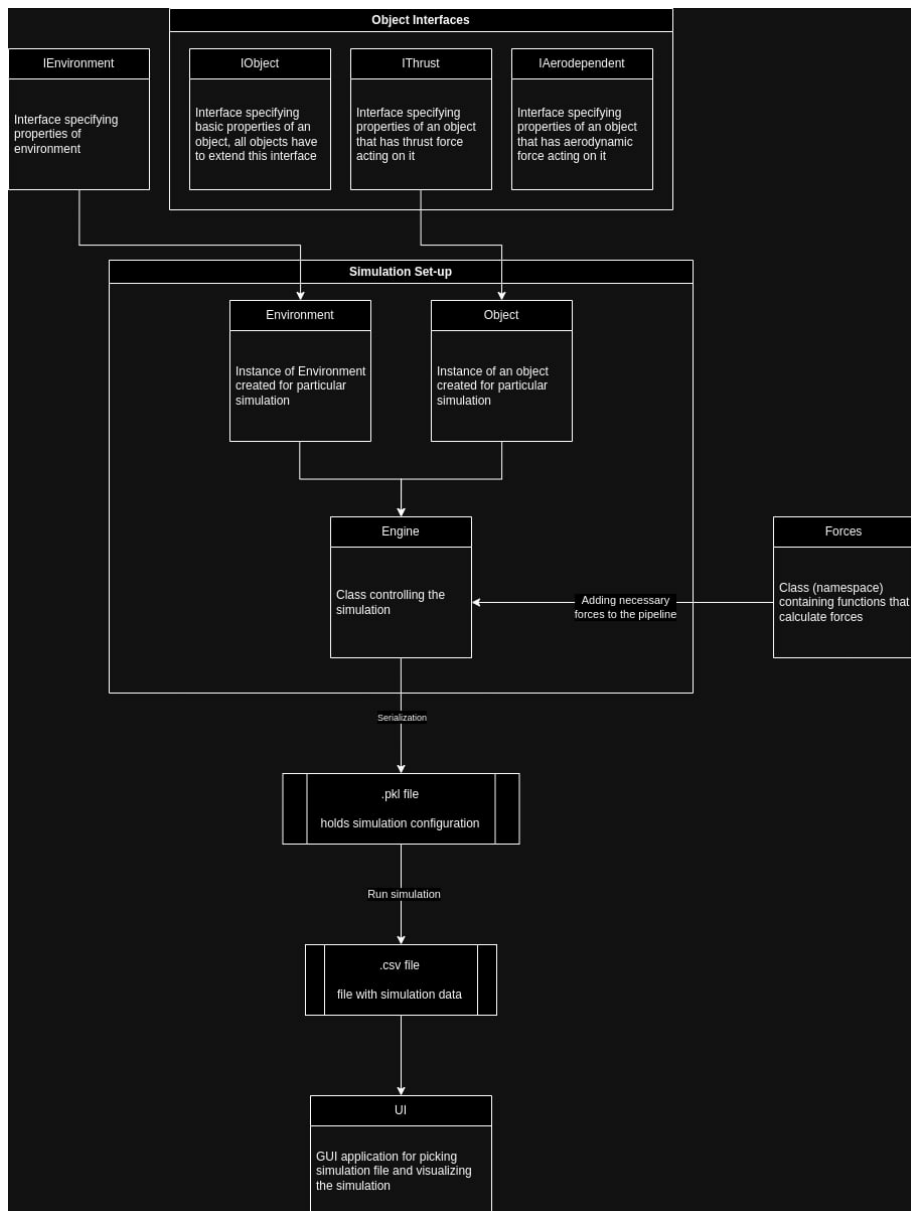
Wymienione powyżej biblioteki są głównymi narzędziami, z których korzystano podczas tworzenia modułu I pracy inżynierskiej. Oprócz nich użyto także wielu innych bibliotek pomocniczych, które ułatwiły pracę nad projektem. Są to między innymi biblioteki do obsługi plików, pracy z pakietami czy zarządzania danymi czasowymi.

2.4 Projekt silnika fizycznego.

System do symulacji lotów rakietowych będący głównym komponentem modułu I niniejszej pracy inżynierskiej został zaprojektowany zgodnie z założeniami opisanymi w rozdziale 2.1. W niniejszym rozdziale przedstawiono szczegóły architektury i implementacji silnika fizycznego.

2.4.1 Architektura systemu symulacji

System symulacji został zaprojektowany z wykorzystaniem paradygmatu programowania obiektowego, co pozwala na modularność i łatwą rozbudowę silnika o wykorzystanie nowych sił. Architektura systemu oparta jest na dobrze zdefiniowanych interfejsach i klasach, co przedstawiono na rysunku 2.11.



Rysunek 2.11. Architektura systemu symulacji, *Źródło: Opracowanie własne.*

W dalszej części tego rozdziału szczegółowo opisane zostały poszczególne elementy architektury systemu symulacji.

2.4.2 Interfejsy

Interfejsy definiują zbiór metod i atrybutów, które muszą być zaimplementowane przez klasy, które je dziedziczą. Dzięki nim obiekty, które powinny posiadać określone właściwości, mogą być traktowane jednolicie, co ułatwia zarządzanie nimi i zapewnia spójność w systemie. Aby obiekt mógł korzystać z konkretnego rodzaju sił, musi implementować odpowiednie interfejsy, które to umożliwiają. Dzięki temu, w łatwy sposób można rozszerzyć dany obiekt o parametry konieczne do symulacji nowych

sił. Pozwala to na elastyczność silnika i jego łatwe dostosowanie do różnych wersji symulacji.

W stworzonym na potrzeby tej pracy silniku fizycznym wyróżniamy następujące interfejsy:

‘IObject‘

Interfejs ‘IObject‘ jest podstawowym interfejsem dla wszystkich obiektów w symulacji. Przez obiekty rozumie się rakiety, na które oddziałują siły podczas symulacji. Interfejs ten definiuje podstawowe właściwości rakiety, które powinna ona posiadać bez względu na to, jakie siły będą ostatecznie na nią oddziaływały. W obecnej implementacji rakietę traktowana jest jako walec o jednolitej dystrybucji masy.

```
1 class IObject(ABC):
2     def __init__(self, name: str, object_model_path: str, height:
3         float, mass: float = 1,
4             radius: float = 0.1, position:
5                 NDArray[np.float64] = None, rotation:
6                     NDArray[np.float64] = None, velocity:
7                         NDArray[np.float64] = None,
8                             acceleration: NDArray[np.float64] = None,
9                                 angular_velocity: NDArray[np.float64] = None,
10                                    angular_acceleration: NDArray[np.float64] =
11                                        None):
12
13         self.config = Config()
14         self.name = name
15         self.position = position
16         self.velocity = velocity
17         self.acceleration = acceleration
18         self.angular_velocity = angular_velocity
19         self.angular_acceleration = angular_acceleration
20         self.rotation = rotation
21         self.mass = mass
22         self._height = height
23         self._radius = radius
24         self._inertia_tensor = self.calculate_inertia_tensor()
25         self.object_model = self.load_model(object_model_path)
```

W konstruktorze klasy implementującej ten interfejs inicjalizowane są następujące parametry:

- 'name': Nazwa obiektu.
- 'object_model_path': Ścieżka do modelu 3D obiektu.
- 'height': Wysokość obiektu.
- 'mass': Masa obiektu (domyślnie 1 kg).
- 'radius': Promień obiektu (domyślnie 0.1 m).
- 'position': Pozycja obiektu w przestrzeni (domyślnie 'None').
- 'velocity': Prędkość obiektu (domyślnie 'None').
- 'acceleration': Przyspieszenie obiektu (domyślnie 'None').
- 'angular_velocity': Kątowa prędkość obrotu obiektu (domyślnie 'None').
- 'angular_acceleration': Przyspieszenie kątowe obiektu (domyślnie 'None').

Dodatkowo, konstruktor inicjalizuje:

- 'self.config': Ogólna konfiguracja silnika.
- 'self._inertia_tensor': Tensor bezwładności obiektu.
- 'self.object_model': Model obiektu 3D.

Metody w interfejsie IObject:

- 'load_model(object_model_path)': Ładuje model 3D obiektu z podanej ścieżki. Model jest skalowany i translokowany do odpowiednich wymiarów.
- 'calculate_inertia_tensor()': Oblicza tensor bezwładności obiektu na podstawie jego masy i wymiarów.
- 'update(position=None, rotation=None, velocity=None)': Uaktualnia parametry obiektu: pozycję, rotację i prędkość.
- 'get_data()': Zwraca dane obiektu w formie tekstowej, zawierającą informacje o pozycji, rotacji, prędkości, przyspieszeniu.
- '__str__()': Zwraca reprezentację obiektu w formie tekstowej (pozycja, rotacja, prędkość, przyspieszenie).

'IThrust'

Interfejs 'IThrust' rozszerza interfejs 'IObject' o właściwości specyficzne dla obiektów posiadających silnik. Każdy obiekt, implementujący ten interfejs posiada wszystkie własności konieczne do obliczania sił ciągu, nań działających.

```

1 class IThrust(ABC):
2     def __init__(self, cot, engine_angle: NDArray[np.float64] =
      None):

```



```

3         self._cot = cot
4         self.engine_angle = engine_angle
5
6         if not isinstance(self, IObject):
7             raise RuntimeError("Object needs to extend IObject to
              be IThrust")

```

W konstruktorze deklarowane są następujące parametry:

- ‘cot’: Odległość środka ciągu (Center of Thrust) w osi Z od środka masy rakiety.
- ‘engine_angle’: Kąt nachylenia silnika względem lokalnego układu współrzędnych rakiety.

Jeśli obiekt nie rozszerza klasy ‘IObject’, zgłaszany jest błąd.

Opis metod:

- ‘mass_flow_rate(time: float)’: Metoda abstrakcyjna, która oblicza współczynnik przepływu masy w danym czasie.
- ‘exhaust_velocity(time: float)’: Metoda abstrakcyjna, która oblicza prędkość wylotową spalin w danym czasie.

‘IAerodependent’

Interfejs ‘IAerodependent’ rozszerza interfejs ‘IObject’ o właściwości związane z oddziaływaniem aerodynamicznym, takie jak powierzchnia czołowa, współczynnik oporu i inne. Dzięki implementacji tego interfejsu, obiekt posiada wszystkie własności i parametry pozwalające na obliczanie sił aerodynamicznych działających na ten obiekt.

Poniżej przedstawiamy kod tego interfejsu:

```

1 class IAerodependent(ABC):
2     def __init__(self, cop, angle_of_attack: NDArray[np.float64] =
      None, relative_velocity: NDArray[np.float64] = None,
      drag_coefficient: float =
      C.DRAG_COEFFICIENT_PARALLEL.value):
3         self._cop = cop
4         self.angle_of_attack = angle_of_attack
5         self.relative_velocity = relative_velocity
6         self._reference_area = self.calculate_reference_area()
7         self.drag_coefficient = drag_coefficient
8

```

```

9         if not isinstance(self, IObject):
10             raise RuntimeError("Object needs to extend IObject to
               be IAerodependent")
11
12     def calculate_reference_area(self: IObject):
13         reference_area = 0
14
15         for face in self.object_model.faces:
16             vertices = self.object_model.vertices[face]
17             projected_vertices = vertices[:, :2]
18
19             x = projected_vertices[:, 0]
20             y = projected_vertices[:, 1]
21
22             area = 0.5 * np.abs(np.dot(x, np.roll(y, 1)) -
               np.dot(y, np.roll(x, 1)))
23
24             reference_area += area
25
26         return reference_area

```

W konstruktorze deklarowane są następujące wartości parametrów:

- ‘cop’: Odległość środka ciśnienia (Center of Pressure) w osi Z od środka masy rakiety.
- ‘angle_of_attack’: Kąt natarcia rakiety względem kierunku przepływu powietrza.
- ‘relative_velocity’: Względna prędkość rakiety względem przepływającego powietrza.
- ‘drag_coefficient’: Współczynnik oporu (domyślnie wartość zdefiniowana w konfiguracji).

Opis metod:

- ‘calculate_reference_area()’: Oblicza powierzchnię czołową obiektu na podstawie modelu 3D.

‘IEnvironment’

Interfejs ‘IEnvironment’ definiuje właściwości środowiska symulacji, takie jak gęstość powietrza, prędkość dźwięku, temperatura, ciśnienie, wiatr i gradient wiatru.

```

1 from abc import ABC, abstractmethod

```

```

2
3 class IEnvironment(ABC):
4     @abstractmethod
5     def get_air_density(self, position=None) -> float:
6         raise NotImplementedError
7
8     @abstractmethod
9     def get_speed_of_sound(self, position=None) -> float:
10        raise NotImplementedError
11
12    @abstractmethod
13    def get_temperature(self, position=None) -> float:
14        raise NotImplementedError
15
16    @abstractmethod
17    def get_pressure(self, position=None) -> float:
18        raise NotImplementedError
19
20    @abstractmethod
21    def get_wind(self, position=None) -> list:
22        raise NotImplementedError
23
24    @abstractmethod
25    def get_wind_gradient(self, position=None) -> list:
26        raise NotImplementedError

```

Opis metod:

- ‘get_air_density(position=None)’: Metoda, która zwraca gęstość powietrza w zadanej pozycji. Jeśli ‘position’ nie zostanie podane, metoda użyje domyślnej wartości.
- ‘get_speed_of_sound(position=None)’: Metoda, która zwraca prędkość dźwięku w zadanej pozycji.
- ‘get_temperature(position=None)’: Metoda, która zwraca temperaturę w zadanej pozycji.
- ‘get_pressure(position=None)’: Metoda, która zwraca ciśnienie w zadanej pozycji.
- ‘get_wind(position=None)’: Metoda, która zwraca listę informacji o wietrze w danej pozycji.
- ‘get_wind_gradient(position=None)’: Metoda, która zwraca gradient wiatru

w danej pozycji.

2.4.3 Klasy

Klasy w silniku fizycznym odpowiadają za modelowanie sił oraz zapewniają działanie symulacji fizycznej. Główne klasy, które pełnią kluczowe role w systemie, to:

- **‘PhysicsEngine’**: Jest to najważniejsza klasa silnika, odpowiedzialna za zarządzanie przebiegiem symulacji. Oblicza siły działające na obiekty, aktualizuje ich pozycje, prędkości oraz momenty obrotowe, a także zapisuje wyniki do plików.
- **‘Forces’**: Klasa ta zawiera różne typy sił działających na obiekty w symulacji. W ramach tej klasy znajdują się konkretne implementacje sił takich jak *Drag*, *Gravity* oraz *Thrust*, które są używane przez system do obliczeń. Więcej o implementacji tych sił opisano w podrozdziale 2.5.

2.4.4 Proces symulacji.

Proces symulacji fizycznej lotu rakiety jest wieloetapowy i składa się z dwóch głównych procesów: konfiguracji oraz przeprowadzenia symulacji. Poniżej opisane szczegółowo zostały oba te procesy.

2.4.5 Konfiguracja symulacji (Setup)

Zanim symulacja zostanie przeprowadzona, tworzony jest plik zawierający konfigurację symulacji. W tym pliku zapisywane są informacje o tym, jakie parametry powinna mieć symulacja.

Pierwszym krokiem jest stworzenie obiektów rakiety, środowiska oraz silnika fizycznego:

```
1 object = Rocket()  
2 environment = Environment()  
3 physics_engine = PhysicsEngine(object, environment, 0.1)
```

W tej części kodu tworzymy:

- **object** - instancję klasy *Rocket*, która reprezentuje raketę,
- **environment** - instancję klasy *Environment*, która definiuje środowisko symulacji,

- `physics_engine` - instancję klasy `PhysicsEngine`, która będzie zarządzać całą symulacją fizyczną.

Po utworzeniu silnika fizycznego, dodajemy do niego różne siły, które będą działać na raketę:

```
1 physics_engine.add_force(Forces.gravity)
2 physics_engine.add_force(Forces.thrust)
3 physics_engine.add_force(Forces.drag)
```

W tej części kodu dodajemy trzy siły:

- `Forces.gravity` - siła grawitacji,
- `Forces.thrust` - siła ciągu,
- `Forces.drag` - siła oporu powietrza.

Na końcu zapisujemy stan silnika fizycznego do pliku przy użyciu `pickle`:

```
1 config = Config()
2 path =
    os.path.join(config["SIMULATION"]["simulation_files_folder_path"],
        object.name)
3 save(physics_engine, path, "physics_engine.pkl")
```

W efekcie działania tego skryptu powstaje konfiguracja symulacji zawierająca informacje o rakiecie, środowisku i siłach, które powinny działać na raketę w trakcie trwania symulacji.

2.4.6 Symulacja

Po zapisaniu plików z konfiguracją, wczytujemy je do skryptu `simulation.py` i uruchamiamy symulację. W tym celu używamy poniższego kodu:

```
1 simulation_name = "name_of_simulation"
2
3 config = Config()
4 path =
    os.path.join(config["SIMULATION"]["simulation_files_folder_path"],
        simulation_name, "physics_engine.pkl")
5
6 with open(path, "rb") as file:
7     physics_engine = pickle.load(file)
8
9 physics_engine.start()
```

W pierwszej kolejności określamy nazwę symulacji, a następnie wczytujemy obiekt `physics_engine` z pliku. Wczytany obiekt silnika symulacji fizycznej jest uruchamiany za pomocą metody `start()`, która inicjuje symulację. Funkcja ta:

- wczytuje wcześniej zapisany stan symulacji,
- rozpoczyna główną pętlę symulacyjną,
- zapisuje kolejne wyniki do pliku.

Działanie pętli symulacyjnej silnika fizycznego

Pętla symulacyjna silnika `PhysicsEngine` jest odpowiedzialna za zarządzanie całym przebiegiem symulacji. Działa w sposób iteracyjny, krok po kroku, aktualizując stan obiektów w symulacji w czasie rzeczywistym. Kluczowym elementem tego procesu jest kontrolowanie czasu symulacji oraz zarządzanie siłami działającymi na obiekty.

Główna pętla symulacyjna działa w następujący sposób:

1. **Aktualizacja czasu:** Po każdym kroku, zmieniane są dwie zmienne: `simulation_time` (całkowity czas symulacji) oraz `simulation_tick` (liczba iteracji).
2. **Obliczanie sił:** system oblicza siły działające na obiekt poprzez wywołanie metod obliczających wynikową siłę i moment obrotowy. Zgodnie z wcześniej dodanymi siłami, dla każdego obiektu w symulacji obliczane są siły, które wpływają na jego ruch.
3. **Aktualizacja stanu obiektu:** Na podstawie obliczonych sił, aktualizowany jest stan obiektu: jego pozycja, prędkość, przyspieszenie oraz orientacja (rotacja). W tym kroku uwzględniane są zmiany zarówno w ruchu translacyjnym, jak i obrotowym.
4. **Sprawdzanie warunków zakończenia:** Pętla symulacyjna sprawdza, czy rakieta nie osiągnęła powierzchni ziemi (czyli gdy jej wysokość jest mniejsza od zera), lub czy nie osiągnięto maksymalnego czasu symulacji. Jeśli jeden z tych warunków jest spełniony, symulacja zostaje zakończona.
5. **Zapis danych:** Po każdej iteracji pętli, dane o stanie obiektu są zapisywane do pliku, aby umożliwić późniejszą analizę wyników symulacji.

2.4.7 Zapisywanie danych symulacji

W trakcie symulacji, po każdym kroku, zapisywane są dane o stanie obiektów w symulacji. Dane te zawierają informacje o pozycji, prędkości, przyspieszeniu, rotacji

oraz prędkości kątowej obiektów. Zapisywane są one w formie plików CSV, które zawierają informacje o stanie obiektów w danym kroku czasowym. Dzięki temu, po zakończeniu symulacji, możliwe jest analizowanie wyników oraz generowanie wizualizacji trajektorii lotu rakiety.

Pliki zapisywane są w folderze odpowiednim dla danej symulacji, w formacie CSV, który zawiera następujące kolumny:

- `sim_tick` - Licznik symulacji (numer kroku symulacji)
- `sim_time` - Czas symulacji (w sekundach), czyli czas od rozpoczęcia symulacji
- `mass` - Masa rakiety (w kilogramach)
- `pos_x` - Pozycja rakiety na osi X (w metrach)
- `pos_y` - Pozycja rakiety na osi Y (w metrach)
- `pos_z` - Pozycja rakiety na osi Z (w metrach)
- `rot_x` - Rotacja rakiety wokół osi X (w radianach)
- `rot_y` - Rotacja rakiety wokół osi Y (w radianach)
- `rot_z` - Rotacja rakiety wokół osi Z (w radianach)
- `vel_x` - Prędkość rakiety na osi X (w m/s)
- `vel_y` - Prędkość rakiety na osi Y (w m/s)
- `vel_z` - Prędkość rakiety na osi Z (w m/s)
- `acc_x` - Przyspieszenie rakiety na osi X (w m/s^2)
- `acc_y` - Przyspieszenie rakiety na osi Y (w m/s^2)
- `acc_z` - Przyspieszenie rakiety na osi Z (w m/s^2)
- `ang_vel_x` - Prędkość kątowa wokół osi X (w rad/s)
- `ang_vel_y` - Prędkość kątowa wokół osi Y (w rad/s)
- `ang_vel_z` - Prędkość kątowa wokół osi Z (w rad/s)
- `ang_acc_x` - Przyspieszenie kątowe wokół osi X (w rad/s^2)
- `ang_acc_y` - Przyspieszenie kątowe wokół osi Y (w rad/s^2)
- `ang_acc_z` - Przyspieszenie kątowe wokół osi Z (w rad/s^2)
- `engine_angle_x` - Kąt wychylenia silnika na osi X (w radianach)
- `engine_angle_y` - Kąt wychylenia silnika na osi Y (w radianach)

Zapisywanie tych danych, jest kluczowym elementem symulacji, ponieważ pozwala na analizę wyników oraz generowanie wizualizacji trajektorii lotu rakiety.

2.4.8 Uzasadnienie przyjętego rozwiązania

Implementacja wyżej przedstawionego rozwiązania pozwala na uzyskanie wielu benefitów, pomagających osiągnąć poczynione założenia. Są to między innymi:

Modułowość

Jednym z kluczowych założeń przy projektowaniu systemu była jego modułowość. Każda z głównych klas, takich jak `PhysicsEngine`, `Forces` czy `Rocket`, pełni dobrze zdefiniowaną rolę, co umożliwia łatwe modyfikowanie i rozwijanie projektu. Dzięki temu każdą część systemu można rozwijać niezależnie, bez wpływu na pozostałe komponenty. Pozwala to na współpracę między współautorami, jasność implementacji oraz łatwe testowanie i debugowanie.

Łatwość dodawania nowych sił

Projekt został zaprojektowany z myślą o prostocie rozszerzania o nowe siły. Obecność klasy `Forces` z różnymi zdefiniowanymi siłami, takimi jak `gravity`, `thrust` i `drag`, pozwala na łatwe dodanie nowych sił, np. oporu magnetycznego, sił aerodynamicznych specyficznych dla różnych kształtów rakiet, czy nawet sił wynikających z interakcji z innymi obiektami w symulacji.

Nowe siły mogą być łatwo zaimplementowane jako funkcje, które przyjmują obiekt rakiety i środowisko jako argumenty, co zapewnia spójność interfejsu. Ponadto mechanizm dodawania sił za pomocą metody `add_force` pozwala na ich dynamiczne przypisywanie i usuwanie w trakcie symulacji. Jeżeli nowe siły wymagają dodania do obiektu rakiety nowych parametrów, w łatwy sposób można rozwinąć jeden z istniejących interfejsów lub dodać zupełnie nowy interfejs umożliwiający tworzenie nowej klasy sił.

Innym dużym atutem jest możliwość wielokrotnej implementacji tej samej siły w różnych wariantach, co pozwala na porównanie różnych modeli sił i ich wpływu, dopracowywanie modelu fizyki oraz optymalizację symulacji.

Łatwość zmiany parametrów

Kolejnym istotnym atutem tego rozwiązania jest możliwość łatwej zmiany parametrów fizycznych oraz konfiguracji symulacji. Wartości takie jak krok czasowy, maksymalny czas symulacji, czy parametry sił (np. współczynniki oporu powietrza) mogą być modyfikowane w prosty sposób, bez konieczności ingerencji w kod samego silnika. Dzięki temu użytkownicy mogą łatwo dostosować symulację do swoich po-

trzeb, na przykład zmieniając warunki środowiskowe lub zachowanie rakiety. Przydatne jest to do wielokrotnego symulowania lotów rakiety w zmiennych warunkach. Sprzyja temu także możliwość tworzenia konfiguracji symulacyjnych.

Integracja z algorytmami sztucznej inteligencji

System jest przystosowany do wykorzystania w kontekście algorytmów sztucznej inteligencji. Dzięki temu, że cała symulacja jest oparta na obiektach i metodach z jasno zdefiniowanymi interfejsami, łatwo jest połączyć ją z systemami uczącymi się.

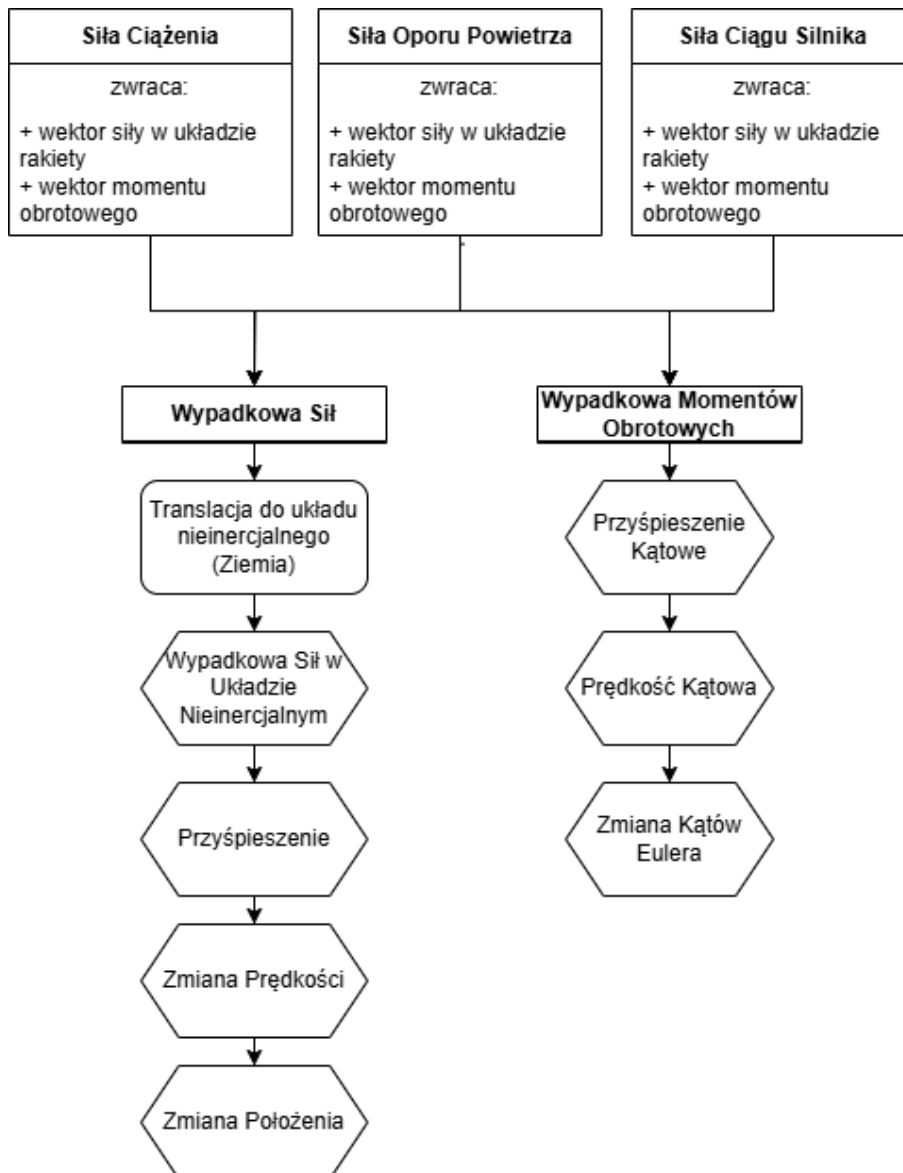
`PhysicsEngine` oraz `Rocket` dostarczają niezbędnych danych o ruchu obiektów (np. prędkości, przyspieszenia, pozycji), które mogą być wykorzystywane przez modele AI do dalszego uczenia się i optymalizacji działań.

Inną własnością ułatwiającą integrację z modelami ML jest krokowość silnika. Po każdym kroku (tick'u), model ma możliwość analizy stanu obiektów i zmiany dostępnych dla niego parametrów, co wpływa na stan obiektów w symulacji. W ten sposób system przystosowany jest do pracy w trybie uczenia modelu sztucznej inteligencji.

2.5 Model sił działających na raketę

Ważną częścią działanie silnika fizycznego jest implementacja sił fizycznych. W przypadku symulacji lotu rakiety, kluczowym elementem są siły, które oddziałują na raketę oraz sposób, w jaki owe siły są składane i jak wyliczany jest ich wpływ na ruch obiektu. W niniejszym podrozdziale opisane zostały te dwa kluczowe zagadnienia.

Na rysunku 2.12 przedstawiona została wysokopoziomowa architektura obliczeń sił w silniku fizycznym.



Rysunek 2.12. Schemat działania sił w symulacji, Źródło: Opracowanie własne.

2.5.1 Funkcje pomocnicze

Funkcje pomocnicze stanowią podstawę obliczeń, umożliwiając między innymi transformację wektorów między lokalnym i globalnym układem współrzędnych. Transformacje te są kluczowe, ponieważ wiele sił, takich jak opór powietrza czy ciąg silnika, jest obliczanych w lokalnym układzie rakiety, a następnie przeliczanych do globalnego układu inercyjnego, w którym odbywają się końcowe obliczenia ruchu.

Funkcja tworząca macierz obrotu

Funkcja `euler_to_rotation_matrix` generuje macierz obrotu na podstawie kątów Eulera (ϕ_1 , ϕ_2 , ϕ_3), odpowiadających rotacjom wokół kolejno osi X, Y oraz Z.

```

1 def euler_to_rotation_matrix(angles):
2     fi1, fi2, fi3 = angles
3
4     Rx = np.array([
5         [1, 0, 0],
6         [0, np.cos(fi1), -np.sin(fi1)],
7         [0, np.sin(fi1), np.cos(fi1)]
8     ])
9
10    Ry = np.array([
11        [np.cos(fi2), 0, np.sin(fi2)],
12        [0, 1, 0],
13        [-np.sin(fi2), 0, np.cos(fi2)]
14    ])
15
16    Rz = np.array([
17        [np.cos(fi3), -np.sin(fi3), 0],
18        [np.sin(fi3), np.cos(fi3), 0],
19        [0, 0, 1]
20    ])
21
22    R = Rz @ Ry @ Rx
23    return R

```

Macierz obrotu jest wynikiem sekwencyjnego mnożenia macierzy rotacji R_x , R_y oraz R_z . Dzięki temu możliwe jest precyzyjne odwzorowanie orientacji rakiety w przestrzeni trójwymiarowej, co jest niezbędne do transformacji sił i prędkości.

Transformacja układów współrzędnych

Funkcje `transform_to_global` i `transform_to_local` służą do konwersji wektorów między lokalnym układem współrzędnych rakiety a globalnym układem inercyjnym.

```

1 def transform_to_global(local_vector, angles):
2     R = euler_to_rotation_matrix(angles)
3     return R @ local_vector

```

Funkcja `transform_to_global` używa macierzy rotacji R , aby przekształcić wektor z układu lokalnego do globalnego. Wektory lokalne, takie jak ciąg silnika, są orientowane zgodnie z orientacją rakiety w przestrzeni.

```

1 def transform_to_local(global_vector, angles):

```

```

2   R = euler_to_rotation_matrix(angles)
3   return R.T @ global_vector

```

Analogicznie, funkcja `transform_to_local` wykorzystuje transponowaną macierz rotacji, aby przekształcić wektor globalny na układ lokalny rakiety. Przykładem zastosowania jest obliczenie siły grawitacji w układzie lokalnym.

2.5.2 Siły działające na raketę

W symulacji uwzględniono cztery główne siły: grawitację, opór powietrza, ciąg silnika oraz siłę nośną. Każda z tych sił jest obliczana na podstawie aktualnego stanu rakiety i środowiska, a ich wypadkowa wpływa na dynamikę ruchu rakiety.

Siła grawitacji

Siła grawitacji działa pionowo w dół, zgodnie z osią Z globalnego układu współrzędnych, i wynika z masy rakiety oraz przyspieszenia grawitacyjnego. Funkcja `gravity` oblicza wartość siły w układzie lokalnym rakiety, przekształcając ją z układu globalnego:

```

1 def gravity(object: IObject, environment: IEnvironment, time:
    float):
2     global_gravity = np.array([0, 0, -object.mass *
        Constants.GRAVITATIONAL_ACCELERATION.value], dtype=float)
3     local_gravity = transform_to_local(global_gravity,
        object.rotation)
4     torque = np.array([0, 0, 0], dtype=float)
5     return local_gravity, torque

```

Opis działania funkcji:

- Obliczana jest wartość siły grawitacji w układzie globalnym jako iloczyn masy rakiety i przyspieszenia grawitacyjnego.
- Wektor siły grawitacji jest przekształcany na układ lokalny, aby uwzględnić aktualną orientację rakiety.
- Ponieważ siła grawitacji działa na środek masy, moment obrotowy wynosi zero.

Siła oporu powietrza

Siła oporu powietrza jest przeciwnie skierowana do wektora prędkości rakiety względem otaczającego ją powietrza. Wartość siły jest proporcjonalna do kwadratu tej

prędkości. Funkcja `drag` oblicza zarówno wartość siły, jak i moment obrotowy wynikający z jej działania:

```

1 def drag(object: IAerodependent, environment: IEnvironment, time:
  float):
2     if (object.velocity == np.array([0, 0, 0], dtype=float)).all():
3         return np.array([0, 0, 0], dtype=float), np.array([0, 0,
4             0], dtype=float)
5
6     Cd = object.drag_coefficient
7     relative_velocity = object.relative_velocity
8     reference_area = object.reference_area
9     drag_mag = 0.5 * environment.get_air_density() *
10         np.linalg.norm(relative_velocity) ** 2 * Cd * reference_area
11     drag_v = -drag_mag * (relative_velocity /
12         np.linalg.norm(relative_velocity))
13     drag_l = transform_to_local(drag_v, object.rotation)
14     torque = np.cross(drag_l, np.array([0, 0, -object.cop],
15         dtype=float))
16     return drag_v, torque

```

Opis działania funkcji:

- Wartość siły oporu jest obliczana na podstawie równania

$$F_d = \frac{1}{2} \rho v^2 C_d A,$$

gdzie ρ to gęstość powietrza, v to prędkość względna, C_d to współczynnik oporu, a A to powierzchnia przekroju poprzecznego.

- Kierunek siły oporu jest przeciwny do wektora kierunku prędkości względnej rakiety względem powietrza.
- Siła i moment obrotowy są przeliczane na układ lokalny rakiety.

Siła ciągu silnika

Siła ciągu jest generowana przez spaliny wydostające się z silnika rakiety. Jej wartość zależy od masowego natężenia przepływu gazów oraz prędkości wylotowej. Funkcja `thrust` oblicza siłę ciągu i moment obrotowy:

```

1 def thrust(object: IThrust, environment: IEnvironment, time:
  float):

```

```

2 thrust_mag = object.mass_flow_rate(time) *
    object.exhaust_velocity(time)
3 thrust_v = np.array([0, 0, thrust_mag], dtype=float)
4 thrust_g = transform_to_global(thrust_v, object.engine_angle)
5 torque_l = np.cross(thrust_g, np.array([0, 0, -object.cot],
    dtype=float))
6 return thrust_g, torque_l

```

Opis działania funkcji:

- Wielkość siły ciągu jest obliczana jako iloczyn natężenia przepływu masy gazów i ich prędkości wylotowej.
- Wektor siły ciągu jest przekształcany z układu lokalnego (związanego z silnikiem) na układ globalny, uwzględniający aktualny kąt nachylenia silnika względem rakiety.
- Moment obrotowy wynika z przesunięcia środka ciągu (*Center of Thrust, CoT*) względem środka masy rakiety.

Siła nośna

Siła nośna powstaje w wyniku oddziaływania powietrza na powierzchnię rakiety, której kształt oraz orientacja wpływają na wartość tej siły. Jest obliczana na podstawie prędkości rakiety względem powietrza, współczynnika nośności oraz powierzchni, na którą działa ta siła. Funkcja `lift` oblicza siłę nośną oraz moment obrotowy:

```

1 def lift(object: IAerodependent, environment: IEnvironment, time:
    float) -> (
2     NDArray[np.float64], NDArray[np.float64]):
3     if (object.velocity == np.array([0, 0, 0], dtype=float)).all():
4         return np.array([0, 0, 0], dtype=float), np.array([0, 0,
5             0], dtype=float)
6
7     Cl = object.lift_coefficient
8     relative_velocity = object.relative_velocity
9     reference_area = object.reference_area
10    air_density = environment.get_air_density()
11    lift_mag = 0.5 * air_density * np.linalg.norm(
12        relative_velocity) ** 2 * Cl * reference_area
13    velocity_unit = relative_velocity /
14        np.linalg.norm(relative_velocity)
15    lift_direction = np.cross(velocity_unit, np.array([0, 0, 1]))
16    if np.linalg.norm(lift_direction) == 0:

```

```

15         lift_direction = np.array([0, 1, 0])
16         lift_v = lift_mag * lift_direction /
            np.linalg.norm(lift_direction)
17         lift_l = transform_to_local(lift_v, object.rotation)
18         torque = np.cross(lift_l, np.array([0, 0, -object.cop],
            dtype=float))
19
20     return lift_l, torque

```

Opis działania funkcji:

- Siła nośna jest obliczana na podstawie wzoru:

$$L = 0.5 \cdot \rho \cdot v^2 \cdot A \cdot C_L,$$

gdzie ρ to gęstość powietrza, v to norma prędkości względnej rakiety, A to powierzchnia referencyjna rakiety, a C_L to współczynnik nośności.

- Wektor prędkości względnej rakiety jest normalizowany, aby uzyskać jednostkowy wektor prędkości (`velocity_unit`).
- Kierunek siły nośnej jest wyznaczany jako wektor prostopadły do prędkości rakiety, który jest obliczany za pomocą iloczynu wektorowego (`lift_direction`).
- Jeśli wektor siły nośnej ma zerową normę (co może się zdarzyć, gdy prędkość rakiety wynosi zero), przypisujemy domyślny kierunek wzdłuż osi Y (`[0, 1, 0]`).
- Siła nośna `lift_v` jest obliczana jako iloczyn wielkości siły nośnej (`lift_mag`) i znormalizowanego kierunku (`lift_direction`).
- Siła nośna i moment obrotowy są przekształcane na układ lokalny rakiety.

2.5.3 Obliczanie zmian w ruchu rakiety

Funkcja `compute_change` odpowiada za aktualizację parametrów dynamicznych rakiety, bazując na wypadkowych siłach i momentach pędu. Zawiera obliczenia zarówno dla ruchu postępowego, jak i obrotowego:

```

1 def compute_change(self, force: NDArray[np.float64], torque:
    NDArray[np.float64]):
2     global_force = transform_to_global(force, self.object.rotation)
3     self.object.acceleration = global_force / self.object.mass
4     velocity_change = self.object.acceleration * self.time_step
5     self.object.velocity += velocity_change
6

```

```

7 position_change = self.object.velocity * self.time_step
8 self.object.position += position_change
9
10 if isinstance(self.object, IAerodependent) and
    isinstance(self.object, IObject):
11     self.object.angle_of_attack =
        self.calculate_angle_of_attack()
12     self.object.relative_velocity = self.object.velocity -
        self.environment.get_wind()
13     self.object.drag_coefficient = (
14         C.DRAG_COEFFICIENT_PARALLEL.value *
            np.cos(self.object.angle_of_attack) ** 2
15         + C.DRAG_COEFFICIENT_PERPENDICULAR.value *
            np.sin(self.object.angle_of_attack) ** 2
16     )
17
18 self.object.angular_acceleration =
    np.linalg.inv(self.object.inertia_tensor) @ torque
19 self.object.angular_velocity +=
    self.object.angular_acceleration * self.time_step
20
21 wx, wy, wz = self.object.angular_velocity
22 phi, theta, psi = self.object.rotation
23
24 dot_phi = wx + (np.tan(theta) * np.sin(phi) * wy + np.cos(phi)
    * wz)
25 dot_theta = np.cos(phi) * wy - np.sin(phi) * wz
26 dot_psi = np.sin(phi) / np.cos(theta) * wy + np.cos(phi) * wz
27
28 self.object.rotation[0] += dot_phi * self.time_step
29 self.object.rotation[1] += dot_theta * self.time_step
30 self.object.rotation[2] += dot_psi * self.time_step

```

W ramach tej funkcji wykonywane są następujące operacje:

- **Obliczenia ruchu postępowego:** Na podstawie siły wypadkowej obliczane są przyspieszenie, zmiana prędkości oraz przesunięcie pozycji rakiety.
- **Obliczenia ruchu obrotowego:** Bazując na momencie pędu, wyznaczane są przyspieszenie kątowe, prędkość kątowna i zmiana orientacji rakiety w przestrzeni.
- **Korekcje aerodynamiczne:** Jeśli obiekt implementuje interfejs IAerodependent,

wyliczane są dodatkowe parametry, takie jak kąt natarcia, względna prędkość wiatru czy współczynnik oporu aerodynamicznego.

Kąt natarcia rakiety

W trakcie aktualizacji stanu rakiety wywoływana jest funkcja `calculate_angle_of_attack`, która oblicza kąt natarcia, czyli kąt między wektorem prędkości rakiety a osią Z w układzie inercyjnym. Implementacja tej funkcji wygląda następująco:

```
1 def calculate_angle_of_attack(self):
2     object_velocity = self.object.velocity
3     wind_velocity = self.environment.get_wind()
4     relative_velocity = object_velocity - wind_velocity
5     relative_velocity_unit = relative_velocity /
6         np.linalg.norm(relative_velocity)
7
8     reference_axis_local = np.array([0, 0, 1], dtype=float)
9     reference_axis_global =
10         transform_to_global(reference_axis_local,
11                             self.object.rotation)
12
13     cos_theta = np.dot(relative_velocity_unit,
14                         reference_axis_global)
15     angle_of_attack = np.arccos(np.clip(cos_theta, -1.0, 1.0))
16
17     return angle_of_attack
```

Obliczanie kąta natarcia ma kluczowe znaczenie dla poprawnego modelowania sił aerodynamicznych, ponieważ wpływa bezpośrednio na wyznaczenie współczynnika oporu oraz siły nośnej działającej na raketę.

Uwagi dotyczące układów współrzędnych

Warto zauważyć, że wszystkie obliczenia dotyczące ruchu postępowego są prowadzone w układzie nieinercyjnym związanym z Ziemią, podczas gdy obliczenia ruchu obrotowego wykonywane są w układzie współrzędnych związanym z raketą. Dzięki temu symulacja uwzględnia złożoność dynamiki obiektów ruchomych w przestrzeni trójwymiarowej.

2.5.4 Podsumowanie

Przedstawiony powyżej model zaimplementowanej fizyki ruchu rakiety w symulacji lotu, jest modelem uproszczonym, nie biorącym pod uwagę wielu czynników. Spowodowane jest to niewystarczającymi zasobami (czasem, wiedzą, mocą obliczeniową). Model ten jednak dzięki modułowej architekturze silnika może być w łatwy sposób rozbudowany, tak aby bardziej realnie odzwierciedlał warunki panujące podczas rzeczywistych lotów rakietowych. Na potrzeby niniejszej pracy inżynierskiej zaimplementowana fizyka wydaje się być wystarczającą.

2.6 Moduł wizualizacji symulacji.

Moduł wizualizacji symulacji jest odpowiedzialny za generowanie wizualizacji lotów rakiety na podstawie danych zapisanych w trakcie symulacji. Struktura zapisywanych danych opisana została w sekcji 2.4.7. Wizualizacja jest generowana w formie animacji, która przedstawia trajektorię lotu rakiety w przestrzeni trójwymiarowej oraz zmiany kluczowych parametrów w czasie.

Moduł wizualizacji stworzony został za pomocą biblioteki `PyQt`, która umożliwia generowanie dynamicznie zmieniających się interfejsów graficznych. Składa się on z dwóch głównych okien aplikacji: okna głównego oraz okna wizualizacji.

Uruchomienie modułu wizualizacji polega na uruchomieniu skryptu `visualisation.py`.

2.6.1 Okno główne

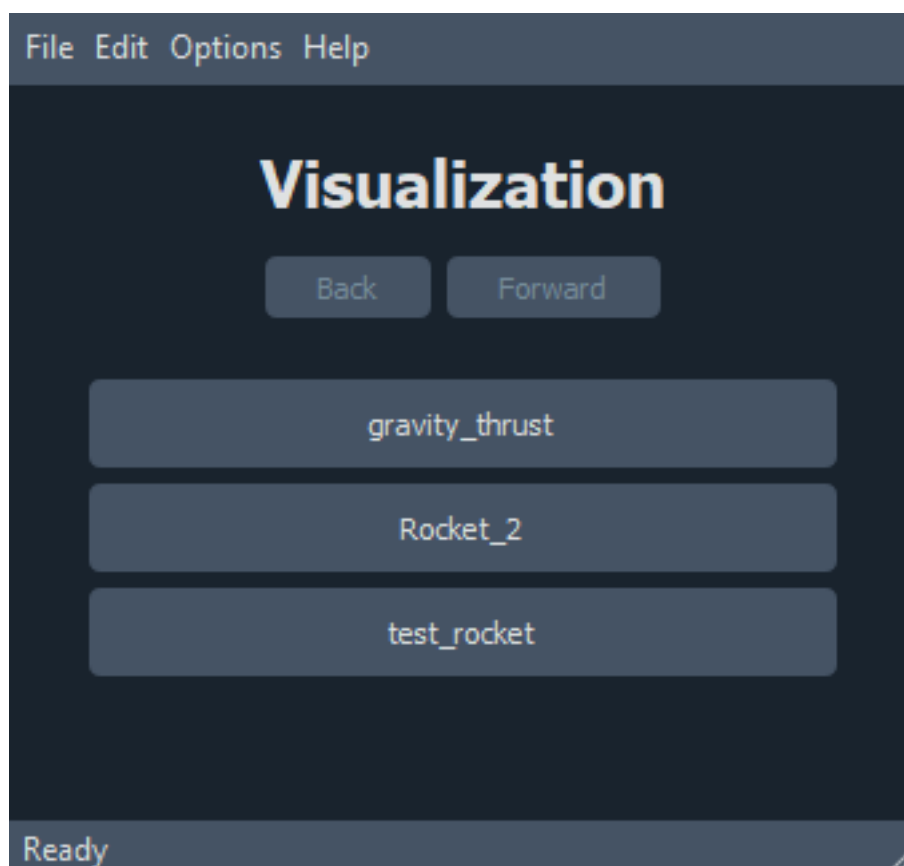
Okno główne aplikacji zawiera interfejs użytkownika, który umożliwia wybranie konfiguracji symulacji, a następnie wybranie konkretnej symulacji przeprowadzonej w ramach tej konfiguracji. Interfejs użytkownika pozwala na przełączanie się pomiędzy oknami wizualizacji oraz wybór symulacji do przeprowadzenia.

Widoki okna głównego aplikacji zarządzane są przez komponent nazwany picker, który przełącza widoki w zależności od wybranych przez użytkownika opcji.

Przedstawiając więc strukturę okna głównego, można wyróżnić następujące komponenty:

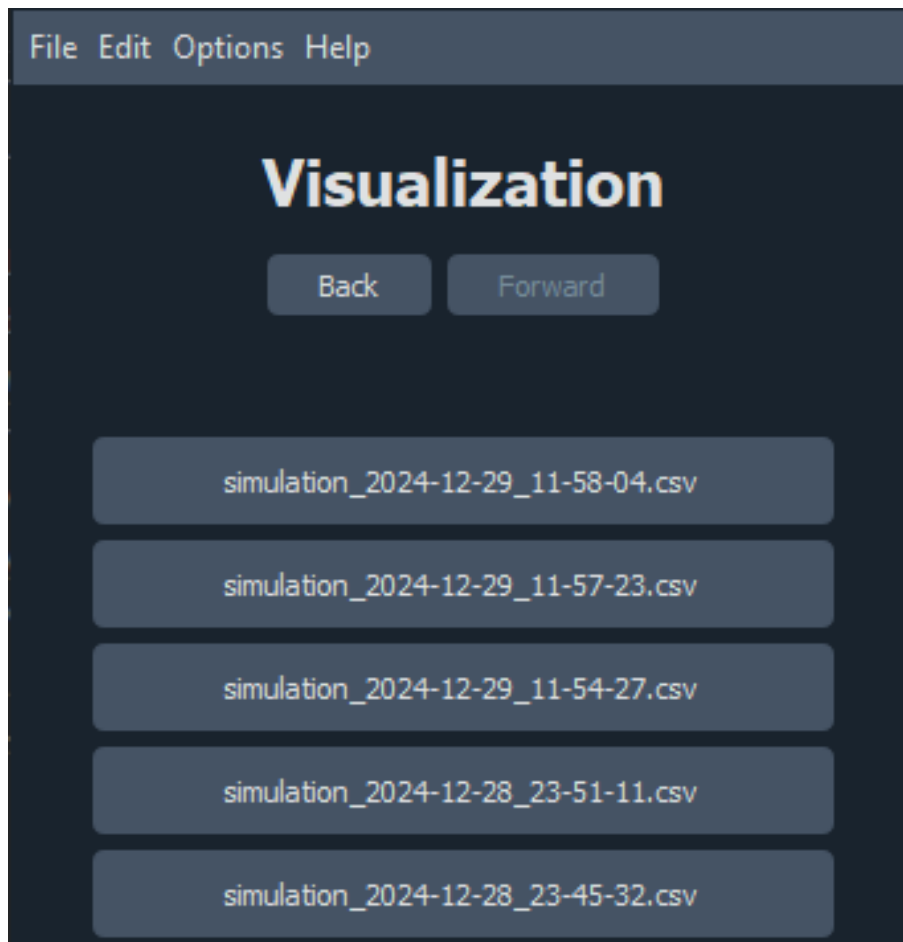
1. **Wybór konfiguracji symulacji:** Użytkownik ma możliwość wyboru jednej z dostępnych konfiguracji symulacji, które zostały zdefiniowane w plikach konfiguracyjnych. Wybór konfiguracji powoduje wyświetlenie listy dostępnych sy-

mulacji przeprowadzonych w ramach tej konfiguracji. Funkcjonalność ta obsługiwana jest przez widget `folder_picker`. Wygląd tego widoku przedstawiony jest na rysunku 2.13



Rysunek 2.13. Okno główne aplikacji - wybór konfiguracji, *Źródło: Opracowanie własne.*

2. **Wybór symulacji:** Po wybraniu konfiguracji, użytkownik dostaje dostęp do plików zapisanych w ramach tej konfiguracji. Może dokonać wyboru jednej z symulacji, której wyniki chce zobaczyć. Tą funkcjonalność umożliwia widget `file_picker`. Wygląd tego widoku przedstawiony jest na rysunku 2.14



Rysunek 2.14. Okno wyboru symulacji, *Źródło: Opracowanie własne.*

2.6.2 Okno wizualizacji

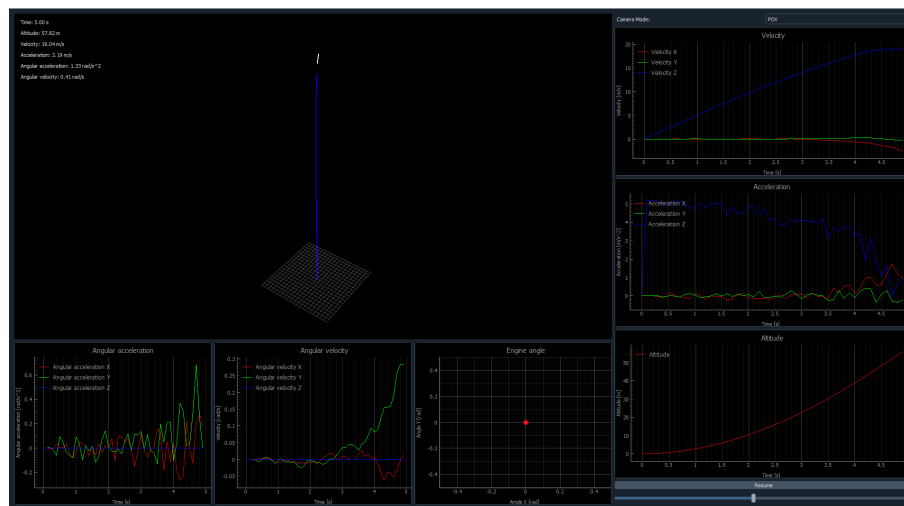
Okno wizualizacji jest odpowiedzialne za generowanie animacji trajektorii lotu oraz zmian parametrów rakiety w czasie. Wizualizacja jest generowana na podstawie danych pobranych z pliku CSV zapisanego w trakcie symulacji.

Na okno wizualizacji składa się kilka głównych komponentów:

1. **Wizualizacja trajektorii lotu:** Głównym elementem wizualizacji jest animacja trajektorii lotu rakiety w przestrzeni trójwymiarowej. Na wizualizacji przedstawione są kolejne klatki animacji, które pokazują poruszający się model rakiety w przestrzeni oraz trajektorię lotu.
2. **Wykresy zmian parametrów ruchu obrotowego:** Pod wizualizacją trajektorii lotu znajdują się dwa wykresy zmian parametrów ruchu obrotowego - prędkości oraz przyspieszenia kątownego wokół osi X, Y i Z. Pozwalają one na analizę zmian orientacji rakiety w czasie.

3. **Wykresy zmian parametrów ruchu postępowego:** Po prawej stronie wizualizacji znajdują się wykresy zmian parametrów ruchu postępowego - prędkości oraz przyspieszenia rakiety na osiach X, Y i Z. Dzięki nim użytkownik może analizować zmiany tych parametrów w czasie.
4. **Wykres zmian wysokości:** W dolnym prawym rogu wizualizacji znajduje się wykres zmian wysokości rakiety w czasie. Pozwala on na analizę wysokości rakiety w trakcie lotu.
5. **Wykres kąta nachylenia silnika rakiety:** Poniżej okna wizualizacji znajduje się wykres zmian kąta nachylenia silnika rakiety w danym kroku symulacji. Wizualizacja tego parametru jest istotna, ponieważ to właśnie kąt nachylenia silnika wpływa na kierunek ciągu i jest sterowany przez model AI.
6. **Przycisk zatrzymujący animację oraz suwak z czasem animacji:** W prawym dolnym rogu wizualizacji znajduje się przycisk, który pozwala na zatrzymanie symulacji w dowolnym momencie. Tam też znajduje się suwak, umożliwiający przewijanie animacji w czasie. Jest to przydatne narzędzie do analizy lotu rakiety w wybranym momencie.

Wszystkie opisane powyżej komponenty wizualizacji przedstawione są na rysunku 2.15.



Rysunek 2.15. Okno wizualizacji symulacji, Źródło: Opracowanie własne.

2.7 Dokładność symulacji.

Badanie dokładności symulacji jest skomplikowanym zagadnieniem, które wymaga przeprowadzenia wielu praktycznych testów oraz analizy wyników, co leży poza za-

kresem niniejszej pracy inżynierskiej. Jest to bowiem proces wymagający stworzenia fizycznego modelu rakiety, co wychodzi poza obszar nauki jaką jest informatyka, ponadto przeprowadzenie takich testów jest wysoce zasobo- i czasochłonne. Z tego powodu, w ramach tej sekcji nie przeprowadzono pełnej analizy dokładności symulacji. Niemniej jednak, można wskazać kilka elementów symulacji sugerujących jej dokładność oraz zaproponować kierunki dalszych potencjalnych badań.

2.7.1 Elementy sugerujące dokładność symulacji

Podczas testowania symulacji zaobserwowano zjawiska, które sugerują, że symulacja jest dokładna i odzwierciedla rzeczywiste zjawiska fizyczne. Poniżej omówiono szczegóły każdego z tych zjawisk:

Stabilizacja aerodynamiczna lotu rakiety

Jednym z kluczowych elementów potwierdzających dokładność symulacji jest stabilizacja aerodynamiczna rakiety w locie. Symulacja poprawnie odzwierciedla momenty aerodynamiczne i działanie sił przywracających, które stabilizują raketę wzdłuż jej osi. Stabilizacja ta jest szczególnie widoczna w trakcie lotu na dużych prędkościach, gdzie efekty aerodynamiczne są dominujące.

Osiąganie prędkości granicznych

W symulacji rakietę osiąga prędkości graniczne, które wynikają z równowagi między siłą ciągu a oporem aerodynamicznym. Występowanie tego zjawiska wskazuje na poprawne uwzględnienie oporu powietrza i jego zależności od prędkości.

Odchylenie się rakiety od osi pod wpływem wiatru

Symulacja dokładnie odwzorowuje zjawisko odchylenia się rakiety od osi lotu pod wpływem bocznego wiatru. Kierunek i wielkość odchylenia odpowiadają przewidywaniom teoretycznym, co wskazuje na poprawne modelowanie sił aerodynamicznych oraz momentów przechyłowych generowanych przez wiatr.

Wpływ kąta nachylenia silnika na kierunek lotu rakiety

Zgodnie z wynikami symulacji, zmiana kąta nachylenia silnika wpływa bezpośrednio na kierunek lotu rakiety. Zjawisko to jest zgodne z rzeczywistością, gdzie siła

ciągu generowana pod kątem w stosunku do osi rakiety prowadzi do zmiany trajektorii lotu. Symulacja poprawnie uwzględnia tę zależność, co pozwala na dokładne przewidywanie kierunku lotu w różnych scenariuszach.

Równoważenie wpływu kąta nachylenia silnika przez siłę nośną

W pewnych przypadkach symulacja pokazuje, że niewielki stały kąt nachylenia silnika jest kompensowany przez siłę nośną generowaną przez powierzchnie aerodynamiczne rakiety. To dynamiczne równoważenie sił wskazuje na poprawne modelowanie zarówno sił ciągu, jak i sił aerodynamicznych, które wspólnie determinują trajektorię lotu.

Podsumowanie

Powyższe obserwacje stanowią mocne dowody na dokładność i wiarygodność symulacji, pozwalając na jej wykorzystanie w analizach trajektorii lotu rakiety i jej zachowania w różnych warunkach.

Trudnym do ustalenia jest także jakość i dokładność opisanych wyżej zjawisk, ponieważ nie ma możliwości porównania wyników do rzeczywistych danych. Istnieje wysokie prawdopodobieństwo, że zjawiska te mimo iż występują w symulacji lotów odbiegają znacząco od odpowiadających im zjawisk w rzeczywistości.

Jednakże, aby potwierdzić dokładność symulacji, konieczne jest przeprowadzenie bardziej szczegółowych, praktycznych badań oraz porównanie wyników z rzeczywistymi danymi z lotów symulowanych rakiet.

2.7.2 Kierunki dalszych badań

W celu dalszej analizy dokładności symulacji oraz poprawy jej jakości konieczne jest porównanie danych otrzymanych podczas symulacji lotów w stworzonym silniku fizycznym z danymi odzwierciedlającymi rzeczywiste loty rakietowe. Uzyskanie takich danych może zostać osiągnięte w następujące sposoby:

Symulacje w innych silnikach fizycznych

Jednym z kierunków dalszych badań jest porównanie wyników symulacji w stworzonym silniku fizycznym z wynikami uzyskanymi w innych silnikach fizycznych dostępnych na rynku, takich jak **OpenRocket** czy **RockSim**. Porównanie wyników z

innymi silnikami pozwoli na ocenę dokładności i wiarygodności symulacji oraz identyfikację ewentualnych rozbieżności.

W takiej sytuacji, konieczne będzie przeprowadzenie identycznych testów w każdym z silników, co oznaczałoby konieczność stworzenia rakiety, oraz środowiska o identycznych parametrach w każdym z silników.

Istotnym problemem w takim podejściu są możliwe różnice w implementacji modeli rakiety czy duży poziom skomplikowania symulacji w innych silnikach. Przykładem może być brak możliwości odtworzenia w silniku stworzonym na potrzeby niniejszej pracy funkcji wyrzutu spalin silnika raketowego, obecnej w `OpenRocket` czy brak możliwości stworzenia rakiety o kształcie i parametrach identycznych w obu silnikach.

Porównanie z rzeczywistymi danymi lotów rakiet

Kolejnym kierunkiem badań jest porównanie wyników symulacji z danymi pochodzącymi z rzeczywistych lotów raketowych. Tego typu badania mogą być kosztowne i czasochłonne, ale pozwalają na uzyskanie najbardziej dokładnych danych. Wymagają one przeprowadzenia rzeczywistych lotów modeli rakiet wyposażonych w systemy pomiarowe, które zbierają dane o locie rakiety.

Problemem w takim podejściu jest konieczność przeprowadzenia wielu lotów, a także konieczność posiadania odpowiednich środków finansowych na budowę modeli rakiet. Ponadto, trudnym do odtworzenia może być środowisko i warunki atmosferyczne, które mogą wpływać na zachowanie rakiety w locie. Symulowanie takich warunków w symulacji komputerowej jest trudne, gdyż chociażby podmuchy wiatru działające na rakietę mogą być trudne do zbadania, mają one ogromny wpływ na trajektorię lotu rakiety.

Prowadzenie lotów rakiety w laboratorium, gdzie można kontrolować warunki atmosferyczne, może być jeszcze bardziej kosztowne i czasochłonne.

Porównanie z danymi testów nieraketowych

Alternatywnym podejściem do porównania wyników symulacji z rzeczywistymi jest przeprowadzenie testów nieraketowych, które mogą posłużyć do weryfikacji działania podstawowych mechanizmów symulacji. Przykładem takich testów mogą być testy polegające na nagraniu i zmierzeniu parametrów upuszczanego z określonej wysokości przedmiotu, o określonych parametrach. Dzięki takiemu nagraniu, możliwe

jest porównanie wyników symulacji zamodelowanego ciała z rzeczywistymi danymi. Tego typu testy mogą być łatwiejsze, jednak dostarczają one mniej informacji o silniku symulacyjnym, oraz nie pozwalają na weryfikację bardziej złożonych zjawisk.

Podsumowanie

Wszystkie powyższe kierunki badań pozwoliłyby na weryfikację dokładności silnika fizycznego oraz identyfikację ewentualnych błędów i rozbieżności. Niestety, nie zostały one przeprowadzone w ramach niniejszej pracy, gdyż wybiegają poza jej zakres. W celu dalszego rozwoju silnika fizycznego zalecane jest przeprowadzenie takich badań oraz weryfikacja działania silnika.

Na potrzeby niniejszej pracy inżynierskiej, zaimplementowany system symulacji lotów raketowych uznaje się za wystarczająco dokładny, aby przeprowadzać symulacje lotów w celach treningowych modelu AI.

2.8 Ewolucja projektu i wnioski

W trakcie pracy nad projektem opracowano kompletny system symulacji fizycznej lotów raket. Składa się on z dwóch głównych podsystemów: silnika fizycznego oraz modułu wizualizacji. Projekt realizowano przez około cztery miesiące, a jego rezultatem jest w pełni funkcjonalny system. Proces realizacji można podzielić na kilka kluczowych etapów.

2.8.1 Historia powstania projektu

Etap badawczy

Pierwszym etapem pracy było zgłębianie zagadnień związanych z fizyką lotu raket oraz zasadami działania silników raketowych. W ramach tego etapu przeprowadzono analizę literatury oraz materiałów online dotyczących lotów raketowych, modelowania dynamiki i symulacji fizycznych. W dużej mierze do zgłębienia tych zagadnień wykorzystano podręcznik "Topics in Advanced Model Rocketry" autorstwa Gordona K. Mandella[16], który stanowił cenne źródło wiedzy o modelowaniu lotów raketowych. Zgromadzona wiedza umożliwiła lepsze zrozumienie koncepcji matematycznych potrzebnych do zaimplementowania fizyki w silniku.

Na tym etapie zdecydowano o wykorzystaniu języka Python jako głównej technologii implementacji, ze względu na jego znajomość przez autorów oraz łatwość w

tworzeniu prototypów. Rozważano także alternatywne rozwiązania, takie jak C++ (ze względu na wydajność) czy środowisko Unity (z wykorzystaniem języka C#). Ostatecznie wybrano Pythona jako najbardziej odpowiedni kompromis między wydajnością a łatwością implementacji.

Etap projektowania

Kolejnym etapem było zaprojektowanie i stworzenie pierwszej wersji silnika fizycznego. Początkowa implementacja pozwalała na symulację lotu rakiety w dwóch wymiarach, co umożliwiło identyfikację podstawowych problemów, takich jak niewystarczająca wydajność biblioteki `Pygame` oraz ograniczenia związane z użyciem `Matplotlib` do wizualizacji danych.

W odpowiedzi na te wyzwania zdecydowano się przebudować architekturę systemu. Wprowadzono rozwiązania oparte na interfejsach, co ułatwiło rozwijanie funkcjonalności. Zastosowano bibliotekę `PyQt` do wizualizacji danych odczytywanych z plików CSV, zamiast generowanych w czasie rzeczywistym. Podejście to umożliwiło szybką identyfikację błędów i znacząco obniżyło koszt czasowy ich naprawy.

Etap implementacji

Trzeci etap obejmował implementację nowej architektury oraz dodatkowych funkcjonalności silnika. Początkowo stworzono szkielet systemu, który pozwolił na weryfikację poprawności architektury. Następnie wdrożono model fizyki lotu rakiety, uwzględniający siły i momenty działające na rakietę. Kluczowym elementem tego etapu było zastosowanie odpowiednich wzorów matematycznych do precyzyjnego opisu dynamiki lotu.

Etap testowania

Ostatnim etapem był proces testowania, w którym przeprowadzono liczne symulacje lotów rakiet w różnych warunkach. Testy te pozwoliły na wykrycie i naprawę błędów oraz optymalizację działania systemu. Przykładem istotnych usprawnień było dodanie siły nośnej, która początkowo nie była uwzględniana. System zyskał także nowe funkcjonalności, które zwiększyły jego użyteczność.

2.8.2 Wnioski

Praca nad projektem pozwoliła na zdobycie cennego doświadczenia w takich dziedzinach jak modelowanie fizyki, implementacja systemów symulacyjnych oraz wizualizacja danych. Stanowiła także doskonałą okazję do zgłębienia tajników lotów raketowych oraz lepszego zrozumienia ich zasad działania.

Podczas realizacji projektu napotkano wiele problemów, zarówno implementacyjnych, jak i teoretycznych, związanych z fizyką lotu. Większość z nich udało się rozwiązać, co znacząco podniosło jakość finalnego rozwiązania.

Jak wskazano w podrozdziale 2.7, nie można jednoznacznie stwierdzić, czy system w pełni odzwierciedla rzeczywistość fizyczną. Wymaga to dalszych badań i testów, co wykracza poza zakres niniejszej pracy.

Finalny projekt spełnia wszystkie założenia określone w rozdziale 2.1, a dodatkowo umożliwia trenowanie modeli AI w symulacjach lotów raketowych. Dzięki swojej modułowej architekturze system stanowi solidną bazę do dalszego rozwoju, w tym dodawania nowych sił lub usprawniania istniejących komponentów.

3 Moduł II: Algorytm sterujący rakieta.

Algorytm sterujący rakieta jest drugim modulem projektu. Jego celem jest stworzenie algorytmu, który będzie w stanie samodzielnie sterować rakieta i utrzymać pionową trajektorie lotu. Algorytm będzie uczony na silniku symulacyjnym z modulu I.

Autorem tego modulu jest Dominik Kubicki.

Niniejszy rozdział opisuje założenia projektowe, teoretyczne podstawy, użyte technologie oraz wybór odpowiedniego algorytmu uczenia maszynowego, a także sposoby i efekty uczenia algorytmu.

3.1 Założenia i cele projektowe.

W trakcie pracy nad algorytmem sterującym rakieta poczynione zostały następujące założenia:

- Algorytm sterujący rakieta będzie miał za zadanie utrzymanie stabilnej trajektorii lotu rakiety, czyli doprowadzenie rakiety do pionu w różnych warunkach atmosferycznych. Bardziej skomplikowane trajektorie lotu nie są obiektem badań w tej pracy.
- Algorytm będzie uczony na silniku symulacyjnym, będącym efektem pracy nad modulem I.
- Algorytm będzie wykorzystywał algorytmy sztucznej inteligencji do nauki, która będzie przebiegała w sposób dynamiczny w trakcie lotu rakiety.

Założenia te pozwalają na usystematyzowanie pracy nad algorytmem oraz ubranie go w ramy teoretyczne, upraszczając skomplikowanie problemu.

3.2 Użyte technologie.

Do implementacji modulu sterowania rakieta wykorzystano następujące technologie i biblioteki:

Gymnasium to biblioteka służąca do tworzenia i zarządzania środowiskami dla algorytmów uczenia przez wzmacnianie. Stanowi rozwinięcie popularnej biblioteki OpenAI Gym, oferując:

- Standardowy interfejs do definiowania środowisk RL
- Narzędzia do tworzenia wrapperów modyfikujących zachowanie środowisk
- Implementacje popularnych środowisk testowych
- Mechanizmy do obsługi przestrzeni obserwacji i akcji

Stable Baselines3 to biblioteka implementująca współczesne algorytmy uczenia przez wzmacnianie. Zawiera:

- Gotowe implementacje popularnych algorytmów RL (PPO, SAC, TD3 itp.)
- Narzędzia do trenowania i ewaluacji modeli
- Wsparcie dla środowisk wektorowych
- Mechanizmy callbacków i logowania

3.2.1 Biblioteki pomocnicze

- **numpy** biblioteka do obliczeń numerycznych i operacji na tablicach wielowymiarowych
- **pandas** narzędzia do analizy i przetwarzania danych tabelarycznych
- **matplotlib** tworzenie wizualizacji i wykresów
- **pyyaml** obsługa plików konfiguracyjnych w formacie YAML

Wszystkie wymienione technologie zostały zintegrowane z istniejącym silnikiem fizycznym opisanym w rozdziale II.

3.3 Podstawy teoretyczne uczenia maszynowego.

W niniejszym rozdziale przedstawione zostały kompleksowe podstawy teoretyczne dotyczące uczenia modeli sztucznej inteligencji, ze szczególnym uwzględnieniem metod stosowanych w sterowaniu lotami raketowymi. Omówione zostały różne paradygmaty uczenia maszynowego, ich matematyczne fundamenty oraz praktyczne aspekty implementacyjne w kontekście systemów autonomicznego sterowania.

3.3.1 Podejścia do uczenia maszynowego.

Współczesne systemy sztucznej inteligencji wykorzystują różne paradygmaty uczenia maszynowego, z których każdy znajduje zastosowanie w innych scenariuszach sterowania rakieta. Poniżej przedstawiono charakterystykę i obszary zastosowań trzech przewodnich podejść do uczenia maszynowego. [12]

- **Uczenie nadzorowane (Supervised Learning)**

W uczeniu nadzorowanym model jest trenowany na podstawie zestawu danych wejściowych i odpowiadających im poprawnych wyników (etykiet). W przypadku sterowania rakieta, dane wejściowe mogą obejmować parametry lotu (np. prędkość, wysokość, kąt nachylenia), a etykiety to optymalne działania sterujące (np. pożądany kąt nachylenia silnika). Algorytm uczy się mapować dane wejściowe na odpowiednie działania, minimalizując błąd pomiędzy przewidywaniami a rzeczywistymi etykietami. Przykłady modeli stosowanych w uczeniu nadzorowanym to sieci neuronowe, drzewa decyzyjne czy modele regresji.

- **Uczenie przez wzmacnianie (Reinforcement Learning)**

W podejściu tym model uczy się poprzez interakcję ze środowiskiem (w tym przypadku symulatorem lotu), otrzymując w każdym kroku sygnał nagrody za dobre działania. W prezentowanym rozwiązaniu, w każdym kroku symulacji model otrzymuje aktualny stan rakiety i zwraca decyzję o kącie wychylenia silnika, a następnie otrzymuje ocenę tej decyzji w postaci funkcji nagrody. Takie podejście pozwala na uczenie się optymalnej strategii sterowania bez potrzeby posiadania gotowych przykładów "poprawnych" decyzji.

- **Uczenie nienadzorowane (Unsupervised Learning)**

W uczeniu nienadzorowanym model analizuje dane bez dostępu do etykiet, skupiając się na odkrywaniu ukrytych wzorców lub struktur w danych. Choć mniej bezpośrednio przydatne w sterowaniu rakieta, metody te mogą być wykorzystane np. do grupowania podobnych stanów lotu lub redukcji wymiarowości danych. Przykłady algorytmów to k-means czy autoenkodery.

W niniejszej pracy zastosowano podejście z zakresu uczenia przez wzmacnianie, które szczególnie dobrze sprawdza się w problemach sterowania dynamicznego, gdzie pełna znajomość modelu fizycznego jest trudna do uzyskania, a tradycyjne metody

kontrolne mogą być niewystarczająco elastyczne.

3.3.2 Algorytmy uczenia przez wzmocnianie

W kontekście sterowania rakieta szczególnie istotne są następujące klasy algorytmów uczenia przez wzmocnianie, każda oferująca unikalne podejście do problemu optymalizacji strategii sterowania:

Metody gradientu polityki (Policy Gradient Methods) [6] Stanowią podstawową rodzinę algorytmów bezpośrednio optymalizujących funkcję nagrody poprzez aktualizację parametrów polityki. Ich matematyczną podstawę opisuje równanie:

$$\nabla_{\theta} J(\theta) = E_{\pi} [\nabla_{\theta} \log \pi_{\theta}(a|s) Q^{\pi}(s, a)] \quad (3.1)$$

gdzie:

- θ reprezentuje parametry polityki sterowania
- $\pi_{\theta}(a|s)$ to prawdopodobieństwo podjęcia akcji a w stanie s
- $Q^{\pi}(s, a)$ oznacza oczekiwaną nagrodę dla danej pary stan-akcja

W praktyce raketowej, metody te pozwalają na bezpośrednie uczenie się strategii sterowania bez konieczności modelowania pełnej funkcji wartości. Ich główną zaletą jest zdolność do obsługi ciągłych przestrzeni akcji, co jest kluczowe dla precyzyjnego sterowania rakieta. Wyzwaniem pozostaje jednak wysoka wariancja estymatorów gradientu, co może prowadzić do niestabilności procesu uczenia.

Architektura Aktor-Krytyk (Actor-Critic) [14] Stanowi hybrydowe podejście łączące zalety metod opartych na funkcji wartości i metod gradientu polityki. Składa się z dwóch współpracujących komponentów:

- **Aktor (policy network)** - odpowiedzialny za generowanie akcji sterujących na podstawie aktualnego stanu rakiety. Działa jako "pilot", podejmujący decyzje w czasie rzeczywistym.
- **Krytyk (value network)** - pełni rolę "instruktora", oceniającego jakość podjętych decyzji poprzez estymację oczekiwanej nagrody. Jego wyjście służy do redukcji wariancji aktualizacji polityki.

W systemach sterowania rakieta, architektura ta szczególnie dobrze sprawdza się dzięki równowadze między eksploracją nowych strategii (aktor) a efektywnym wykorzystaniem zebranych doświadczeń (krytyk). Implementacje takie jak A2C (Advantage Actor-Critic) czy A3C (Asynchronous Advantage Actor-Critic) dodatkowo poprawiają efektywność poprzez wykorzystanie równoległych środowisk.

Projektowanie funkcji nagrody Kluczowym aspektem skutecznego zastosowania RL w sterowaniu rakieta jest odpowiednie sformułowanie funkcji nagrody, która musi precyzyjnie odzwierciedlać cele misji. Typowa funkcja nagrody dla problemu stabilizacji rakiety może uwzględniać następujące komponenty:

- **Śledzenie trajektorii:** Nagroda za zmniejszanie odległości od zadanej ścieżki lotu, często wyrażana jako ujemna norma różnicy między aktualną a docelową pozycją ($\|p_t - p_{target}\|$)
- **Stabilność orientacji:** Kara za odchylenia od pożądanego orientacji, mierzona poprzez kąty Eulera (roll, pitch, yaw)
- **Efektywność energetyczna:** Nagroda za oszczędne zużycie paliwa, często proporcjonalna do kwadratu zastosowanego sterowania (a_t^2)
- **Bezpieczeństwo:** Ostre kary za przekroczenie dopuszczalnych parametrów lotu (np. zbyt duże przyspieszenia, kąty natarcia)

Przykładowa funkcja nagrody łącząca te elementy:

$$r_t = -w_1\|p_t - p_{target}\| - w_2\|\theta_t\| - w_3a_t^2 - w_4I_{awaria} \quad (3.2)$$

gdzie wagi w_i determinują względne znaczenie każdego komponentu, a I_{awaria} to funkcja indykatora sytuacji awaryjnej. Odpowiednie zbalansowanie tych składowych jest kluczowe - zbyt mocny nacisk na stabilizację może prowadzić do zachowań zbyt konserwatywnych, podczas gdy nadmierna nagroda za śledzenie trajektorii może powodować niebezpieczne oscylacje.

Wyzwania w zastosowaniu do sterowania rakieta Implementacja systemów RL dla sterowania rakieta wiąże się z szeregiem istotnych wyzwań technicznych:

- **Wysoka wymiarowość przestrzeni stanów:** Pełny stan dynamiczny rakiety może obejmować kilkanaście zmiennych (pozycja, prędkość, orientacja, prędkość kątowna, parametry silnika), co wymaga zaawansowanych technik aproksymacji funkcji wartości.

- **Zmienna dynamika systemu:** Zmieniająca się masa rakiety (w wyniku spalania paliwa) oraz zmienne warunki atmosferyczne prowadzą do ciągłych zmian parametrów systemu, wymagając algorytmów zdolnych do adaptacji.
- **Ryzyko katastrofalnych awarii:** Błędy w fazie uczenia mogą prowadzić do niestabilności i utraty kontroli, co w rzeczywistych systemach jest niedopuszczalne. Konieczne są mechanizmy bezpieczeństwa i symulacje w pętli sprzężenia zwrotnego (hardware-in-the-loop).
- **Wymagania wierności symulacji:** Niedokładności w modelu fizycznym mogą prowadzić do uczenia się strategii nieprzenoszących się na rzeczywisty system. Rozwiązaniem jest wykorzystanie technik randomizacji domeny (domain randomization) lub uczenia meta-RL.

Pomimo tych wyzwań, ostatnie postępy w dziedzinie głębokiego RL pokazują, że odpowiednio zaprojektowane systemy są w stanie osiągać nadludzką precyzję w sterowaniu złożonymi systemami dynamicznymi, włączając w to problematykę lotów raketowych. Kluczem jest staranne połączenie solidnych podstaw teoretycznych z praktycznymi technikami stabilizacji uczenia i walidacji bezpieczeństwa.

3.3.3 Modele stosowane w sterowaniu rakieta.

Wybór odpowiedniej architektury modelu ma kluczowe znaczenie dla skuteczności systemu sterowania. Poniżej przedstawiono analizę najważniejszych podejść.

Modele dla uczenia nadzorowanego

- **Sieci neuronowe MLP (Multi-Layer Perceptron)** Architektura MLP sprawdza się w zadaniach regresji parametrów sterowania. Wielowarstwowa struktura z nieliniowymi funkcjami aktywacji pozwala na modelowanie złożonych zależności między stanem rakiety a optymalnymi komendami sterującymi. Typowa implementacja zawiera 3-5 warstw ukrytych o rozmiarze 64-256 neuronów.
- **Regresja Gaussa (Gaussian Process Regression)** Metoda bayesowska szczególnie przydatna gdy dostępna jest ograniczona liczba danych treningowych. Pozwala na estymację niepewności predykcji, co może być cenne w krytycznych fazach lotu. Wadą jest wyższa złożoność obliczeniowa.

Modele dla uczenia nienadzorowanego

- **Autoenkodery wariacyjne (VAE)** Pozwalają na kompresję wielowymiarowych danych sensorycznych do reprezentacji latentnej, zachowując kluczowe informacje o stanie rakiety. Mogą służyć jako wstępny etap przetwarzania przed właściwym algorytmem sterującym.
- **GMM (Gaussian Mixture Models)** Użyteczne do klasteryzacji stanów rakiety i identyfikacji typowych konfiguracji. Mogą wspomagać system sterowania poprzez szybką klasyfikację sytuacji awaryjnych.

Modele dla uczenia przez wzmacnianie [25]

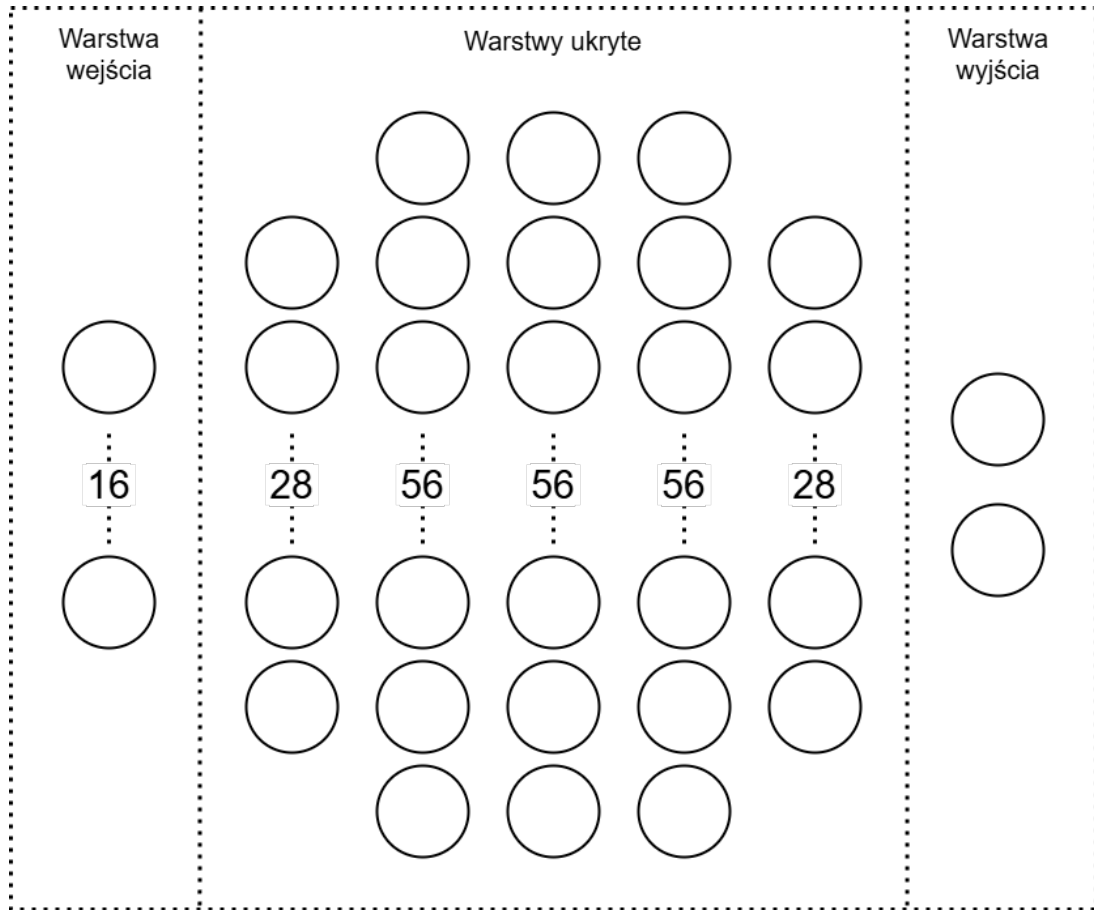
- **DDPG (Deep Deterministic Policy Gradient)** Algorytm łączący podejście policy gradient z Q-learningiem, szczególnie skuteczny dla ciągłych przestrzeni akcji. Wymaga jednak starannego dostrojenia hiperparametrów i może być niestabilny.
- **SAC (Soft Actor-Critic)** Nowoczesny algorytm maksymalizujący zarówno nagrodę jak i entropię polityki, co prowadzi do bardziej eksploracyjnych zachowań. Dobrze radzi sobie z zadaniami o ciągłej przestrzeni akcji.

3.4 System sterowania lotu rakieta

System sterowania rakieta oparty jest na wytrenowanej sieci neuronowej typu *MLP* (Multilayer Perceptron). Została ona zaprojektowana tak, aby przekształcać wektor stanu rakiety na odpowiednie komendy sterujące. Wektor wejściowy modelu zawiera dwadzieścia wartości opisujących aktualny stan rakiety:

- Położenie $[x, y, z]$
- Prędkość liniowa $[v_x, v_y, v_z]$
- Przyspieszenie liniowe $[a_x, a_y, a_z]$
- Orientacja rakiety w przestrzeni wyrażona jako kąty Eulera $[roll, pitch, yaw]$
- Prędkość kątowna $[w_x, w_y, w_z]$
- Przyspieszenie kątowe $[a_x, a_y, a_z]$
- Aktualny kąt wychylenia silnika $[\alpha_x, \alpha_y]$

które następnie przechodzą przez warstwy ukryte sieci neuronowej. Sieć składa się z pięciu warstw ukrytych o rozmiarach kolejno: 32, 64, 64, 64 i 32 neurony.



Rysunek 3.1. Schemat sieci neuronowej systemu sterowania, *źródło: Opracowanie własne.*

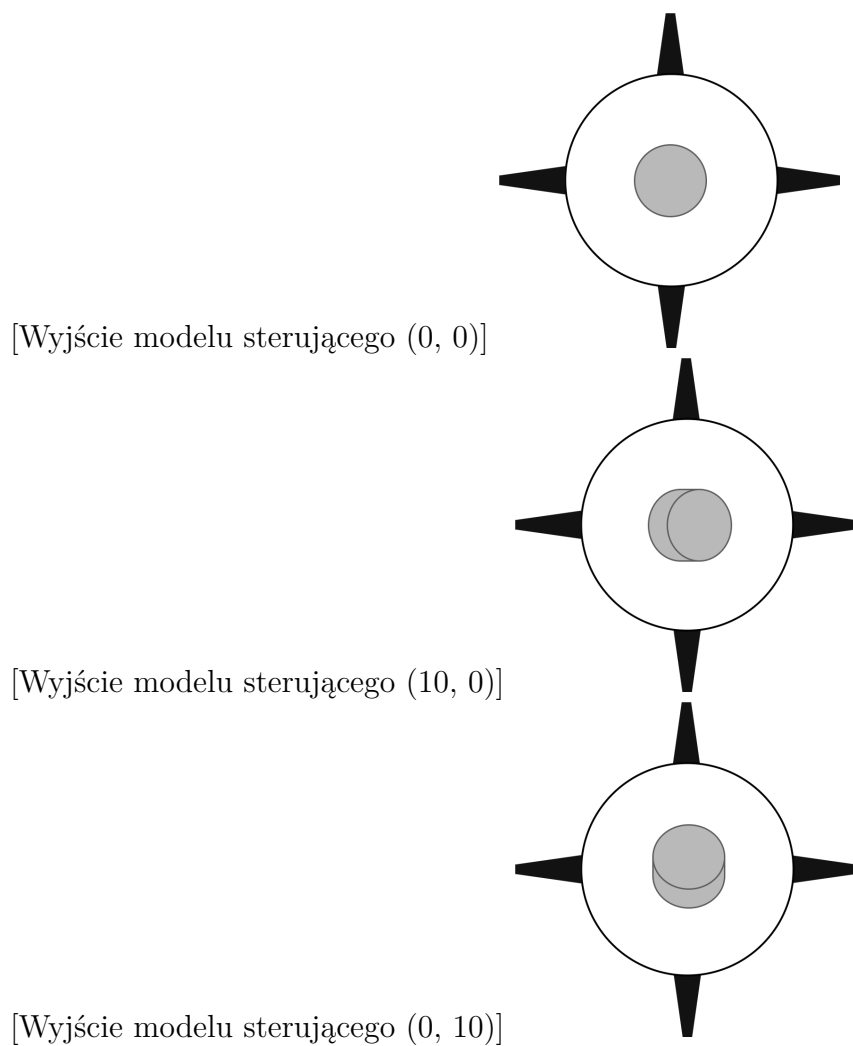
Wyjściem modelu jest wektor $[\alpha_x, \alpha_y]$, który reprezentuje kąt odchylenia silnika w osi x oraz y , ograniczony do przedziału od -20° do 20° .

3.4.1 Funkcja aktywacji

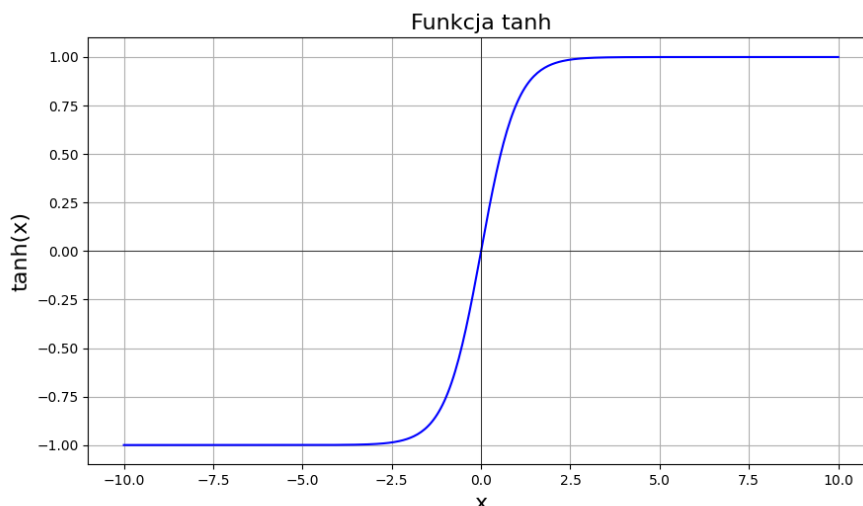
W każdej warstwie ukrytej sieci neuronowej zastosowano funkcję aktywacji typu tangens hiperboliczny ($\tanh$), która przekształca sumę ważoną wejść neuronu w zakres $(-1, 1)$. Funkcja ta opisana jest wzorem:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \quad (3.3)$$

Zastosowanie funkcji $\tanh$ zapewnia nieliniowość przekształceń w sieci, co umożliwia modelowi aproksymację złożonych zależności pomiędzy stanem rakiety a odpowiednimi komendami sterującymi. W przeciwieństwie do funkcji ReLU, funkcja $\tanh$ jest symetryczna względem zera, co może przyspieszyć zbieżność w sytuacjach, gdy dane wejściowe są również odpowiednio znormalizowane wokół zera. Wybór tej



Rysunek 3.2. Wpływ wartości warswy wyjściowej na kąty wychylenia silnika. Wyjście (x, y) powoduje ustawienie silnika pod kątami x i y w osiach X i Y], *źródło: Opracowanie własne.*



Rysunek 3.3. Funkcja aktywacji tangens hiperboliczny, źródło: *Opracowanie własne*.

funkcji aktywacji wspomaga stabilność propagacji sygnału w głębokich sieciach neuronowych.

3.4.2 Inicjalizacja wag

Wagi sieci neuronowej zostały zainicjalizowane metodą ortogonalną, co oznacza, że każda macierz wag w warstwach ukrytych została początkowo ustawiona jako macierz ortogonalna. Metoda ta polega na wygenerowaniu macierzy o ortonormalnych kolumnach (lub wierszach), spełniających warunek:

$$\mathbf{W}^T \mathbf{W} = \mathbf{I} \quad (3.4)$$

Inicjalizacja ortogonalna ma na celu utrzymanie odpowiedniego przepływu gradientów podczas procesu uczenia – szczególnie w głębokich sieciach. Zmniejsza to ryzyko eksplozji lub zaniku gradientu w czasie wstecznej propagacji błędu. W praktyce metoda ta często przyczynia się do szybszej i bardziej stabilnej konwergencji modelu, a także poprawia jego końcową skuteczność.

3.4.3 Działanie sieci MLP

Sieć neuronowa typu MLP (Multilayer Perceptron) jest klasycznym modelem sieci neuronowej składającym się z kilku warstw neuronów, w tym jednej warstwy wejściowej, kilku warstw ukrytych oraz warstwy wyjściowej. Celem tej sieci jest nauka

nieliniowej funkcji odwzorowującej dane wejściowe na pożądane wyjście, które w przypadku systemu sterowania rakieta reprezentuje kąty wychylenia silnika w osiach x oraz y .

W każdej warstwie sieci neuronowej obliczana jest suma ważona wejść $\mathbf{h}^{(l-1)}$ z poprzedniej warstwy, do której dodawany jest wektor przesunięcia $\mathbf{b}^{(l)}$. Wynik tej operacji przechodzi przez funkcję aktywacji $\tanh$, aby uzyskać wyjście z warstwy ukrytej. Proces ten dla każdej warstwy l opisuje wzór:

$$\mathbf{h}^{(l)} = \tanh(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}) \quad (3.5)$$

gdzie:

- $\mathbf{h}^{(l-1)}$ to wektor wyjściowy z poprzedniej warstwy (dla warstwy wejściowej $\mathbf{h}^{(0)}$ to wektor wejściowy $\mathbf{x}$),
- $\mathbf{W}^{(l)} \in R^{d_l \times d_{l-1}}$ to macierz wag dla warstwy l ,
- $\mathbf{b}^{(l)} \in R^{d_l}$ to wektor przesunięcia dla warstwy l ,
- $\tanh(\cdot)$ to funkcja aktywacji tangens hiperboliczny.

W przypadku sieci MLP z pięcioma warstwami ukrytymi, obliczenia te będą zachodziły pięciokrotnie, aż do uzyskania wyjścia z ostatniej warstwy ukrytej $\mathbf{h}^{(5)}$.

Po przejściu przez wszystkie warstwy ukryte, ostatnia warstwa generuje ostateczne wyjście, które w przypadku systemu sterowania rakieta reprezentuje wektor $[\alpha_x, \alpha_y]$ – kąty wychylenia silnika w osiach x oraz y . Wyjście tej warstwy jest obliczane za pomocą:

$$\mathbf{y} = \mathbf{W}^{(6)}\mathbf{h}^{(5)} + \mathbf{b}^{(6)} \quad (3.6)$$

gdzie:

- $\mathbf{y} \in R^2$ to wektor wyjściowy, który zawiera kąty wychylenia silnika w osiach x oraz y ,
- $\mathbf{W}^{(6)} \in R^{2 \times 32}$ to macierz wag dla warstwy wyjściowej,
- $\mathbf{b}^{(6)} \in R^2$ to wektor przesunięcia dla warstwy wyjściowej.

Z racji ograniczeń na kąty wychylenia silnika, wartości α_x oraz α_y są później ograniczane do przedziału od -20° do 20° .

3.5 Algorytm uczący - PPO

Wśród różnych algorytmów uczenia przez wzmacnianie, Proximal Policy Optimization (PPO) [24] wyróżnia się jako szczególnie odpowiedni wybór dla problemów sterowania rakieta. Opracowany w 2017 roku przez zespół badawczy OpenAI pod kierownictwem Johna Schulmana, PPO szybko stał się złotym standardem w zastosowaniach wymagających stabilnego uczenia w ciągłych przestrzeniach akcji — dokładnie taki przypadek mamy w sterowaniu rakieta.

3.5.1 Charakterystyka teoretyczna

Podstawą PPO jest funkcja kosztu, która ogranicza zakres zmian w polityce podczas aktualizacji. Dzięki temu algorytm unika niestabilności charakterystycznych dla wcześniejszych metod. Funkcja kosztu PPO w wersji z mechanizmem *clipping* jest zdefiniowana jako:

$$L^{CLIP}(\theta) = E_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad (3.7)$$

gdzie:

- $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ — stosunek nowych i starych prawdopodobieństw wyboru akcji, mierzący zmianę polityki,
- $\hat{A}_t$ — estymata funkcji przewagi (advantage), która wskazuje, jak bardzo dana akcja była lepsza niż oczekiwano,
- ϵ — hiperparametr (np. 0.2), ograniczający zakres dopuszczalnych zmian w polityce.

Mechanizm *clip* ogranicza aktualizację funkcji polityki, jeśli zmiana $r_t(\theta)$ jest zbyt duża, co zapobiega destabilizacji podczas uczenia.

3.5.2 Aktualizacja wag sieci MLP

Polityka $\pi_\theta(a_t|s_t)$ jest reprezentowana przez sieć neuronową typu MLP (wielowarstwowy perceptron), której parametry θ są trenowane w oparciu o gradient funkcji kosztu $L^{CLIP}(\theta)$. Po każdej serii interakcji ze środowiskiem (np. po kilku epizodach lotu rakiety), algorytm PPO oblicza gradient:

$$\nabla_\theta L^{CLIP}(\theta) \quad (3.8)$$

i wykorzystuje go do aktualizacji wag MLP przy pomocy optymalizatora (najczęściej Adam). Proces ten polega na modyfikacji wag w kierunku maksymalizacji funkcji nagrody przy jednoczesnym ograniczeniu zmian w polityce. Dodatkowo, równolegle aktualizowana jest także sieć wartości (critic), która minimalizuje błąd estymacji wartości funkcji stanu $V(s_t)$, najczęściej poprzez funkcję strat typu MSE:

$$L^{\text{value}}(\phi) = \left(V_{\phi}(s_t) - R_t \right)^2, \quad (3.9)$$

gdzie R_t to zwrot zdyskontowany, a ϕ oznacza parametry sieci krytyka. Obie sieci — aktora i krytyka — uczą się jednocześnie, lecz niezależnie, umożliwiając stabilne i szybkie dostosowanie strategii sterowania.

W przypadku rakiety, stan s_t obejmuje informacje o położeniu, prędkościach, orientacji oraz aktualnych ustawieniach silnika. Na przykład:

$$s_t = [x, y, z, v_x, v_y, v_z, \text{pitch}, \text{yaw}] \quad (3.10)$$

Akcja $a_t = [\alpha_x, \alpha_y]$ odpowiada kątom wychylenia silnika w osi x i y . Polityka PPO $\pi_{\theta}(a_t|s_t)$ reprezentowana jest przez sieć MLP, która przekształca stan na akcję w sposób ciągły. Dzięki funkcji kosztu $L^{CLIP}(\theta)$ model uczy się tak aktualizować parametry θ , by wybierać coraz lepsze komendy sterujące przy zachowaniu stabilności uczenia.

Mimo licznych zalet, PPO nie jest pozbawiony wad, które należy uwzględnić w projektowaniu systemu:

- **Eksploracja przestrzeni stanów:** W porównaniu do algorytmów jak SAC (Soft Actor-Critic), PPO może mieć trudności z wyjściem z lokalnych minimów w szczególnie złożonych scenariuszach. W praktyce raketowej często wymaga to wprowadzenia dodatkowych mechanizmów eksploracji.
- **Wrażliwość na funkcję nagrody:** Podobnie jak wszystkie metody RL, PPO jest bardzo czuły na projekt funkcji nagrody. Niewłaściwie zbalansowane wagi poszczególnych składowych mogą prowadzić do nieoczekiwanych strategii sterowania.
- **Precyzja kontroli:** Choć PPO generalizuje lepiej niż wiele innych metod, w niektórych zastosowaniach może ustępować specjalistycznym kontrolerom (np. PID z adaptacyjnymi parametrami) w zadaniach wymagających ekstremalnej

precyzji.

3.5.3 Podsumowanie

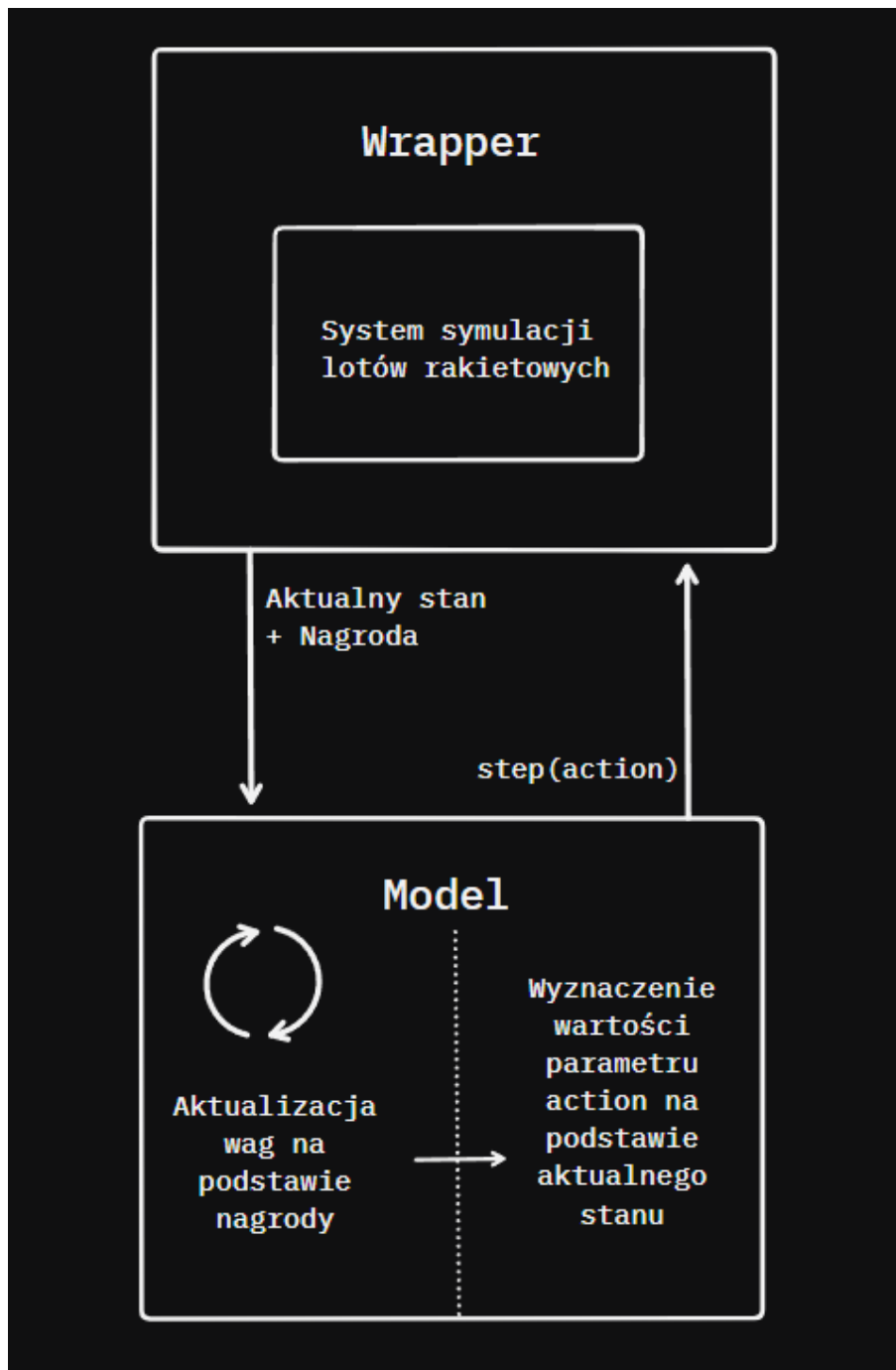
PPO stanowi doskonały kompromis między stabilnością, efektywnością i praktyczną użytecznością w problemach sterowania rakieta. Jego unikalne właściwości, szczególnie mechanizm ograniczonej aktualizacji polityki, czynią go preferowanym wyborem dla zastosowań wymagających zarówno bezpieczeństwa, jak i adaptacyjności. Mimo pewnych ograniczeń, odpowiednio zaimplementowany i dostrojony algorytm PPO jest w stanie osiągać wyniki porównywalne z zaawansowanymi kontrolerami specjalistycznymi, oferując przy tym znaczną elastyczność.

3.6 Architektura i implementacja modelu uczącego

W niniejszym rozdziale szczegółowo opisano architekturę oraz implementację systemu umożliwiającego trenowanie modelu sterowania rakieta metodą uczenia przez wzmacnianie. Przedstawiona struktura łączy w sobie zarówno komponent symulacyjny opisany wcześniej, jak i nowe elementy specyficzne dla procesu uczenia maszynowego.

3.6.1 Architektura systemu uczącego

Na Rysunku 3.4 przedstawiono ogólną architekturę rozwiązania, które składa się z dwóch głównych komponentów:



Rysunek 3.4. Schemat architektury systemu uczącego. źródło: *Opracowanie własne*.

- **Wrapper środowiska symulacyjnego** - interfejs łączący silnik fizyczny z modelem uczącym
- **Model uczący** - implementacja algorytmu RL odpowiedzialna za podejmowanie decyzji

Wrapper środowiska symulacyjnego

Wrapper pełni kluczową rolę w integracji silnika fizycznego z frameworkiem do uczenia maszynowego. Jego główne zadania to:

- Standaryzacja interfejsu komunikacji zgodnie z wymaganiami Gymnasium
- Transformacja danych między formatami używanymi przez silnik fizyczny i model RL
- Zarządzanie cyklem życia symulacji (inicjalizacja, kroki symulacyjne, reset)
- Obliczanie i dostarczanie funkcji nagrody

Wrapper implementuje dwa kluczowe metody:

- `step(action)` - wykonuje krok symulacji z podaną akcją sterującą
- `reset()` - inicjalizuje nową symulację, zwracając początkowy stan

Funkcja nagrody

Funkcja nagrody została zaprojektowana tak, aby promować stabilny lot pionowy. Jej implementacja w języku Python przedstawia się następująco:

```
1 def reward_function(obj):
2     # Ekstrakcja podstawowych parametrów
3     pitch, yaw, _ = obj.rotation
4     x, y, z = obj.position
5
6     # Obliczenie odchylenia od osi Z
7     distance_from_z = math.sqrt(x ** 2 + y ** 2)
8
9     # Transformacja do współrzędnych globalnych
10    rot_matrix = euler_to_rotation_matrix(obj.rotation)
11    local_z = np.array([z, 0, 1])
12    rotated_z = rot_matrix @ local_z
13
14    # Obliczenie kąta odchylenia od pionu
15    world_z = np.array([0, 0, 1])
16    cos_theta = np.dot(rotated_z, world_z) / \
17        (np.linalg.norm(rotated_z) *
18         np.linalg.norm(world_z))
19
20    angle = np.degrees(np.arccos(np.clip(cos_theta, -1.0, 1.0)))
21
22    # Korekta znaku kąta
23    rotated_z_xy = np.array([rotated_z[0], rotated_z[1], 0])
```

```

22     position_xy = np.array([x, y, 0])
23     dot_product = np.dot(rotated_z_xy, position_xy)
24
25     if dot_product < 0:
26         angle = -angle
27
28     return angle

```

Funkcja ta oblicza kąt odchylenia rakiety od idealnej osi pionowej, uwzględniając zarówno orientację, jak i pozycję obiektu. Im mniejsze odchylenie, tym wyższa nagroda, co skutecznie promuje stabilny lot pionowy. Jest to kluczowy element systemu, bezpośrednio wpływający na skuteczność uczenia.

3.6.2 Model uczący

Jak szczegółowo opisano w rozdziale 3.5, wykorzystany został algorytm PPO (Proximal Policy Optimization). Model otrzymuje od wrappera następujący wektor obserwacji:

- Położenie
- Prędkość liniowa
- Przyspieszenie liniowe
- Orientacja
- Prędkość kątowna
- Przyspieszenie kątowe
- Aktualny kąt wychylenia silnika

Model wykonuje dwie kluczowe operacje:

- **Aktualizacja parametrów** - modyfikacja wag sieci neuronowych na podstawie otrzymywanych nagród
- **Generowanie akcji** - obliczenie optymalnego kąta wychylenia silnika na podstawie aktualnego stanu

Ta architektura pozwala na stopniowe doskonalenie strategii sterowania poprzez ciągłą interakcję ze środowiskiem symulacyjnym.

3.6.3 Proces uczenia

Proces uczenia modelu sterowania rakieta stanowi iteracyjny cykl interakcji pomiędzy modelem uczącym a środowiskiem symulacyjnym. Poniżej szczegółowo opisano poszczególne etapy tego procesu:

Inicjalizacja środowiska

Proces rozpoczyna się od inicjalizacji środowiska symulacyjnego. W tym kroku rakietą jest ustawiana w pozycji startowej, zwykle na powierzchni lub na niewielkiej wysokości. Losowe niewielkie odchylenie od pionu wprowadza różnorodność do procesu uczenia. Silnik fizyczny jest resetowany, a wszystkie parametry symulacji przywracane są do wartości początkowych.

Główna pętla uczenia

Dla każdego kroku epizodu ($t = 1$ do T):

1. **Pobranie stanu bieżącego**

Model otrzymuje aktualny stan rakiety w postaci znormalizowanego wektora obserwacji. Dane te obejmują pełną informację o pozycji, prędkości, orientacji i ustawieniach sterowania. Normalizacja wartości do zakresu $[-1, 1]$ poprawia stabilność procesu uczenia.

2. **Generacja akcji sterującej**

Na podstawie bieżącego stanu, sieć polityki generuje akcję sterującą. Proces ten uwzględnia zarówno wyuczone strategie, jak i komponent losowy umożliwiający eksplorację nowych rozwiązań. Wygenerowana akcja jest następnie transformowana do odpowiedniego zakresu kątów wychylenia silnika.

3. **Wykonanie kroku symulacji**

Wyliczona akcja jest przekazywana do silnika fizycznego, który aktualizuje stan symulacji na jeden krok czasowy. W tym etapie obliczane są wszystkie siły działające na raketę i aktualizowane jej parametry ruchu. Sprawdzane są również warunki zakończenia epizodu.

4. **Obliczenie nagrody**

System oblicza nagrodę na podstawie aktualnego zachowania rakiety. Nagroda uwzględnia zarówno stabilność lotu, jak i postęp w osiągnięciu celów misji. Wartość ta jest kluczowym sygnałem uczącym dla modelu.

5. Aktualizacja parametrów modelu

Po zebraniu odpowiedniej liczby próbek, następuje aktualizacja parametrów sieci neuronowych. Proces ten optymalizuje zarówno politykę sterowania, jak i estymację wartości, wykorzystując specjalnie zaprojektowaną funkcję celu PPO.

Proces uczenia charakteryzuje się epizodyczną strukturą, gdzie każda próba stanowi niezależną symulację lotu. Równowaga między eksploracją nowych strategii a wykorzystaniem sprawdzonych rozwiązań jest kluczowa dla skutecznego uczenia. System stopniowo doskonali politykę sterowania, optymalizując długoterminową nagrodę poprzez ciągłą interakcję ze środowiskiem.

3.6.4 Podsumowanie

Przedstawiona architektura stanowi kompleksowe rozwiązanie łączące zaawansowany silnik fizyczny z nowoczesnymi technikami uczenia maszynowego. Kluczowe elementy systemu to:

- Elastyczny wrapper integrujący środowisko symulacyjne z frameworkiem RL
- Starannie zaprojektowana funkcja nagrody promująca pożądane zachowania
- Wydajny model uczący oparty na algorytmie PPO
- Skalowalny proces uczenia umożliwiający iteracyjne doskonalenie strategii

Ta struktura nie tylko umożliwia efektywne trenowanie modelu sterowania rakieta, ale także zapewnia możliwość łatwej rozbudowy i modyfikacji poszczególnych komponentów. W kolejnych rozdziałach przedstawione zostaną szczegółowe wyniki osiągnięte dzięki tej architekturze.

3.7 Wyniki i wnioski.

3.7.1 Cel działania algorytmu

Celem działania wytrenowanego algorytmu PPO było zapewnienie stabilnego, pionowego lotu rakiety. Zakładano, że model nauczy się kompensować działanie sił zakłócających, takich jak boczny wiatr, oraz samodzielnie korygować odchylenia od osi pionowej za pomocą kontroli wektora ciągu (TVC).

3.7.2 Metodyka pomiaru skuteczności

Aby ocenić skuteczność działania algorytmu, zastosowano następujące kryteria:

- Odchylenie katowe: średnie i maksymalne odchylenie rakiety od osi pionowej w stopniach.
- Stabilność lotu: procent czasu epizodu, w którym odchylenie nie przekraczało ustalonego progu (np. 5°).
- Nagroda całkowita: suma nagród osiągniętych podczas pojedynczego epizodu.

Pomiarów dokonywano podczas treningu oraz podczas osobnych sesji testowych, przy stałych warunkach środowiskowych.

3.7.3 Wyniki

Model PPO nauczył się podstawowej stabilizacji rakiety, jednak nie osiągnął pełnej niezawodności:

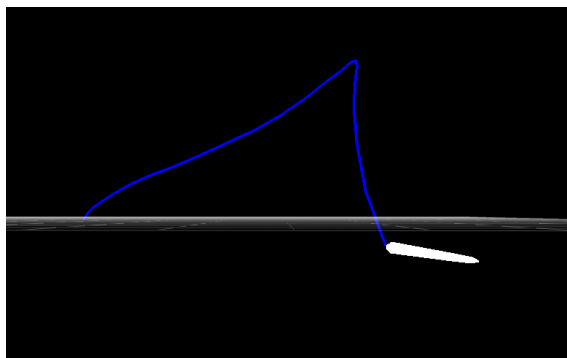
- Średnie odchylenie od osi pionowej wynosiło około $4,8^\circ$, a w niektórych momentach maksymalne odchylenia przekraczały 12° , zwłaszcza przy nagłych zakłóceniach.
- Czas stabilnego lotu wynosił średnio około 68% długości epizodu, przy czym w trudniejszych warunkach rakietę częściej przekraczała próg 5° .
- Krzywa nagrody w czasie treningu wzrastała nieregularnie, z wyraźnymi fluktuacjami, osiągając względne ustabilizowanie po około 500 000 kroków środowiska.
- Trajektorie lotu pokazywały zauważalne oscylacje, które model często tłumiał dopiero po kilku sekundach.

Wizualizacje wyników (krzywe nagrody i odchylenia) wskazywały na postępy w nauce, choć widoczne były również trudności w utrzymywaniu stabilnego lotu w bardziej dynamicznych sytuacjach. Poniżej zostały przedstawione tory lotu rakiety w wybranych epizodach treningowych.

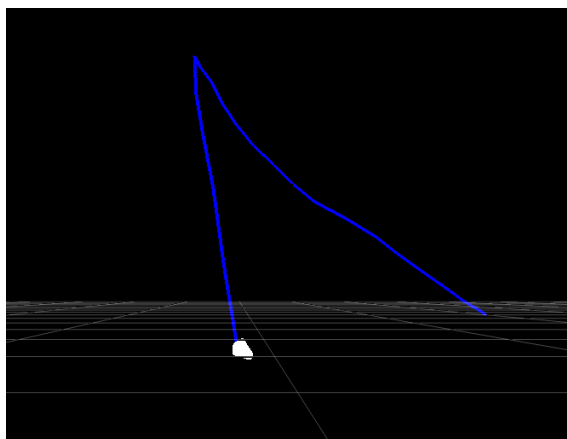
3.7.4 Wnioski

Na podstawie przeprowadzonych eksperymentów można stwierdzić, że algorytm PPO częściowo nauczył się stabilizować rakietę w pionowym locie, jednak osiągnięta jakość kontroli była ograniczona.

[Rzut na płaszczyznę YZ]

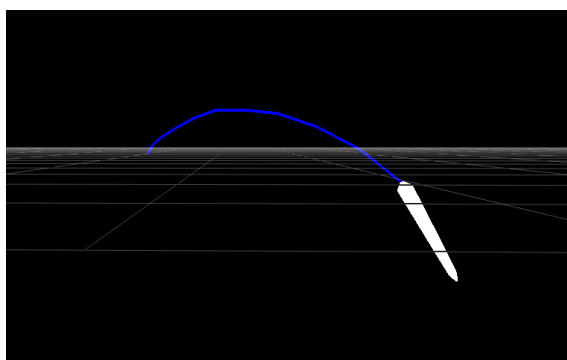


[Rzut na płaszczyznę XZ]

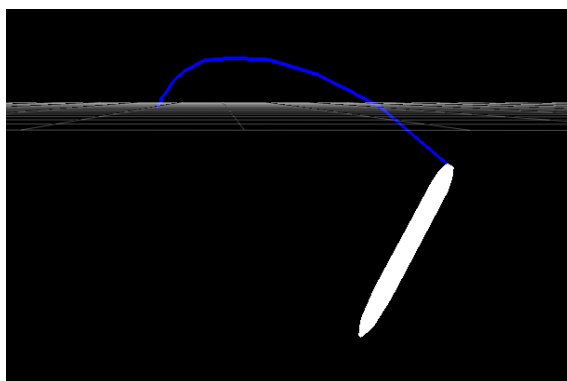


Rysunek 3.5. Trajektoria lotu rakiety przed trenowaniem modelu, *źródło: Opracowanie własne.*

[Rzut na płaszczyznę YZ]

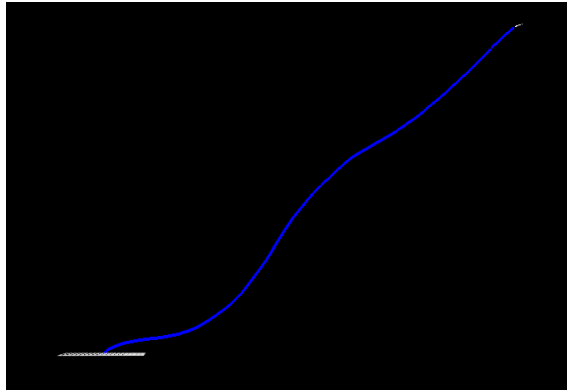


[Rzut na płaszczyznę XZ]

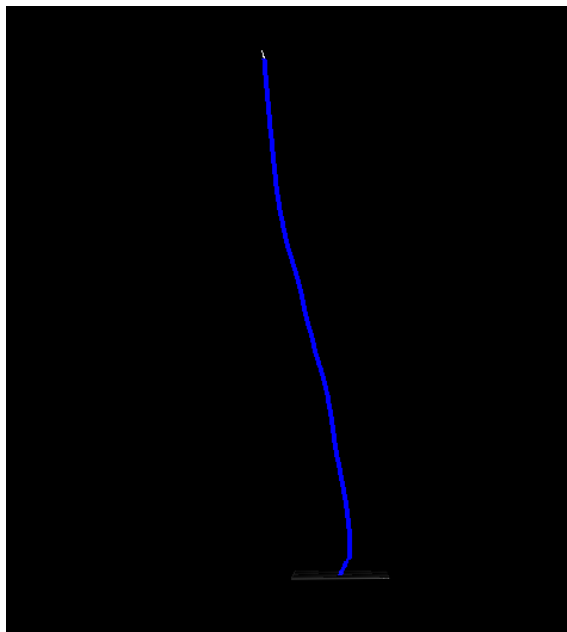


Rysunek 3.6. Trajektoria lotu rakiety po 150 episodach, *źródło: Opracowanie własne.*

[Rzut na płaszczyznę YZ]



[Rzut na płaszczyznę XZ]



Rysunek 3.7. Trajektoria lotu rakiety wytrenowanego modelu, *źródło: Opracowanie własne.*

Mocnymi stronami uzyskanego modelu były:

- Zdolność do podstawowej korekty odchyień, szczególnie w przypadku umiarkowanych zakłóceń.
- Zdobycie podstawowej strategii sterowania, umożliwiającej utrzymanie lotu przez większość czasu epizodu.

Ograniczenia zaobserwowane podczas testów obejmowały:

- Niska skuteczność przy silnych zakłóceniach, prowadząca do dużych wychyleń i długich czasów stabilizacji.
- Tendencja do utrzymywania kursu pod kątem 45 stopni w stosunku do osi Z wzdłuż jednej z osi.
- Powolne tłumienie oscylacji, co zwiększało ryzyko dalszych destabilizacji po początkowym wychyleniu.

3.7.5 Podsumowanie

Podsumowując, zastosowanie algorytmu PPO do sterowania rakieta wykazało zarówno potencjał, jak i ograniczenia tej metody. Choć model nauczył się podstawowych zasad stabilizacji, wymaga dalszej optymalizacji i dostrojenia, aby osiągnąć pożądaną poziom niezawodności i precyzji. W przyszłych badaniach warto rozważyć zastosowanie bardziej zaawansowanych technik eksploracji oraz dodatkowych mechanizmów stabilizacji, aby poprawić wyniki w trudnych warunkach lotu.

4 Podsumowanie

4.1 Podsumowanie

Praca przedstawia kompleksowe rozwiązanie problemu autonomicznego sterowania rakieta, które łączy zaawansowany silnik fizyczny z nowoczesnymi technikami uczenia maszynowego. W ramach projektu opracowane zostały dwa główne moduły:

- **Moduł symulacji fizycznej** – stworzono model dynamiki lotu rakiety, który uwzględnia kluczowe siły i zjawiska fizyczne. Silnik symulacyjny został zoptymalizowany pod kątem wydajności, co pozwoliło na szybkie przeprowadzanie symulacji na standardowym sprzęcie komputerowym.
- **Moduł uczenia maszynowego** – zaimplementowano system uczący oparty na algorytmie PPO, który okazał się skuteczny w sterowaniu rakieta. Zaprojektowana architektura obejmowała specjalny wrapper łączący środowisko symulacyjne z frameworkiem RL oraz precyzyjnie zdefiniowaną funkcję nagrody.

Eksperymenty przeprowadzone na zaprojektowanej platformie wykazały, że połączenie modelu fizycznego z algorytmem uczenia maszynowego pozwala na skuteczną stabilizację i kontrolę lotu rakiety. System wykazał zdolność do adaptacji do różnych warunków początkowych i utrzymania stabilnego lotu pionowego.

4.2 Wnioski końcowe

Na podstawie przeprowadzonych badań można sformułować kilka kluczowych wniosków:

- Projekt okazał się trudniejszy niż początkowo zakładałem, co potwierdziło skomplikowanie procesu zarówno w aspekcie modelowania fizycznego, jak i implementacji algorytmu sterowania.
- Jakość symulacji i jej dokładność mają ogromne znaczenie dla całego procesu. Wyważenie między dokładnością modelu a wydajnością obliczeniową stanowi istotne wyzwanie przy projektowaniu silnika symulacyjnego.
- Tworzenie zaawansowanego silnika symulacyjnego jest trudnym zadaniem, w którym należy uwzględnić wiele skomplikowanych metod i technik. Opracowa-

nie odpowiedniej architektury silnika, uwzględniającej zarówno efektywność, jak i dokładność, jest kluczowe.

- Trenowanie modelu wymaga precyzyjnego dostrojenia funkcji nagrody, co jest decydującym elementem procesu uczenia. Właściwy dobór metod optymalizacji tej funkcji jest kluczowy dla uzyskania pożądanych rezultatów.
- Wybór odpowiedniej metody i modelu RL ma duży wpływ na wyniki. Algorytm PPO okazał się skuteczny, ale warto rozważyć testowanie innych algorytmów w przyszłości, aby sprawdzić, czy oferują one lepszą efektywność.

4.3 Perspektywy rozwoju i dalsze badania

Praca otwiera kilka perspektyw dla przyszłych badań i rozwoju:

- **Ulepszenie silnika fizycznego** – w celu uzyskania bardziej precyzyjnych symulacji, konieczne jest dodanie stabilizacji numerycznej, zaimplementowanie metod Computational Fluid Dynamics (CFD), a także poprawienie implementacji funkcji fizycznych, co przyczyni się do zwiększenia dokładności modelu.
- **Testowanie symulacji** – interesującym kierunkiem rozwoju jest porównanie wyników symulacji z rzeczywistymi testami, na przykład poprzez przeprowadzenie symulacji ruchu obiektów (np. rzut jabłka) w silniku i porównanie wyników z rzeczywistym filmem tego samego zjawiska. Tego typu testy pozwolą zweryfikować poprawność modelu.
- **Ulepszenie modelu trenującego** – poprawa funkcji nagrody oraz dalsze fine-tuningowanie algorytmu sterującego są niezbędne, aby uzyskać jeszcze lepsze wyniki. Możliwe jest również przetestowanie alternatywnych algorytmów uczenia maszynowego, co mogłoby poprawić efektywność systemu.
- **Rozwój metody symulacji** – warto kontynuować badania nad zastosowaniem zaawansowanych technik modelowania i optymalizacji procesów sterowania, które umożliwią rozszerzenie zastosowań systemu, np. w kontekście symulacji w bardziej złożonych warunkach.

Podsumowując, zaprezentowane w pracy rozwiązanie stanowi autorskie podejście do rozwoju autonomicznych systemów sterowania rakietami. Połączenie precyzyjnego modelu fizycznego z nowoczesnymi technikami uczenia maszynowego pozwoliło na stworzenie funkcjonalnego systemu, który wykazuje dużą skuteczność w symulacji i kontrolowaniu lotu rakiety. Uzyskane wyniki potwierdzają potencjał tego

podejścia w praktycznych zastosowaniach w inżynierii kosmicznej. Mimo pewnych uproszczeń, opracowane rozwiązanie stanowi solidną podstawę do dalszych badań, które mogą prowadzić do znaczących postępów w autonomicznym sterowaniu rakietami i rozwijaniu technologii kosmicznych. Dalszy rozwój tego systemu uwzględnia wyzwania związane z dokładnością modelu, optymalizacją algorytmów oraz adaptacją do rzeczywistych warunków.

Bibliografia

- [1] Martín Abadi i in. “TensorFlow: A system for large-scale machine learning”. W: *CoRR* abs/1605.08695 (2016). arXiv: 1605.08695. URL: <http://arxiv.org/abs/1605.08695>.
- [2] AeroTech i Madcow Rocketry. *RockSim*. Available at <https://www.rocketryforum.com/>. 2024. URL: <https://www.rocketryforum.com/>.
- [3] Inc. ANSYS. *ANSYS Fluent: Computational Fluid Dynamics (CFD) Software*. Version 2020, <https://www.ansys.com/products/fluids/ansys-fluent>. ANSYS, Inc., 2020. URL: <https://www.ansys.com/products/fluids/ansys-fluent>.
- [4] Scott Chacon i Ben Straub. *Pro Git*. Accessed: 2024-12-29. Apress, 2014. URL: <https://git-scm.com/book/en/v2>.
- [5] Leszek Chmielewski. *SGGW Thesis LaTeX Template*. <https://github.com/lchmiel/SGGW-Thesis-LaTeX/tree/master>. Accessed: 2024-12-30. 2024.
- [6] Kamil Ciosek i Shimon Whiteson. *Expected Policy Gradients for Reinforcement Learning*. 2020. arXiv: 1801.03326 [stat.ML]. URL: <https://arxiv.org/abs/1801.03326>.
- [7] ROS Consortium. *Gazebo ROS Packages*. https://github.com/ros-simulation/gazebo_ros_pkgs. Accessed: 2024-12-30. 2024.
- [8] Open Source Robotics Foundation. *Gazebo: Robot Simulation with ROS*. <http://gazebosim.org/>. Accessed: 2024-12-30. 2024.
- [9] Python Software Foundation. *Python 3 Documentation*. <https://docs.python.org/3/>. Accessed: 2024-12-29. 2024.
- [10] The OpenFOAM Foundation. *OpenFOAM 2312 Documentation*. Available at <https://doc.openfoam.com/2312/>. 2023. URL: <https://doc.openfoam.com/2312/>.
- [11] The OpenFOAM Foundation. *OpenFOAM: The Open Source Computational Fluid Dynamics (CFD) Toolbox*. Version 7, <https://www.openfoam.com>. 2020. URL: <https://www.openfoam.com>.
- [12] Aurélien Géron. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems, 3rd Edition*. 3rd. Sebastopol, CA, USA: O’Reilly Media, 2022.

- [13] GitHub. *GitHub - Platform for Open Source and Collaboration*. Accessed: 2024-12-29. 2024. URL: <https://github.com/>.
- [14] Minghao Han i in. *Actor-Critic Reinforcement Learning for Control with Stability Guarantee*. 2020. arXiv: 2004.14288 [cs.R0]. URL: <https://arxiv.org/abs/2004.14288>.
- [15] JetBrains. *PyCharm: The Python IDE for Professional Developers*. <https://www.jetbrains.com/pycharm/>. Accessed: 2024-12-29. 2024.
- [16] Gordon K. Mandell, George J. Caporaso i William P. Bengen. *Topics in advanced model rocketry [by] Gordon K. Mandell, George J. Caporaso [and] William P. Bengen*. eng. Cambridge, Mass: MIT Press, 1973. ISBN: 0262020963.
- [17] MathWorks. *Simulink: Model-Based Design*. Version R2019b, <https://www.mathworks.com/products/simulink.html>. MathWorks, 2019. URL: <https://www.mathworks.com/products/simulink.html>.
- [18] Microsoft. *AirSim: A Simulator for Autonomous Vehicles*. <https://github.com/microsoft/AirSim>. Accessed: 2024-12-30. 2024.
- [19] Frank Mittelbach i in. *The LaTeX Companion*. 2nd. Boston: Addison-Wesley, 2004.
- [20] Sampo Niskanen. “OpenRocket technical documentation”. W: *Development of an Open Source model rocket simulation software* (2013), s. 11–13.
- [21] Adam Paszke i in. “PyTorch: An Imperative Style, High-Performance Deep Learning Library”. W: *CoRR* abs/1912.01703 (2019). arXiv: 1912.01703. URL: <http://arxiv.org/abs/1912.01703>.
- [22] Dhariwal Rafael i in. *Stable-Baselines3: Reliable Reinforcement Learning Algorithms*. <https://github.com/DLR-RM/stable-baselines3>. Accessed: 2024-12-30. 2020.
- [23] Inc. RAS. *RASaero*. Available at <https://www.rasaero.com/>. 2024. URL: <https://www.rasaero.com/>.
- [24] John Schulman i in. *Proximal Policy Optimization Algorithms*. 2017. arXiv: 1707.06347 [cs.LG]. URL: <https://arxiv.org/abs/1707.06347>.
- [25] Richard S. Sutton i Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd. Cambridge, MA, USA: MIT Press, 2018.
- [26] The OpenRocket Team. *OpenRocket*. Available at <https://openrocket.info/>. 2024. URL: <https://openrocket.info/>.
- [27] Inc. The MathWorks. *Simulink Aerospace Blockset*. <https://www.mathworks.com/products/aerospace-blockset.html>. Accessed: 2024-12-30. 2024.

Wyrażam zgodę na udostępnienie mojej pracy w czytelniach Biblioteki SGGW
w tym w Archiwum Prac Dyplomowych SGGW.

.....
(czytelny podpis autora pracy)

